



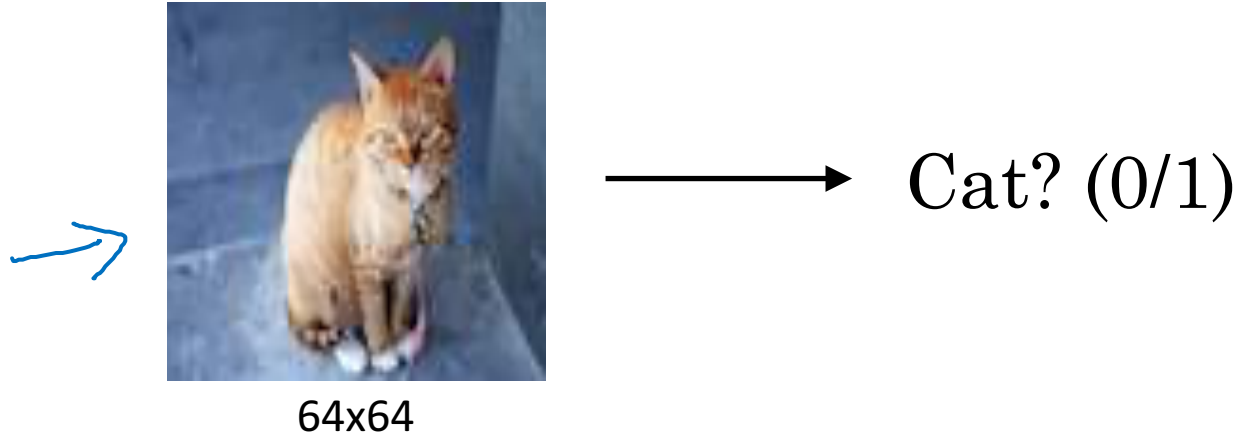
deeplearning.ai

Convolutional Neural Networks

Computer vision

Computer Vision Problems

Image Classification



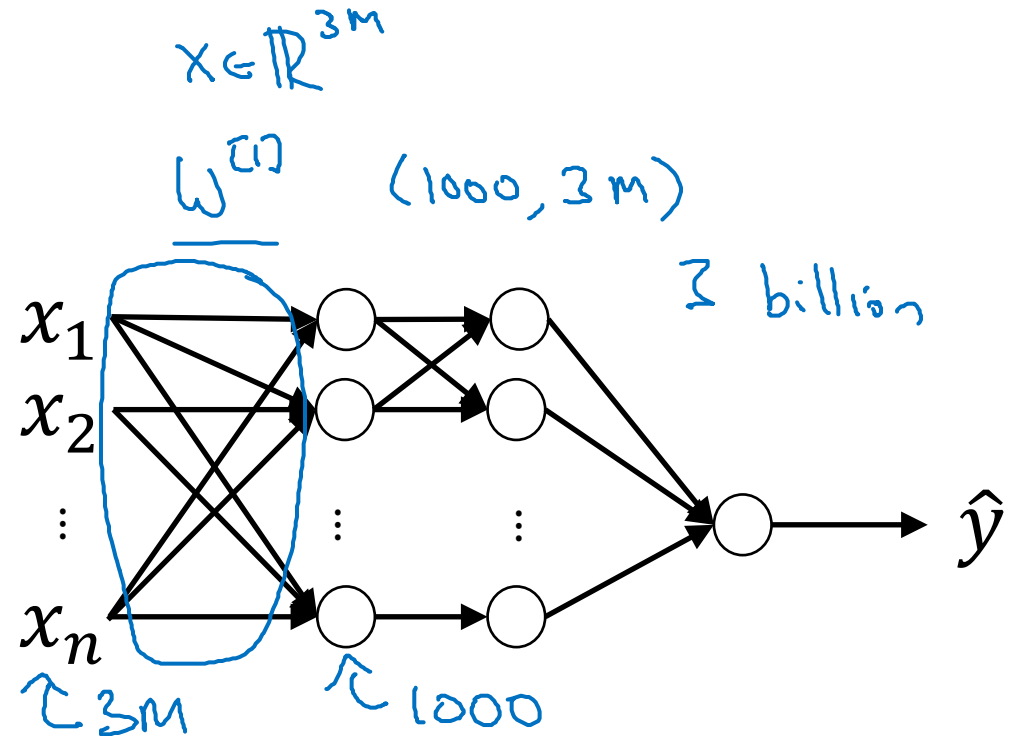
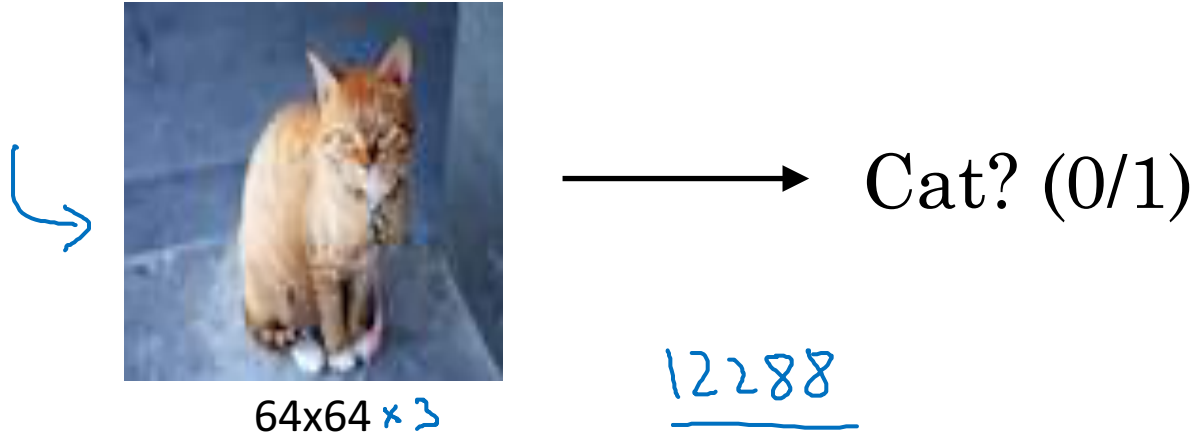
Neural Style Transfer



Object detection



Deep Learning on large images



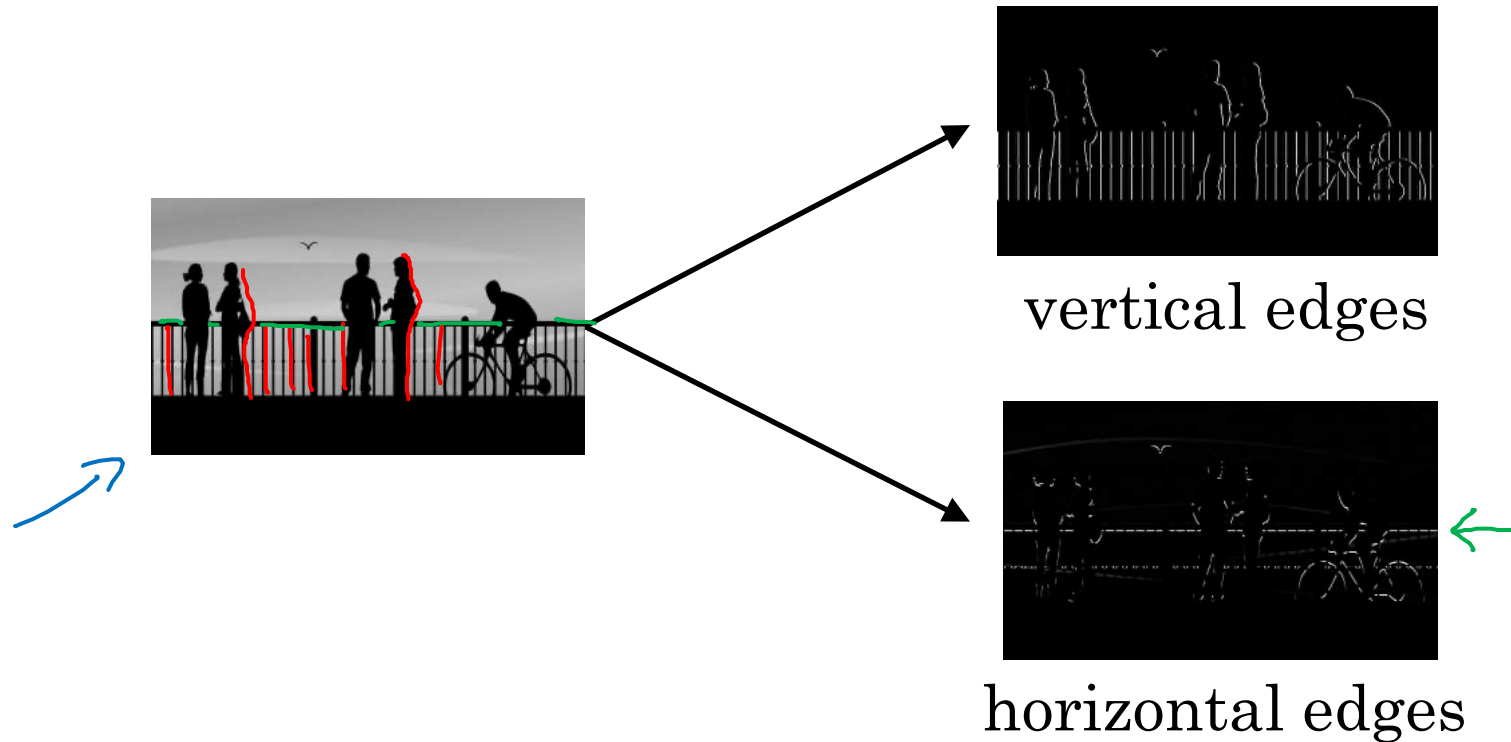
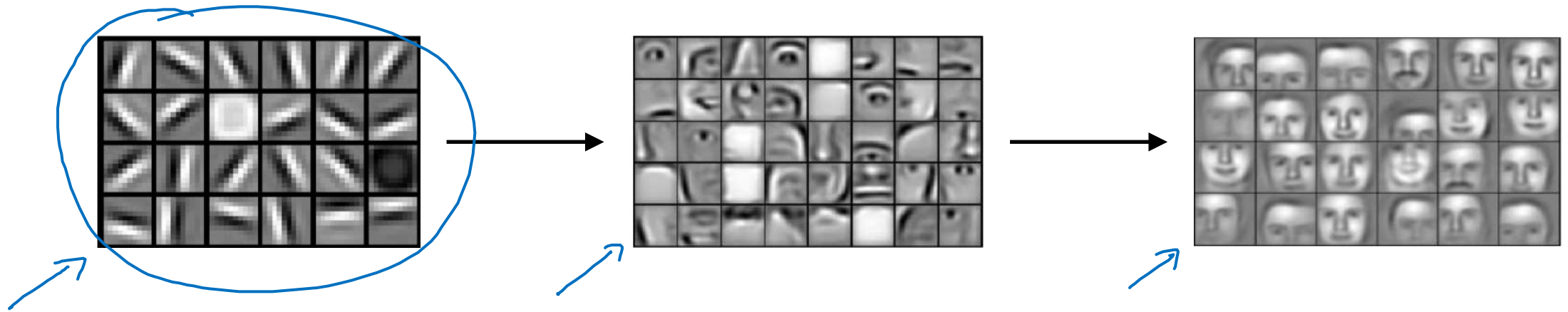


deeplearning.ai

Convolutional Neural Networks

Edge detection
example

Computer Vision Problem

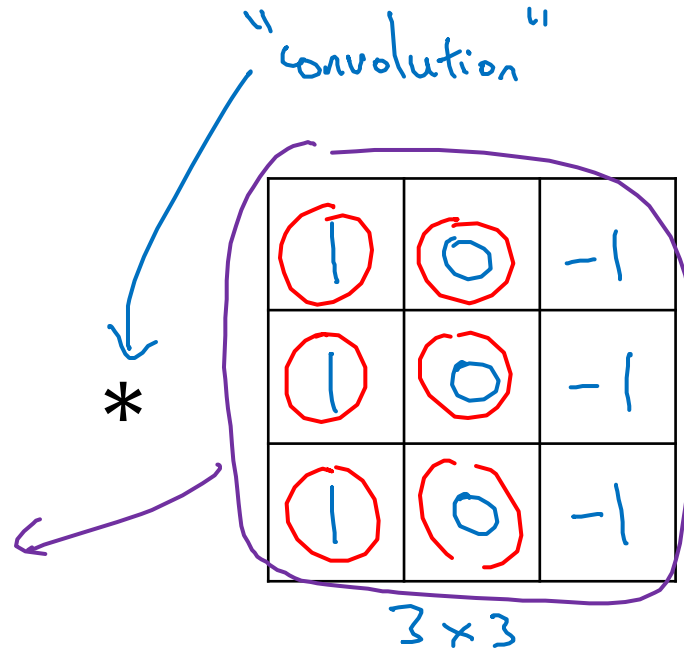


Vertical edge detection

$$\rightarrow 3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3 ¹	0 ¹	1 ⁻¹	2 ⁻¹	7 ⁻⁰	4 ⁻¹
1 ¹	5 ¹	8 ⁻¹	9 ⁻¹	3 ⁻⁰	1 ⁻¹
2 ¹	7 ¹	2 ⁻¹	5 ⁻¹	1 ⁻⁰	3 ⁻¹
0 ¹	1 ¹	3 ⁻¹	1 ⁻¹	7 ⁻⁰	8 ⁻¹
4	2	1	6	2	8
2	4	5	2	3	9

6x6



=

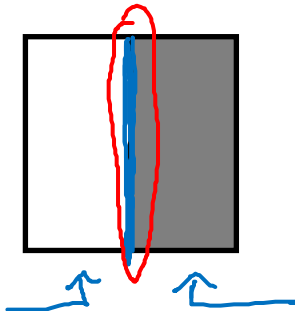
-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4x4

Vertical edge detection

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	<u>10</u>	<u>10</u>	<u>0</u>	0	0
10	<u>10</u>	<u>10</u>	<u>0</u>	0	0
10	<u>10</u>	<u>10</u>	<u>0</u>	0	0

6x6

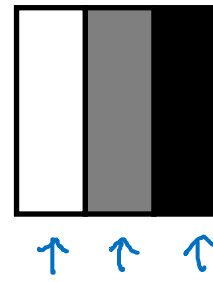


*

1	0	-1
1	0	-1
1	0	-1

3x3

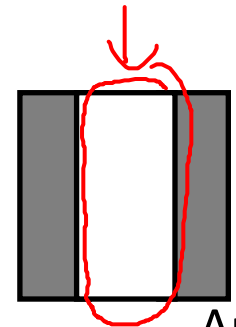
*



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4x4





deeplearning.ai

Convolutional Neural Networks

More edge
detection

Vertical edge detection examples

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10



*

1	0	-1
1	0	-1
1	0	-1



=

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0



Vertical and Horizontal Edge Detection



1	0	-1
1	0	-1
1	0	-1

Vertical



1	1	1
0	0	0
-1	-1	-1

Horizontal

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

6x6

*



1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

Learning to detect edges

1	0	-1
1	0	-1
1	0	-1



1	0	-1
2	0	-2
1	0	-1



Sobel filter

3	0	-3
10	0	-10
3	0	-3

Scharr filter



3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

convolution
×

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

3x3

=

45°
70°
73°





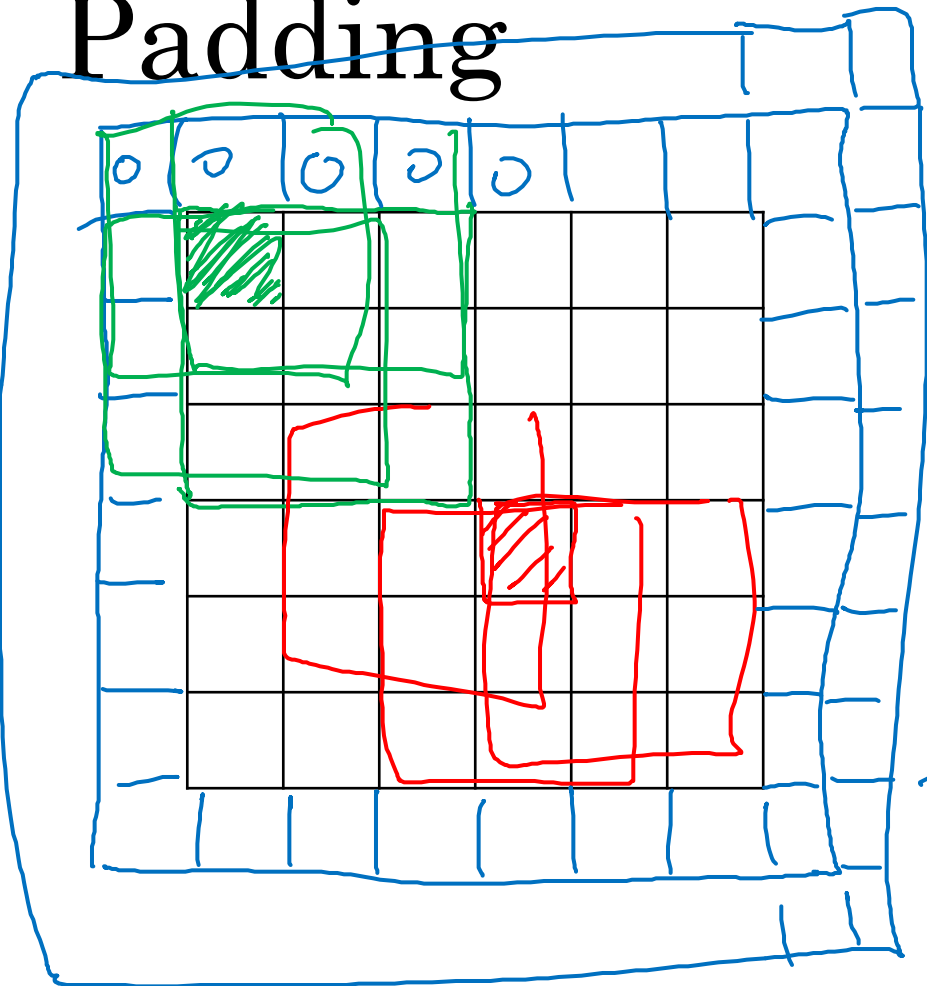
deeplearning.ai

Convolutional Neural Networks

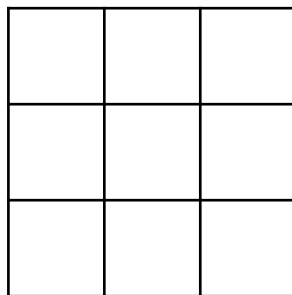
Padding

Padding

- shrinky output
- throw away info from edge

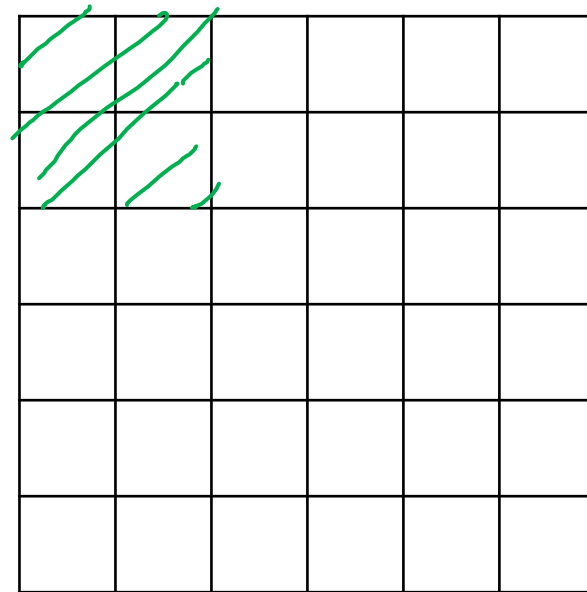


*



3x3
f x f

=



6x6

6x6 → 8x8
n x n

$n - f + 1 \times n - f + 1$
 $6 - 3 + 1 = 4$

$p = \text{padding} = \underline{1}$

$n + 2p - f + 1 \times n + 2p - f + 1$
 $6 + 2 - 3 + 1 \times \underline{\underline{4}} = 6 \times 6$

Valid and Same convolutions

→ no padding

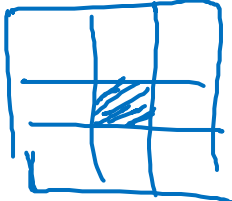
“Valid”: $n \times n$ $\times$ $f \times f$ $\rightarrow \frac{n-f+1}{1} \times n-f+1$
 6×6 $\times$ 3×3 $\rightarrow 4 \times 4$

“Same”: Pad so that output size is the same as the input size.

$$n+2p-f+1 \times n+2p-f+1$$
$$\cancel{n+2p-f+1} = \cancel{n} \Rightarrow \boxed{p = \frac{f-1}{2}}$$
$$3 \times 3 \quad p = \frac{3-1}{2} = 1 \quad \left| \begin{array}{l} 5 \times 5 \\ f=5 \end{array} \right.$$

f is usually odd

1x1
3x3
5x5
7x7



$p=2$

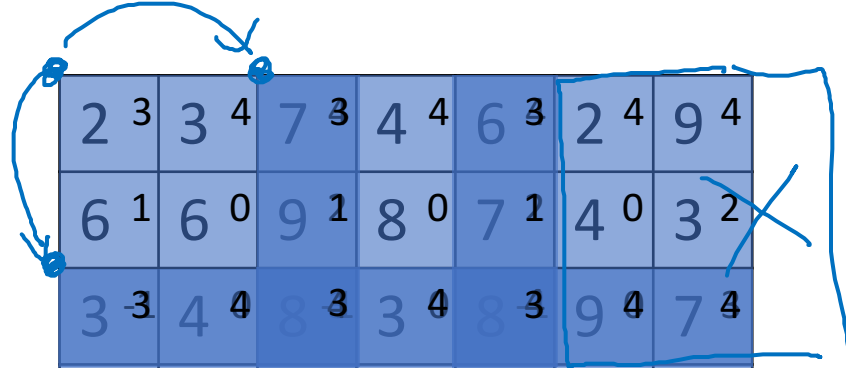


deeplearning.ai

Convolutional Neural Networks

Strided convolutions

Strided convolution



2	3	3	4	7	3	4	4	6	3	2	4	9	4
6	1	6	0	9	1	8	0	7	1	4	0	3	2
3	3	4	4	8	3	3	4	8	3	9	4	7	4
7	1	8	0	3	1	6	0	6	1	3	0	4	2
4	3	2	4	1	3	8	4	3	3	4	4	6	4
3	1	2	0	4	1	1	0	9	1	8	0	3	2
0	-1	1	0	3	-1	9	0	2	-1	1	0	4	3

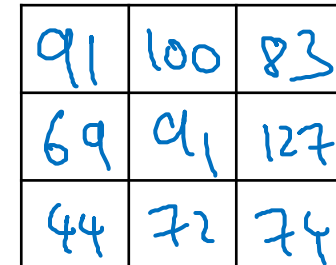
7x7

*

3	4	4
1	0	2
-1	0	3

3x3

=



91	100	83
69	91	127
44	72	74

3x3

Stride = 2

$\lfloor z \rfloor = \text{floor}(z)$

$n \times n$ * $f \times f$
 padding p stride s
 $s=2$

$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

$$\frac{7 + 0 - 3}{2} + 1 = \frac{4}{2} + 1 = 3$$

Summary of convolutions

$n \times n$ image $f \times f$ filter

padding p stride s

Output Size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \underbrace{\frac{n+2p-f}{s}} + 1 \right\rfloor$$

Technical note on cross-correlation vs. convolution

Convolution in math textbook:

2 ⁷	3 ²	7 ⁵	4	6	2
6 ⁹	6 ⁰	9 ⁴	8	7	4
3 ⁻¹	4 ¹	8 ³	3	8	9
7	8	3	6	6	3
4	2	1	8	3	4
3	2	4	1	9	8

3	4	5
1	0	2
-1	9	7

7	2	5
9	0	4
-1	1	3

$$(A * B) * C = A * (B * C)$$

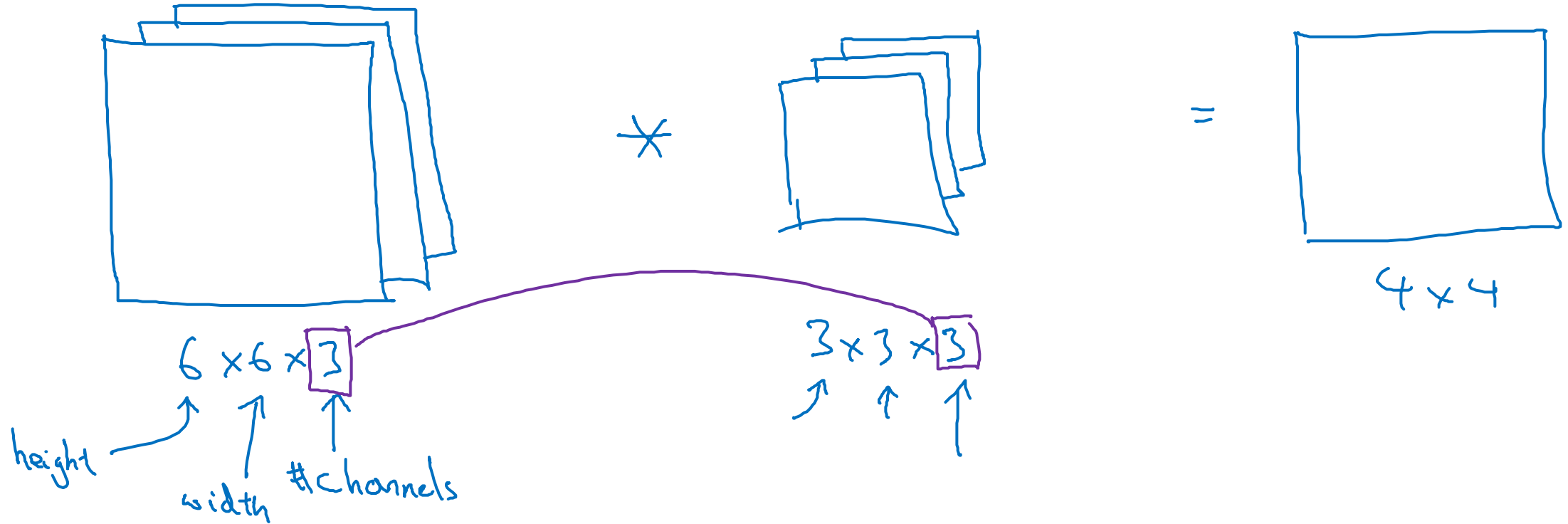


deeplearning.ai

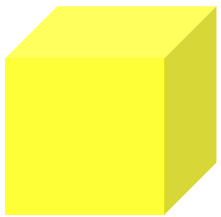
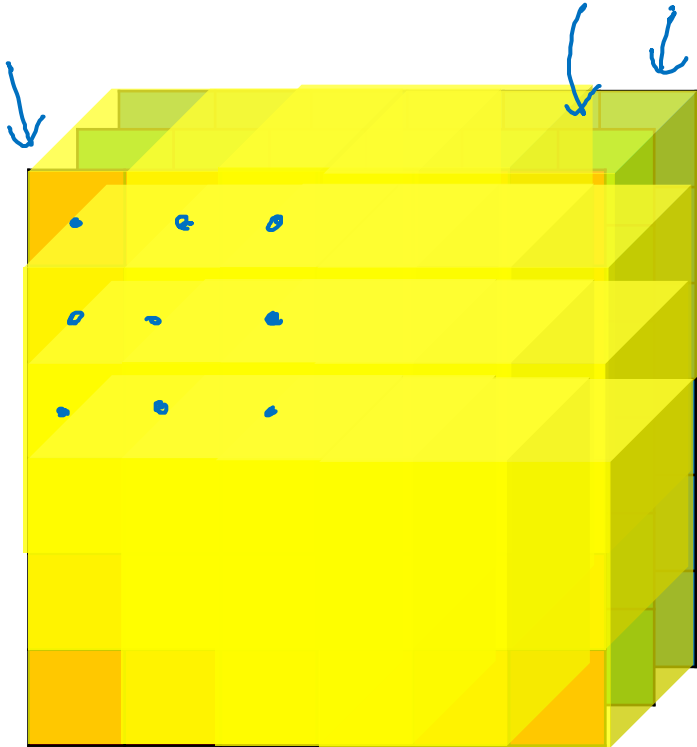
Convolutional Neural Networks

Convolutions over volumes

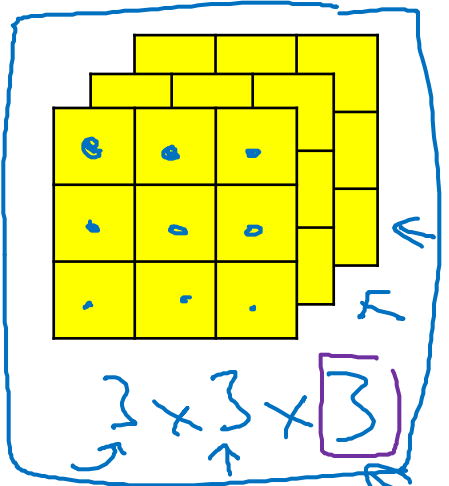
Convolutions on RGB images



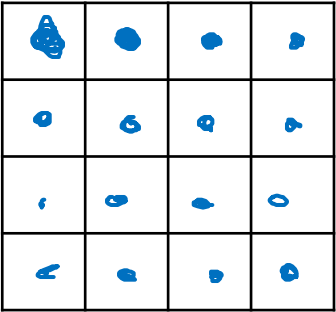
Convolutions on RGB image



*

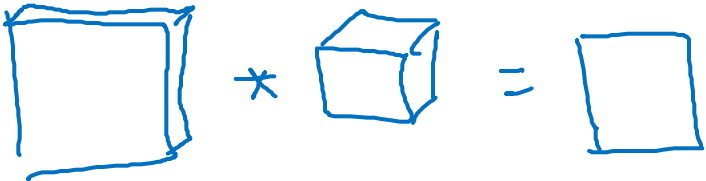


=

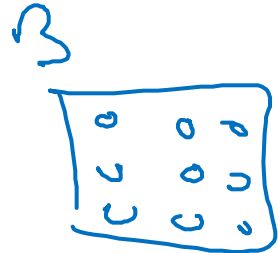
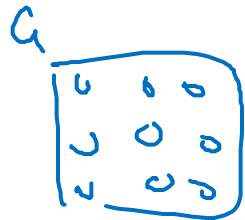
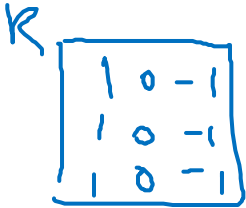


4 x 4

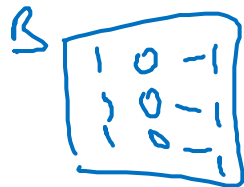
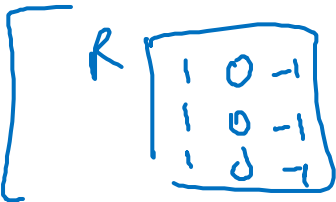
$6 \times 6 \times 3$
↑ ↑ ↑



27 numbers

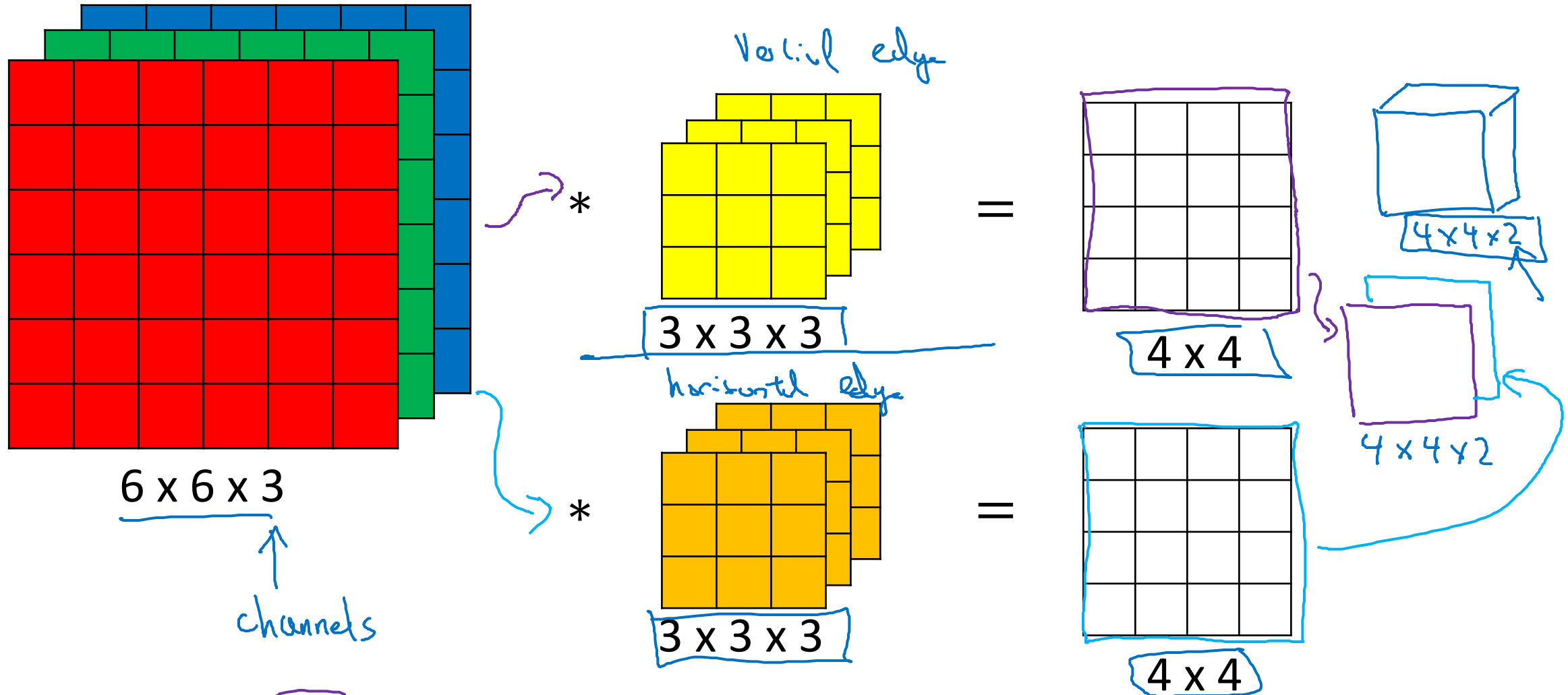


→ 3x3x3



→ 3x3x3

Multiple filters



Summary: $n \times n \times n_c$ $\times$ $f \times f \times n_c$ $\rightarrow$ $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times \frac{n_c}{2}$

$6 \times 6 \times 3$ $3 \times 3 \times 3$ $4 \times 4 \times 2$ # filters

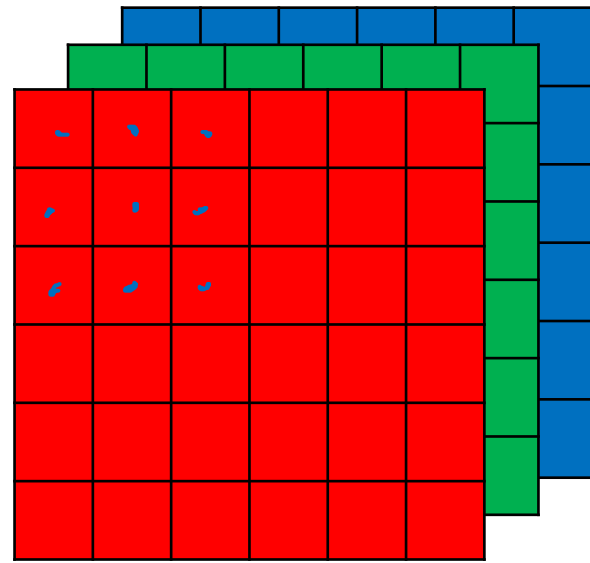


deeplearning.ai

Convolutional Neural Networks

One layer of a
convolutional
network

Example of a layer



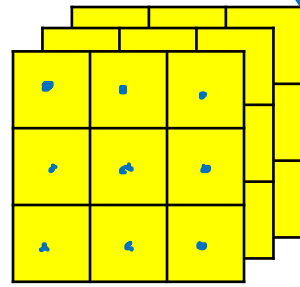
6 x 6 x 3

$a^{[0]}$

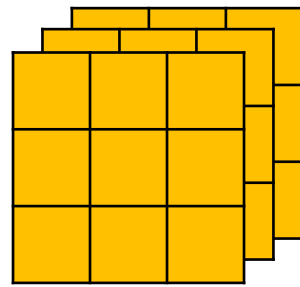
$$z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

*
*

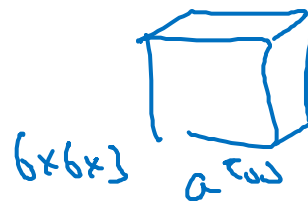


3 x 3 x 3



3 x 3 x 3

" $W^{[1]}$ "

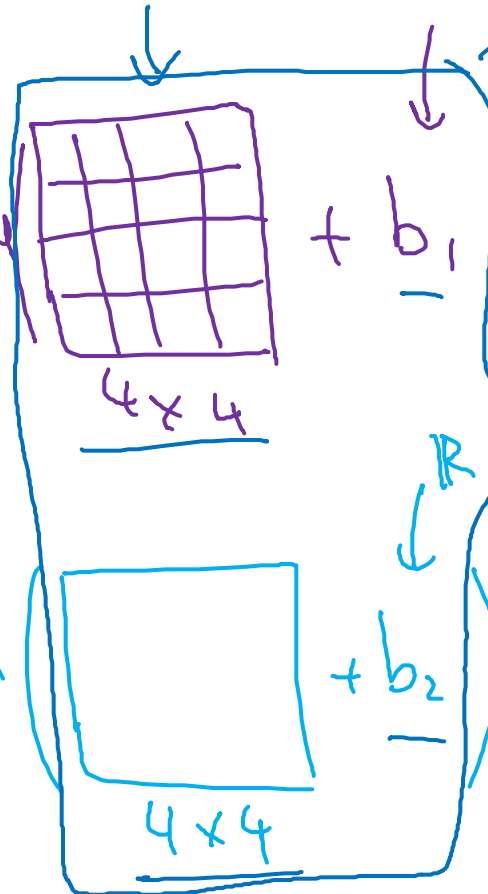


6 x 6 x 3

$a^{[0]}$

→ Relu

→ Relu



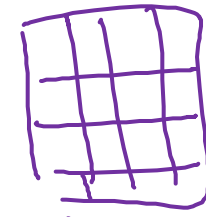
+ b₁

4 x 4

+ b₂

4 x 4

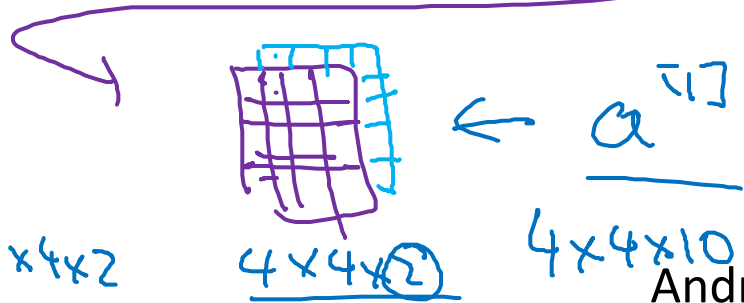
R



4 x 4



4 x 4



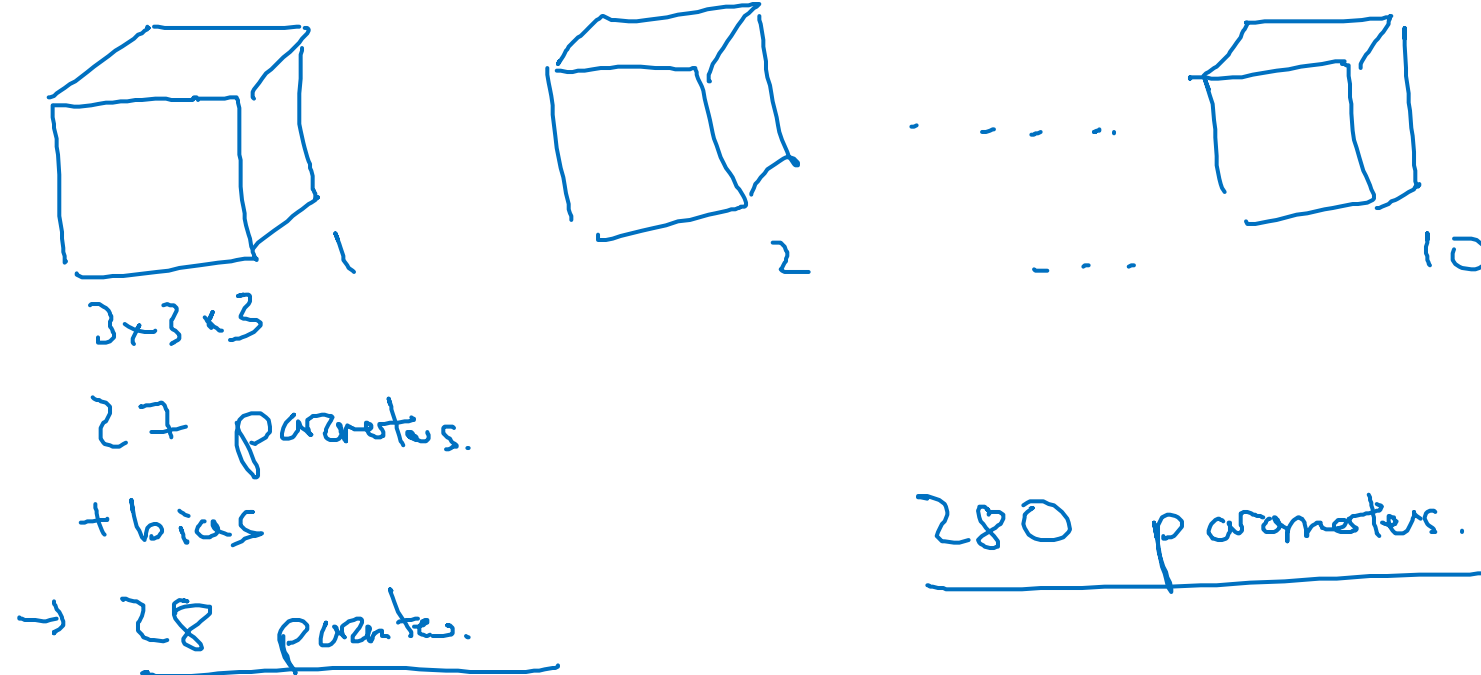
4 x 4 x 2

$a^{[1]}$

4 x 4 x 10

Number of parameters in one layer

If you have 10 filters that are $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?



Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters

→ Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$

Activations: $a^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$

bias: $n_c^{[l]} - (1, 1, 1, n_c^{[l]})$ ← #filters in layer l.

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$A^{[l]} \rightarrow m \times \underbrace{n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}}_{n_c^{[l]} \times n_H^{[l]} \times n_W^{[l]}}$$

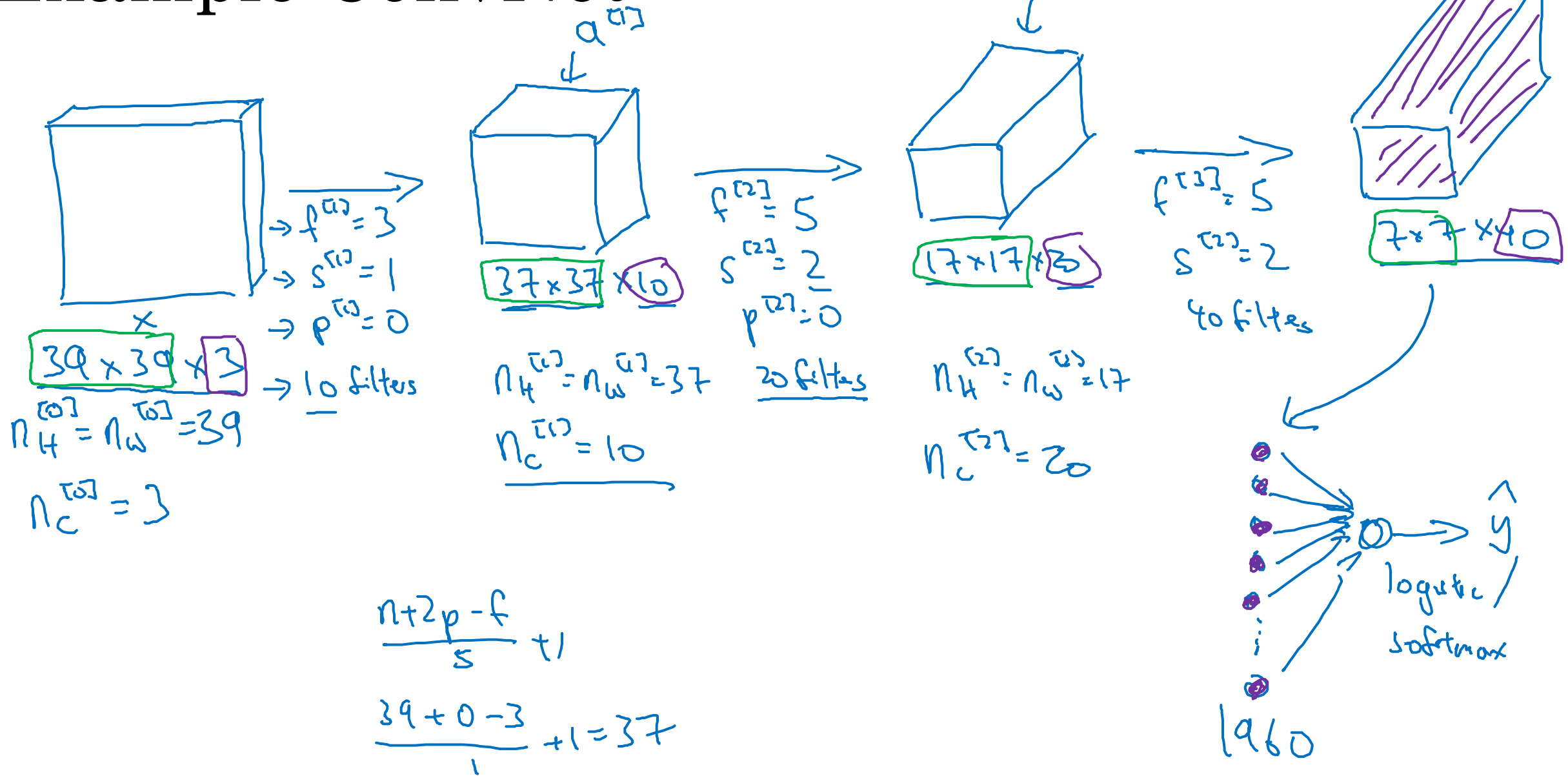


deeplearning.ai

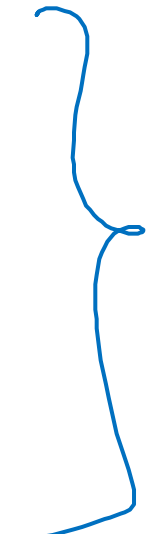
Convolutional Neural Networks

A simple convolution
network example

Example ConvNet



Types of layer in a convolutional network:

- Convolution (conv) ←
 - Pooling (pool) ←
 - Fully connected (Fc) ←
- 



deeplearning.ai

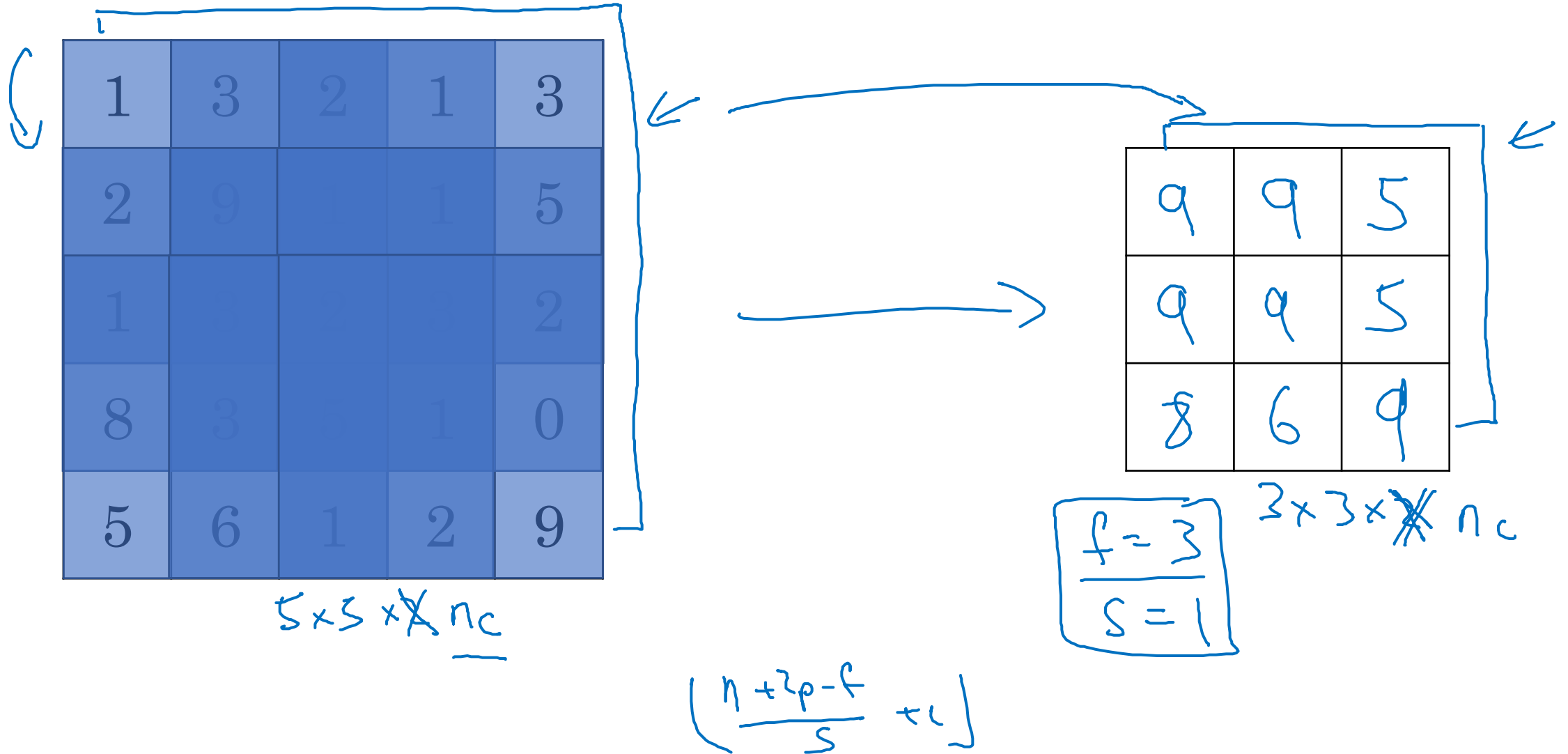
Convolutional Neural Networks

Pooling layers

Pooling layer: Max pooling

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

Pooling layer: Max pooling



Pooling layer: Average pooling

1	3	2	1
2	9	1	1
1	4	2	3
5	6	1	2



3.75	1.25
4	2

$$f=2$$

$$s=2$$

$$\underline{7 \times 7 \times 1000} \rightarrow 1 \times 1 \times 1000$$

Summary of pooling

Hyperparameters:

f : filter size

$$f=2, s=2$$

s : stride

$$f=3, s=2$$

Max or average pooling

~~⇒ p: padding.~~

No parameters to learn!

$$n_H \times n_W \times \underline{n_C}$$



$$\left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \times \underline{n_C}$$

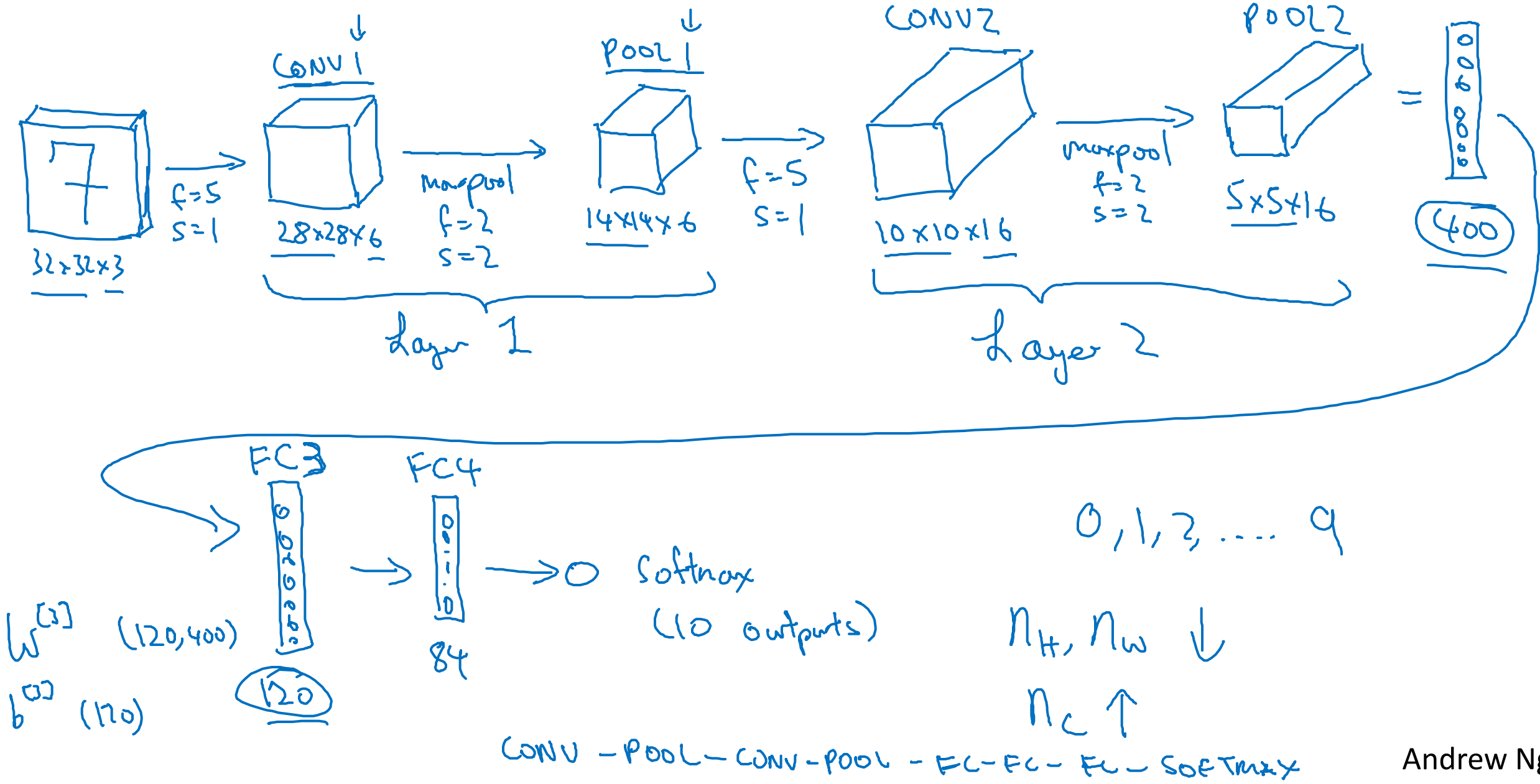


deeplearning.ai

Convolutional Neural Networks

Convolutional neural network example

Neural network example (LeNet-5)



Neural network example

	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072 $a^{[0]}$	0
CONV1 (f=5, s=1)	(28,28,8)	<u>6,272</u>	608 ←
POOL1	(14,14,8)	<u>1,568</u>	0 ←
CONV2 (f=5, s=1)	(10,10,16)	<u>1,600</u>	3216 ←
POOL2	(5,5,16)	<u>400</u>	0 ←
FC3	(120,1)	<u>120</u>	48120 }
FC4	(84,1)	<u>84</u>	10164 }
Softmax	(10,1)	<u>10</u>	850

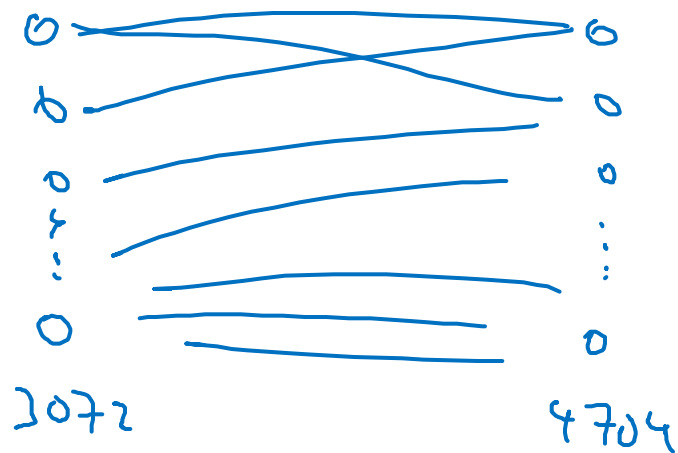
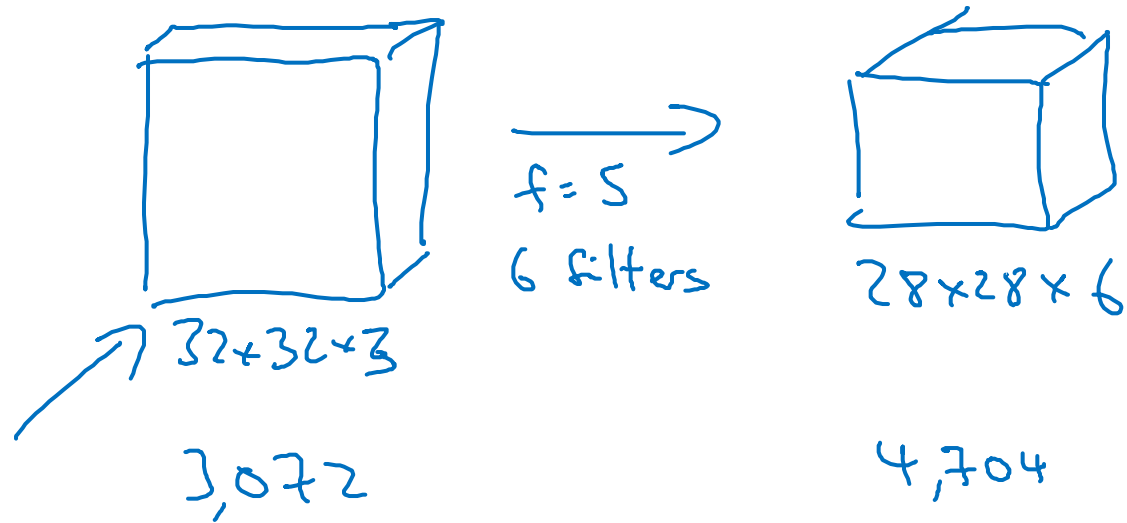


deeplearning.ai

Convolutional Neural Networks

Why convolutions?

Why convolutions



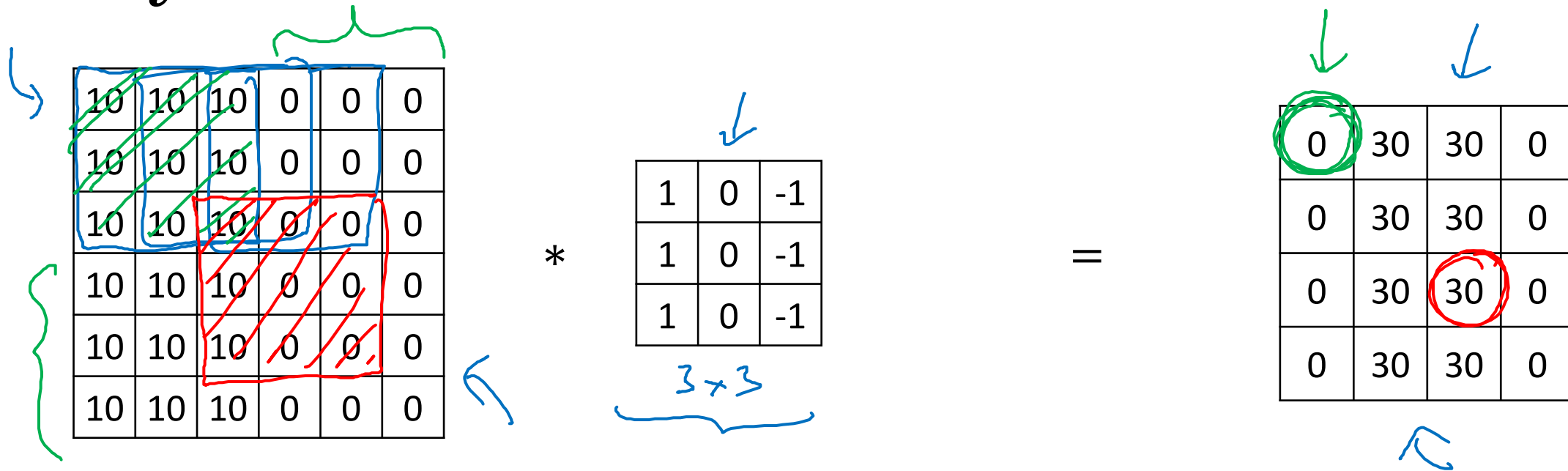
$$5 \times 5 = 25$$

$$26$$

$$6 \times 26 = 156 \text{ parameters}$$

$$3,072 \times 4,704 \approx \underline{14M}$$

Why convolutions

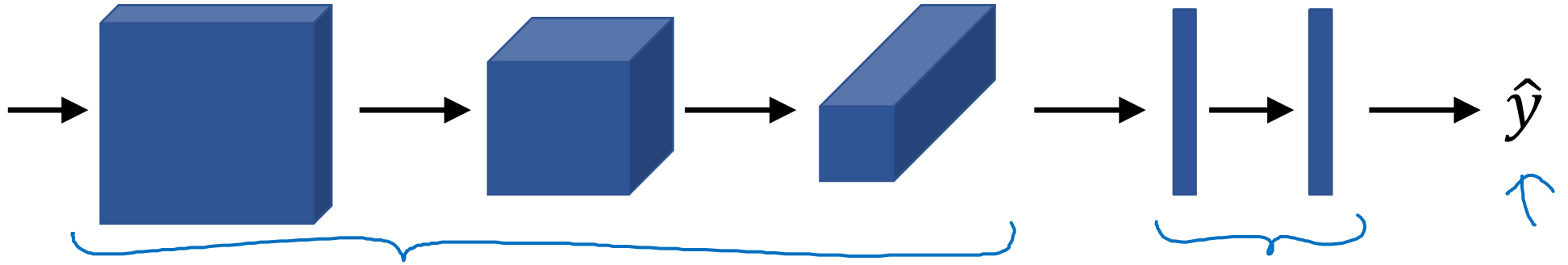


Parameter sharing: A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

Putting it together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J



deeplearning.ai

Case Studies

Why look at
case studies?

Outline

Classic networks:

- LeNet-5 ←
- AlexNet ←
- VGG ←

ResNet (152)

Inception

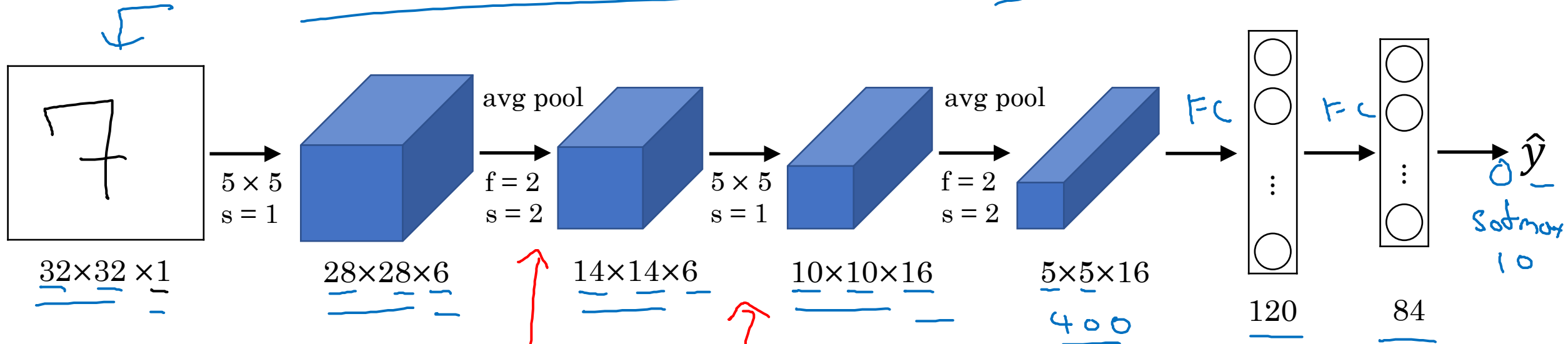


deeplearning.ai

Case Studies

Classic networks

LeNet - 5



60K parameters.

$n_H, n_W \downarrow$ $n_C \uparrow$

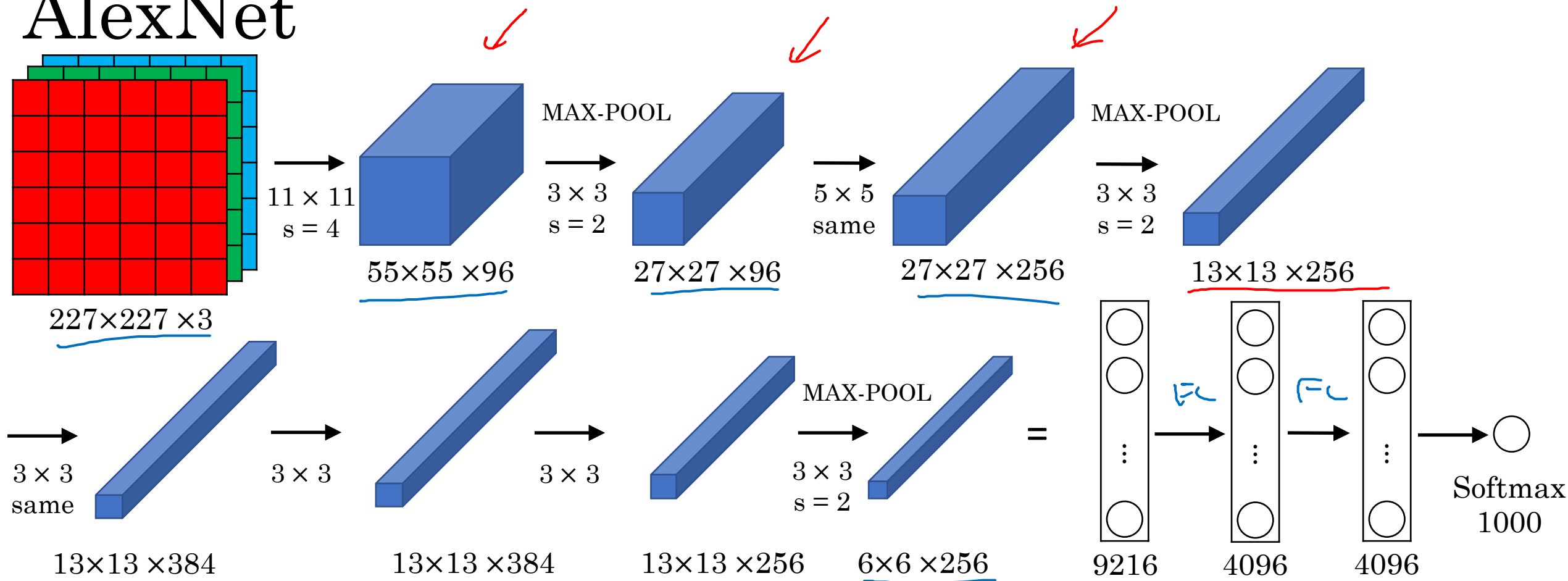
conv pool conv pool fc fc output

Advanced: sigmoid/tanh ReLU

II, III.

↓

AlexNet

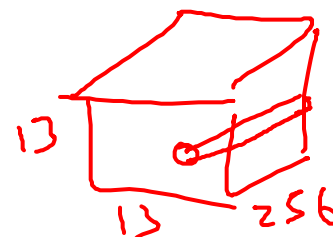


- Similar to LeNet, but much bigger.

- ReLU

- Multiple GPUs.

- Local Response Normalization (LRN)

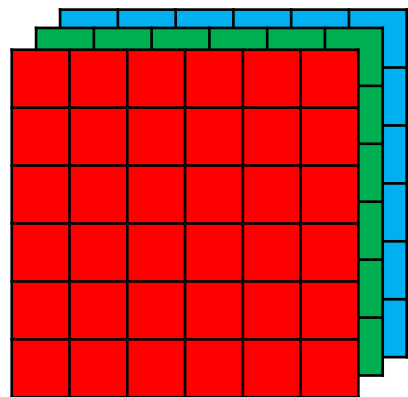


~60M parameters

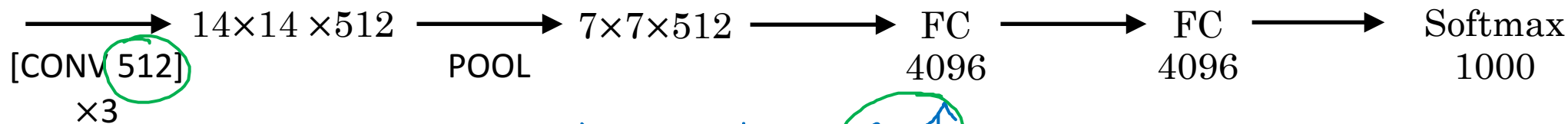
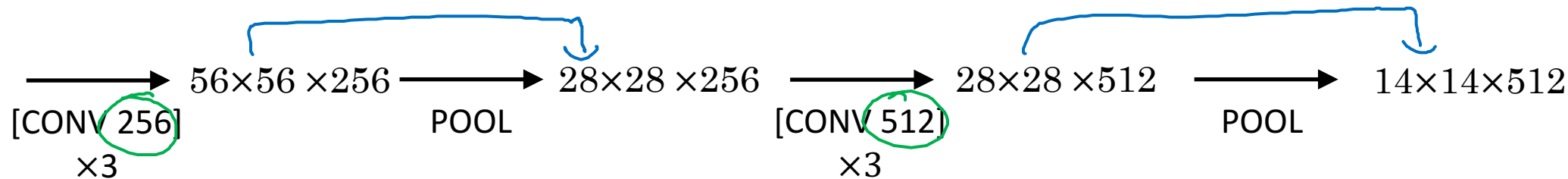
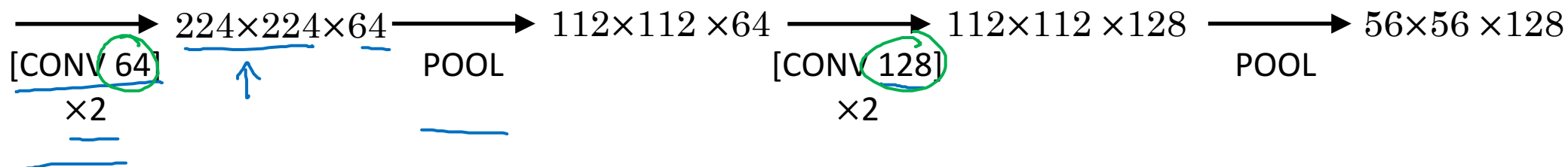
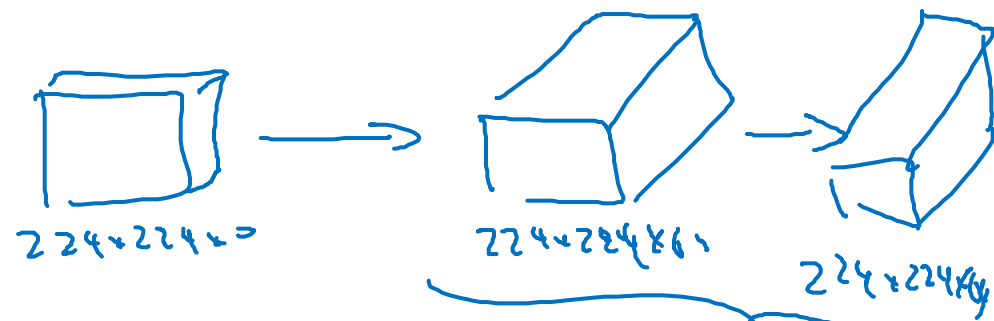
VGG - 16

CONV = 3x3 filter, s = 1, same

MAX-POOL = 2x2, s = 2



VGG-19



$n_h, n_w \downarrow$

$n_c \uparrow$

~138M

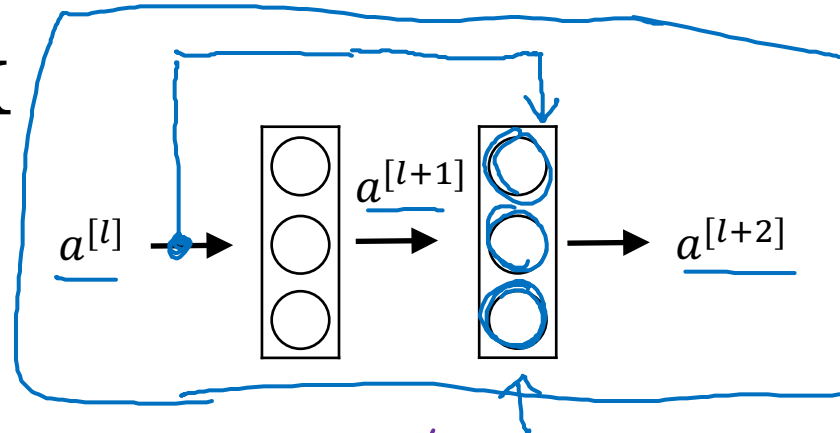


deeplearning.ai

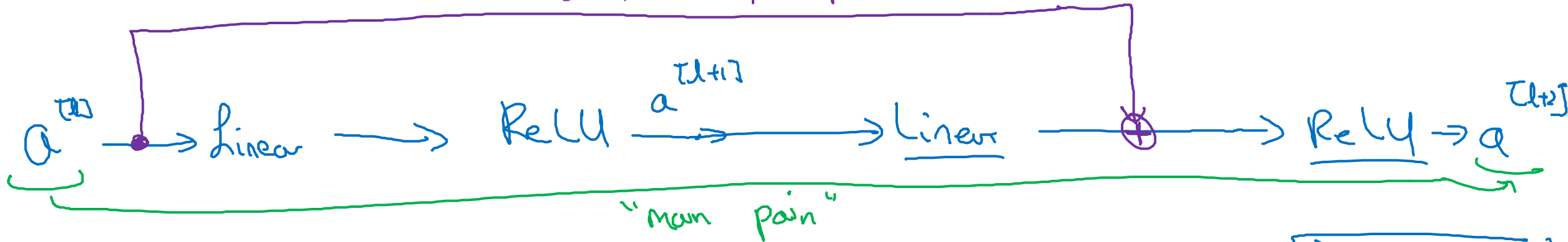
Case Studies

Residual Networks (ResNets)

Residual block



"short cut" / skip connection



$$\underline{z^{[l+1]}} = W^{[l+1]} \underline{a^{[l]}} + b^{[l+1]}$$

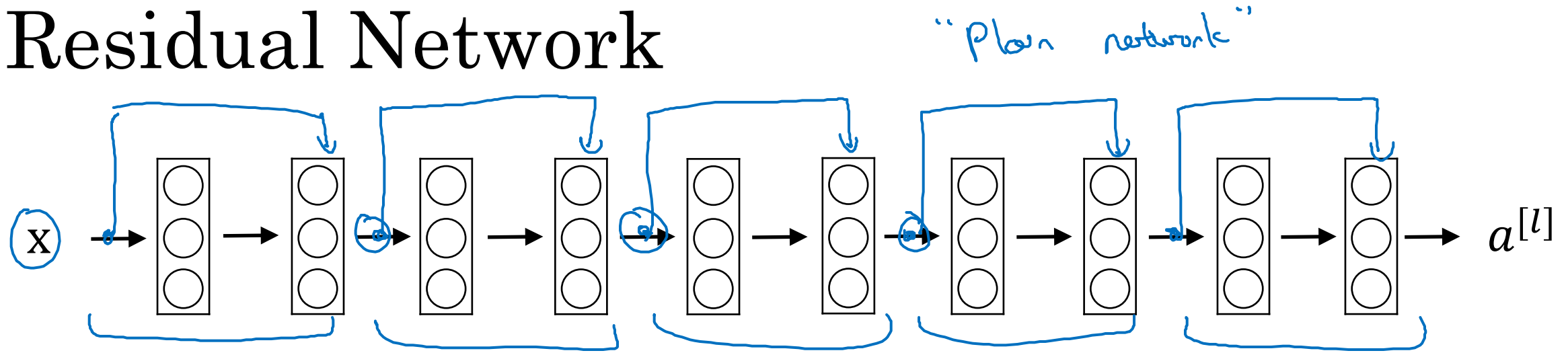
$$\underline{a^{[l+1]}} = g(\underline{z^{[l+1]}})$$

$$\underline{z^{[l+2]}} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

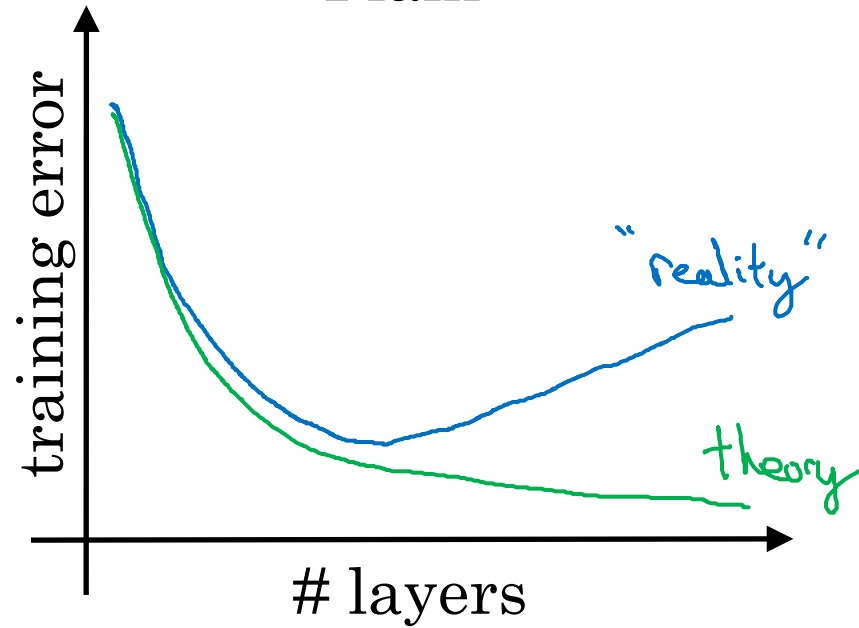
~~$$a^{[l+2]} = g(z^{[l+2]})$$~~

$$a^{[l+2]} = g(z^{[l+2]} + \underline{a^{[l]}})$$

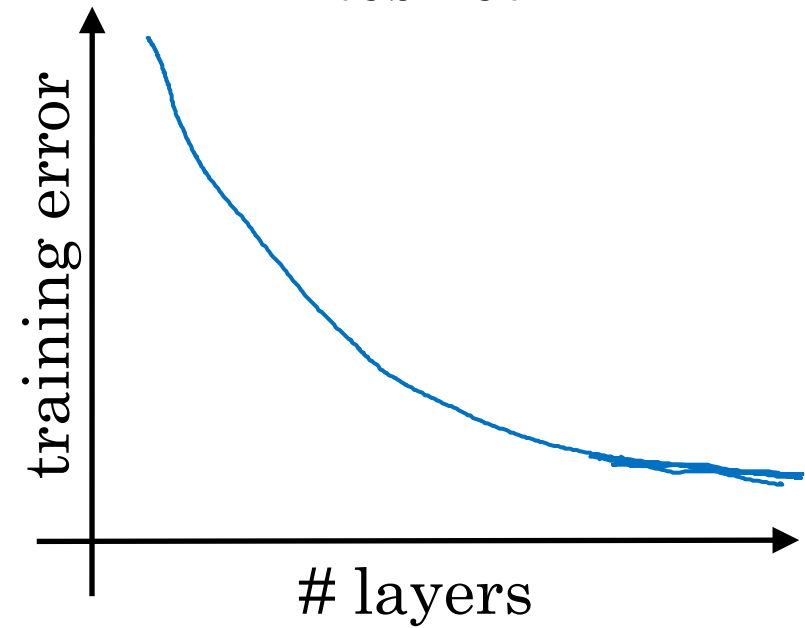
Residual Network



Plain



ResNet



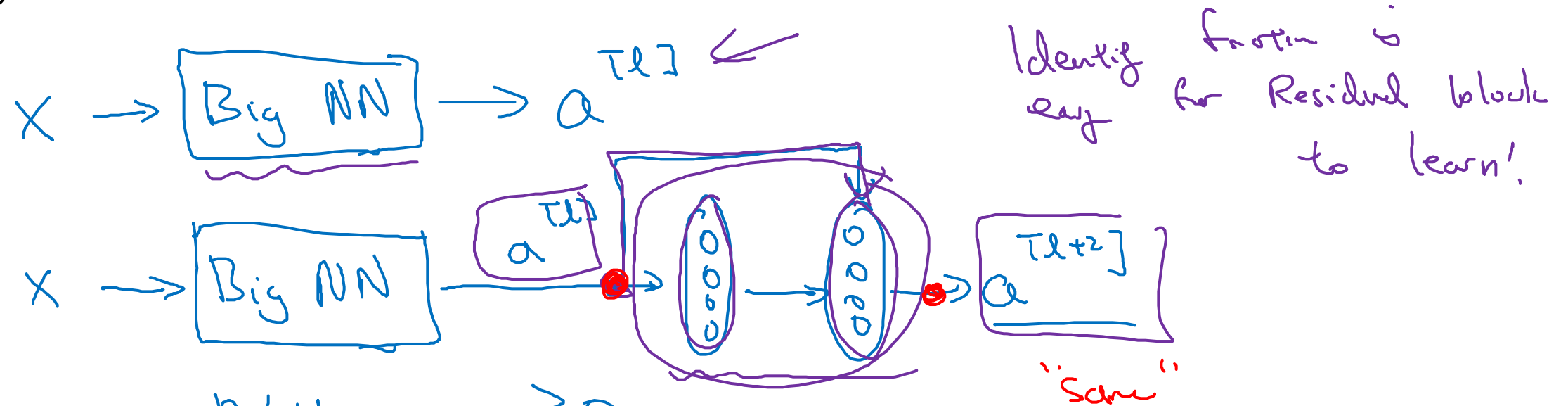


deeplearning.ai

Case Studies

Why ResNets work

Why do residual networks work?



ReLU. $a \geq 0$

$$a^{[L+2]} = g(\underbrace{z^{[L+2]} + a^{[L]}}_{\text{ReLU}})$$

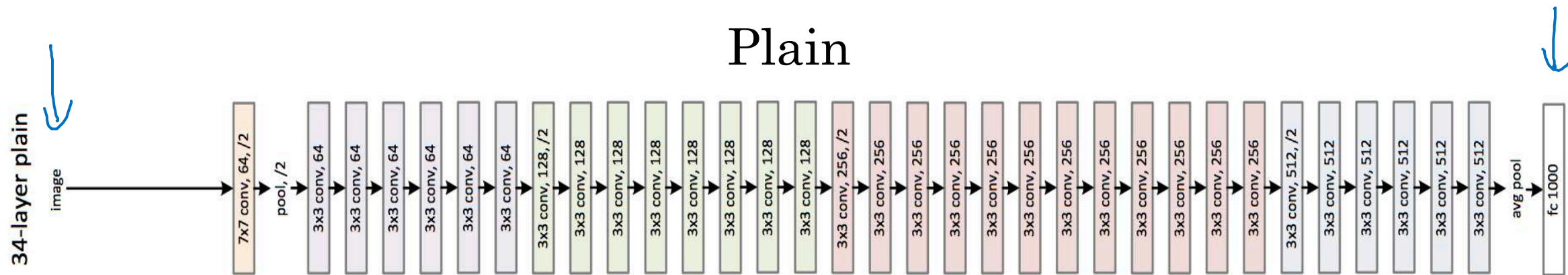
$$= g(\underbrace{W^{[L+2]} a^{[L]} + b^{[L+2]}}_{\text{If } W^{[L+2]}=0, b^{[L+2]}=0} + \underbrace{W_s a^{[L]}}_{128}) = g(a^{[L]}) = \underline{a^{[L]}}$$

256×128

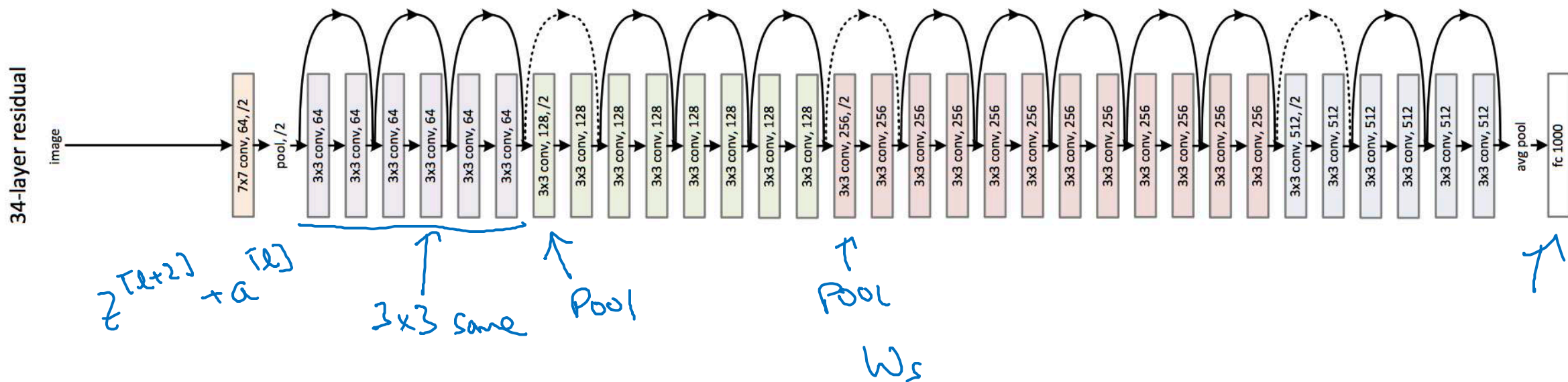
128

ResNet

Plain



ResNet





deeplearning.ai

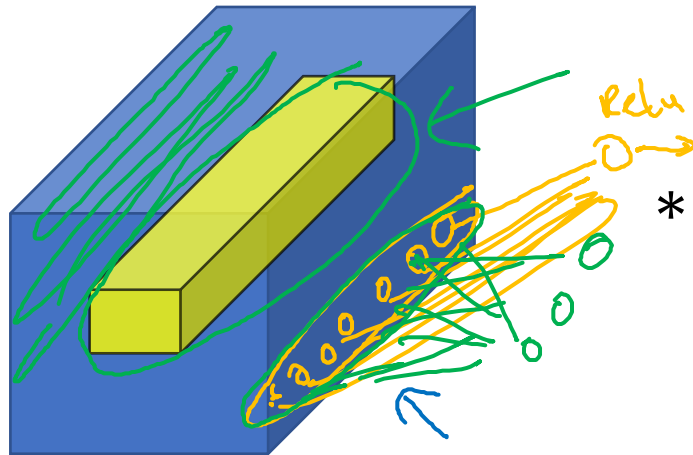
Case Studies

Network in Network
and 1×1 convolutions

Why does a 1×1 convolution do?

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

$6 \times 6 \times 1$



$6 \times 6 \times 32$

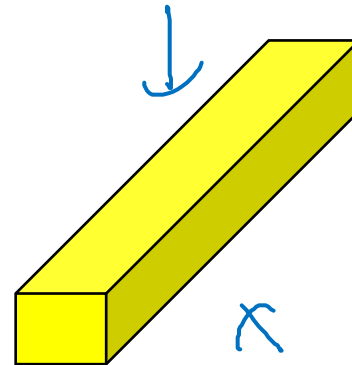
*

2

=

2	4	6	...		

32 $\rightarrow$ # filters.
 $n_c^{[l+1]}$

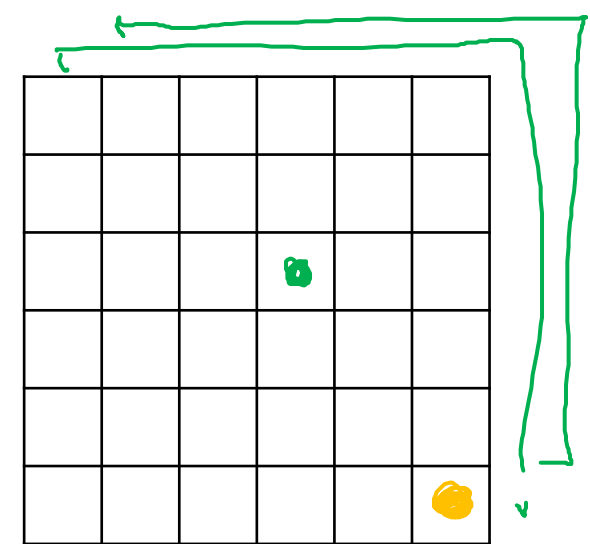


$1 \times 1 \times 32$

=

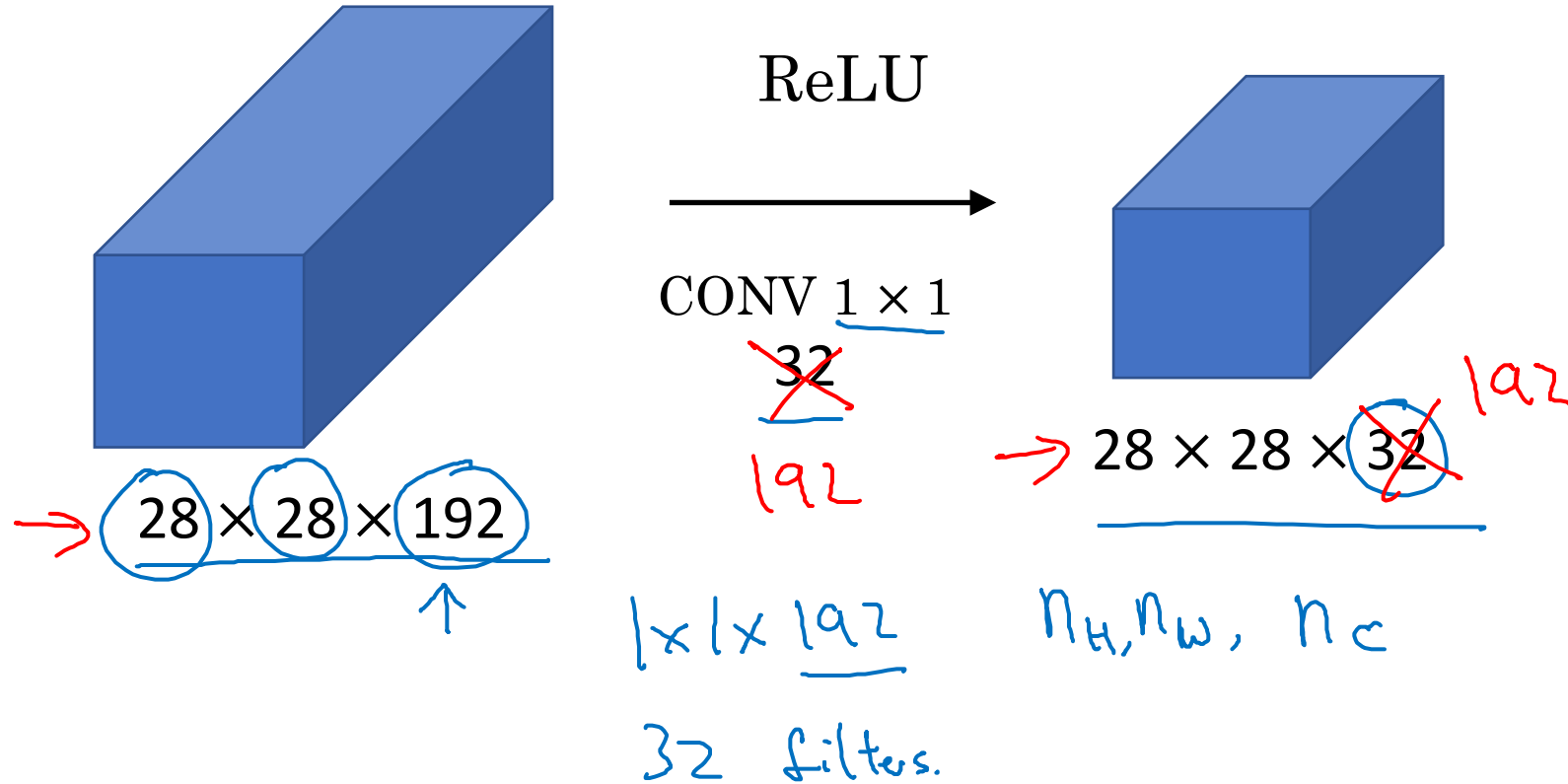
ReLU

Network in
Network



$6 \times 6 \times \# \text{ filters}$

Using 1×1 convolutions



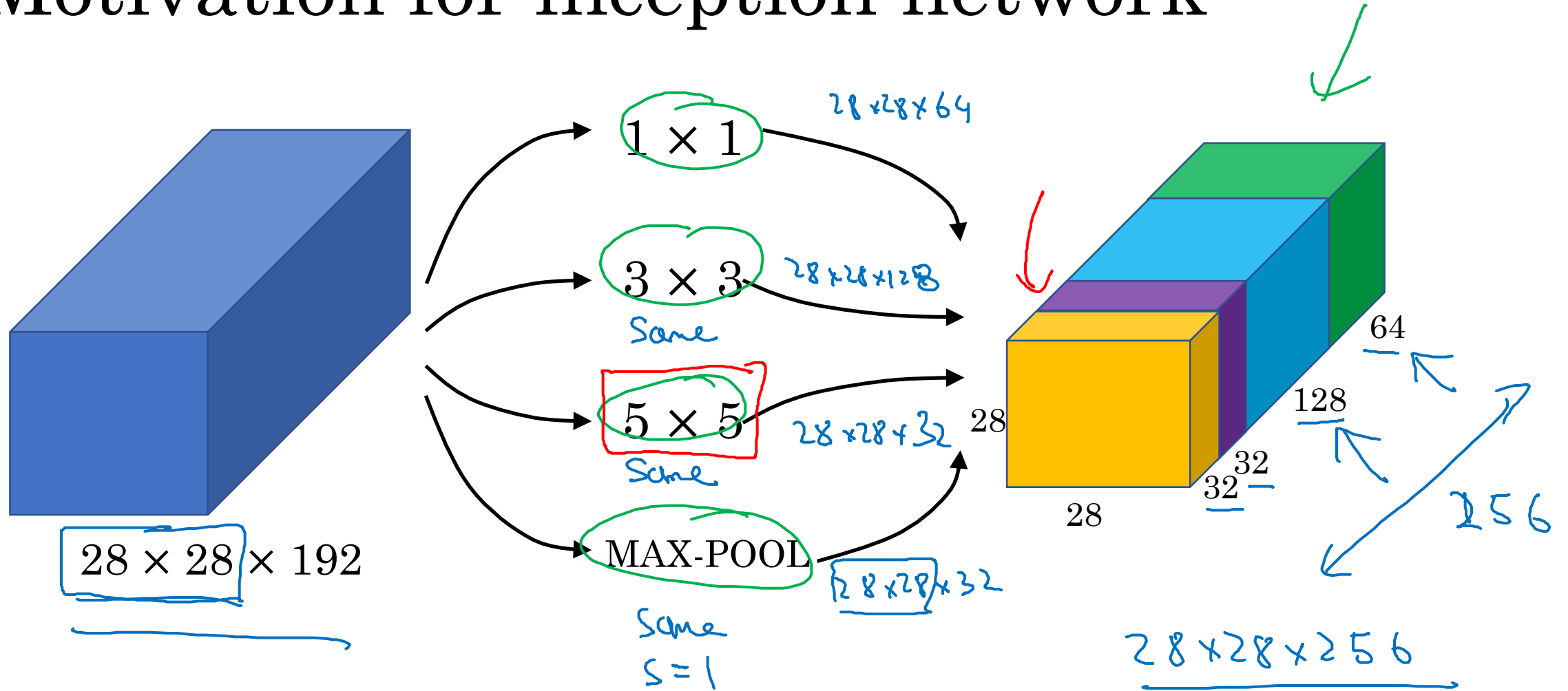


deeplearning.ai

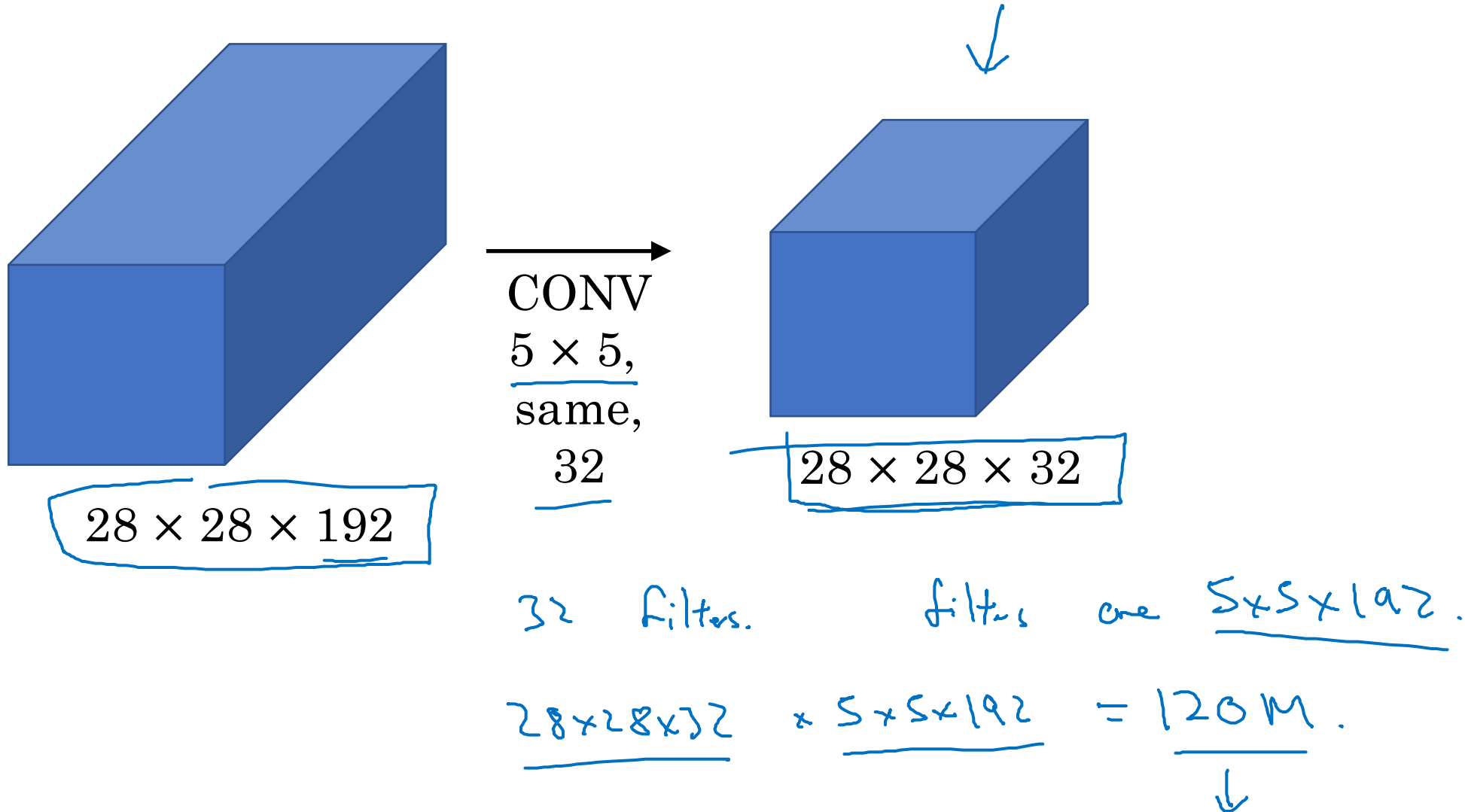
Case Studies

Inception network motivation

Motivation for inception network

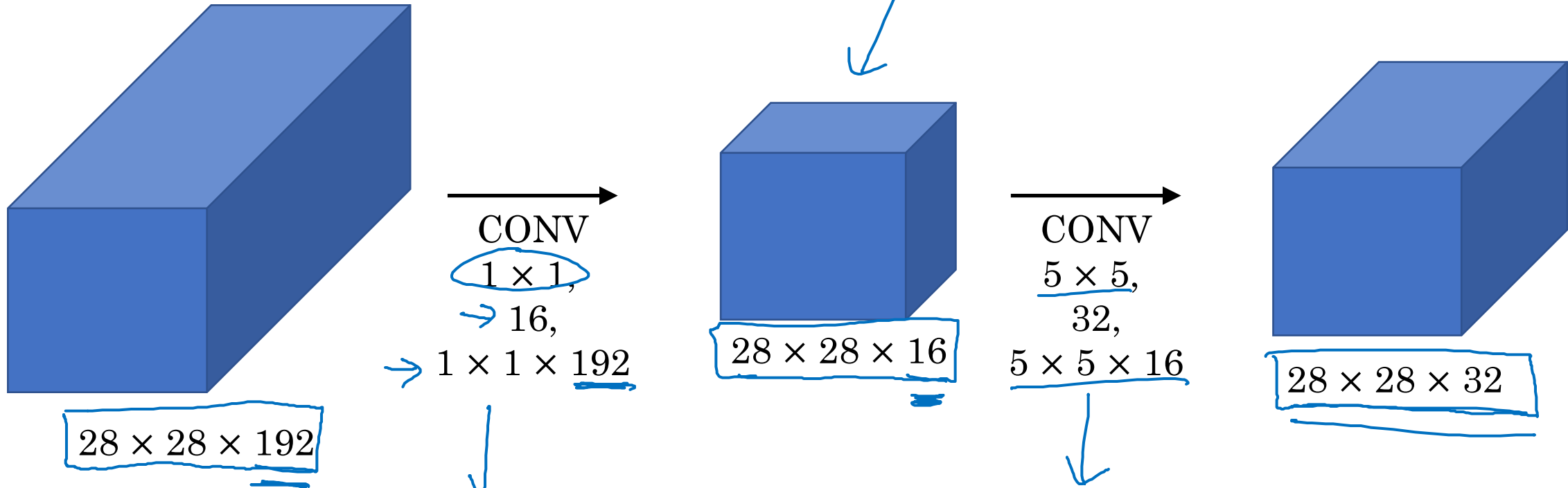
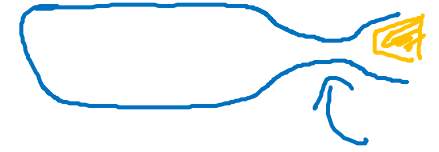


The problem of computational cost



Using 1×1 convolution

"bottleneck layers"



$$28 \times 28 \times 16 \times 192 = 2.4M$$

$$28 \times 28 \times 32 \times 5 \times 5 \times 16 = 10.0M$$

12.4M

120M

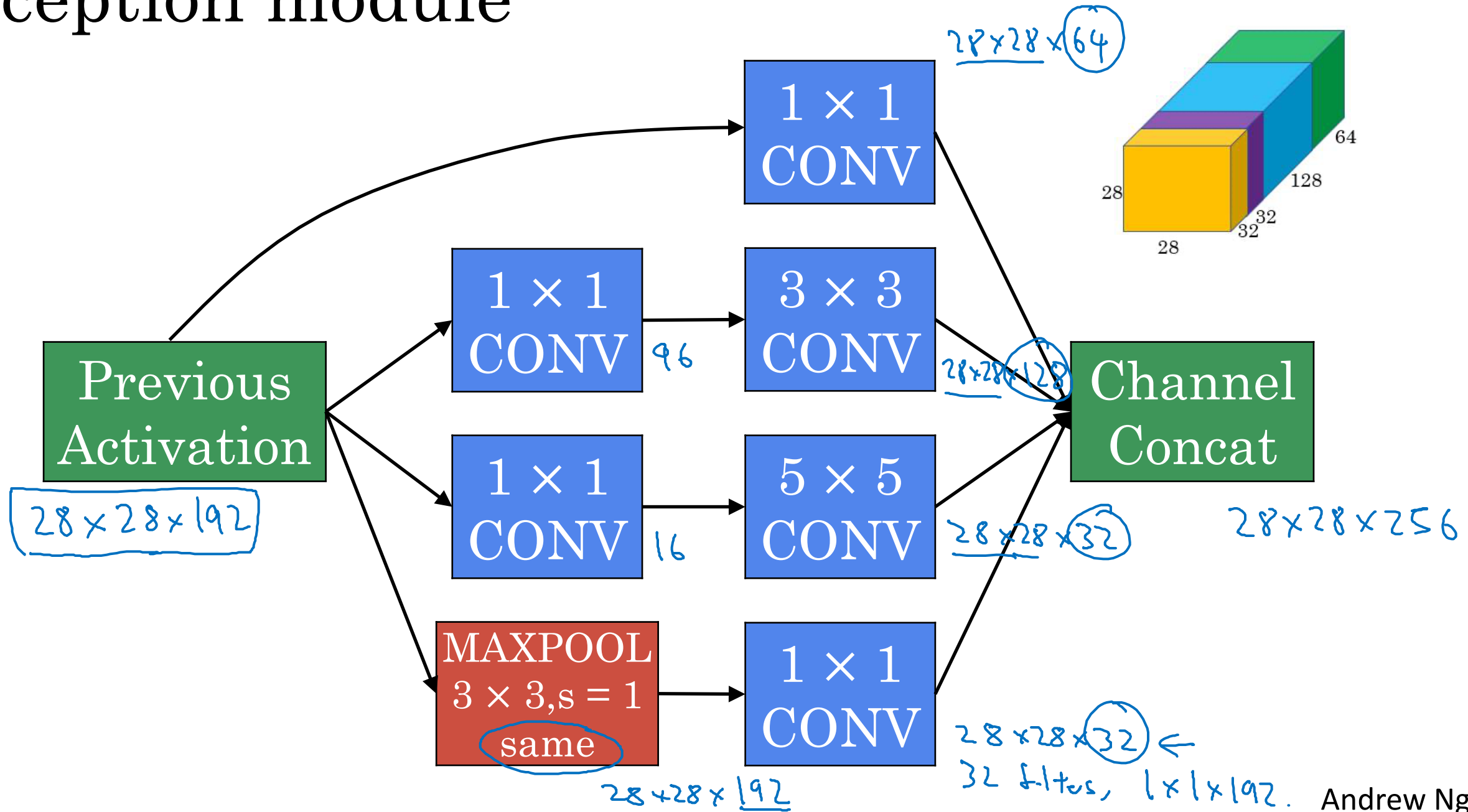


deeplearning.ai

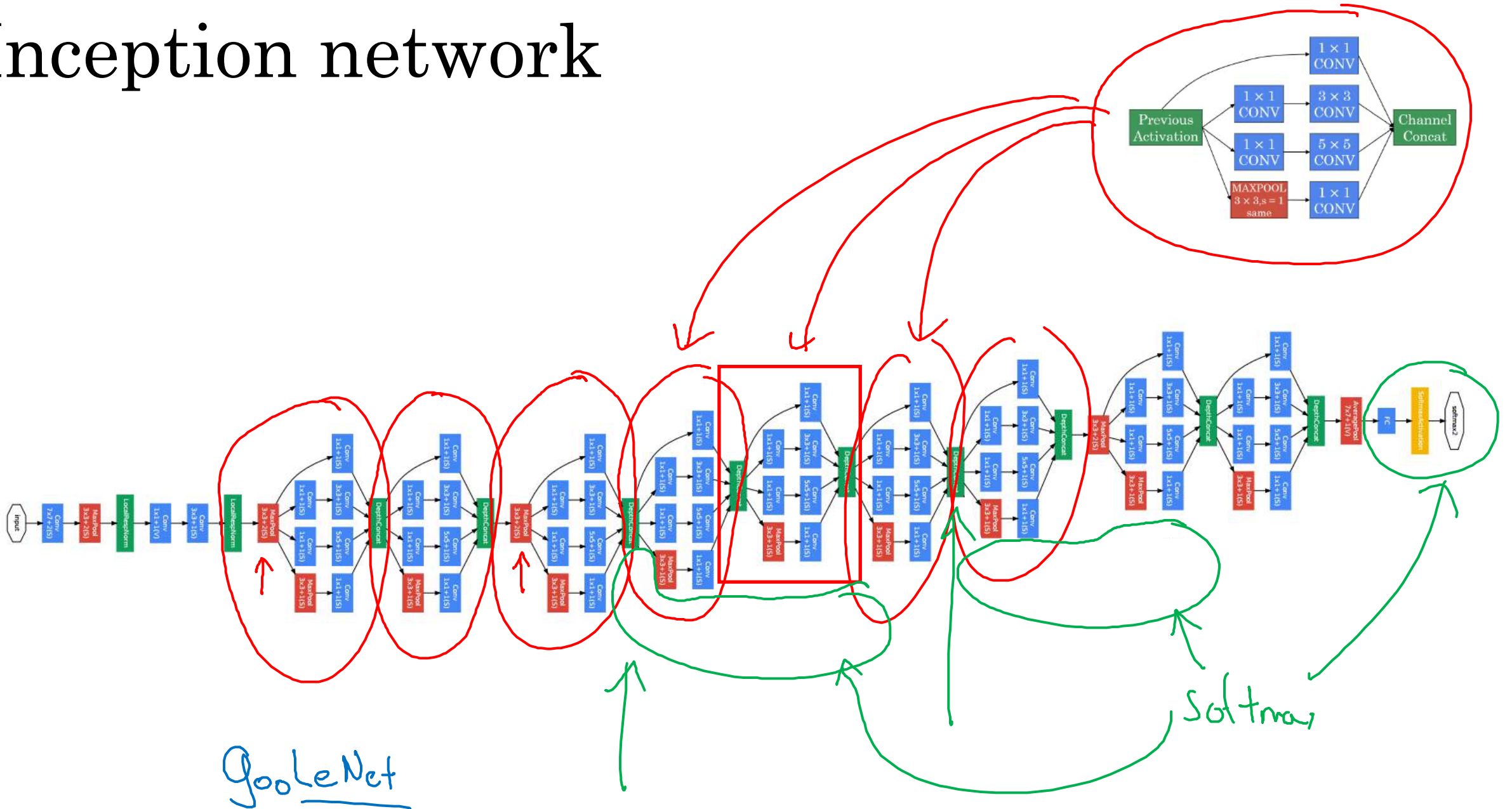
Case Studies

Inception network

Inception module



Inception network







deeplearning.ai

Convolutional Neural Networks

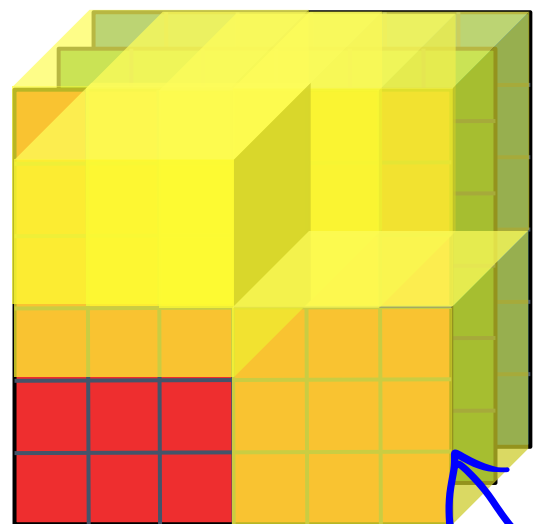
MobileNet

Motivation for MobileNets

- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions

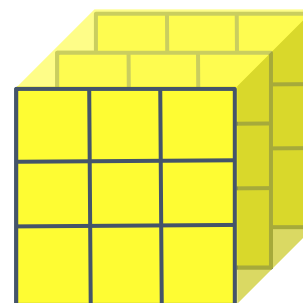


Normal Convolution



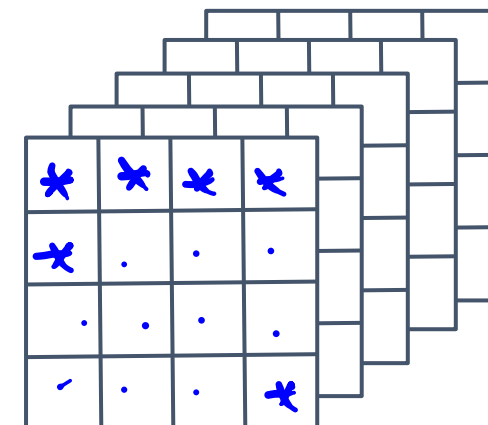
$6 \times 6 \times 3$
 $n \times n \times n_c$

*



$3 \times 3 \times 3$
 $f \times f \times n_c$

=

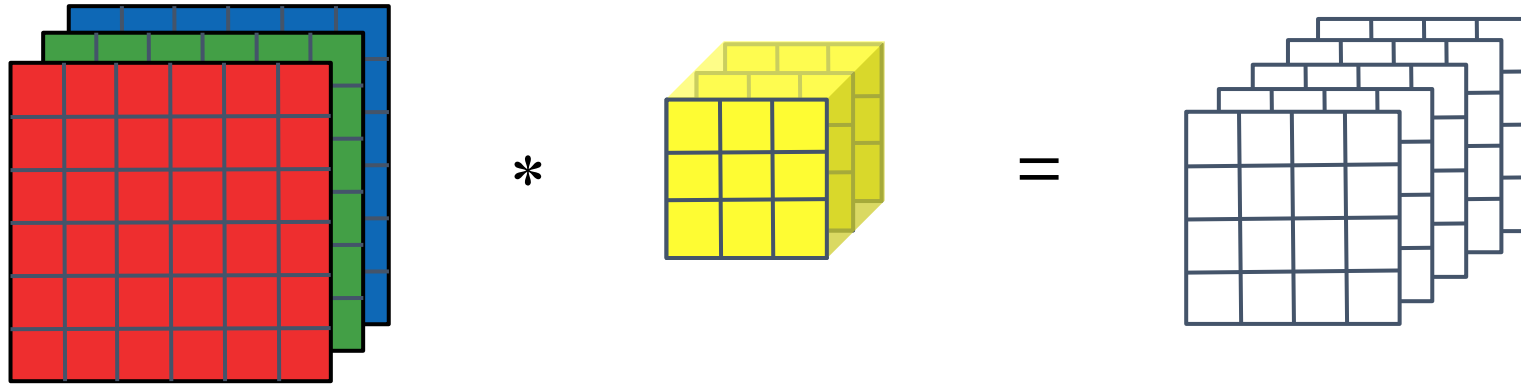


$4 \times 4 \times 5$
 $n_{out} \times n_{out} \times n_c'$

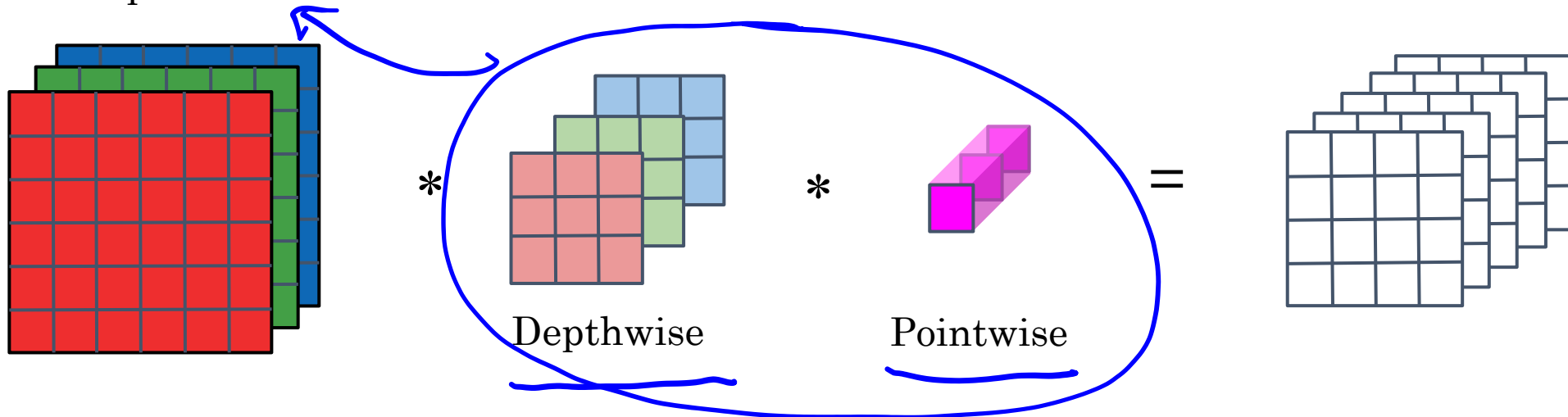
$$\begin{aligned}
 \text{Computational cost} &= \frac{\text{\#filter params}}{3 \times 3 \times 3} \times \frac{\text{\# filter positions}}{4 \times 4} \times \text{\# of filters} \\
 &\rightarrow \underline{2160} = \uparrow \quad \quad \uparrow \quad \quad \uparrow
 \end{aligned}$$

Depthwise Separable Convolution

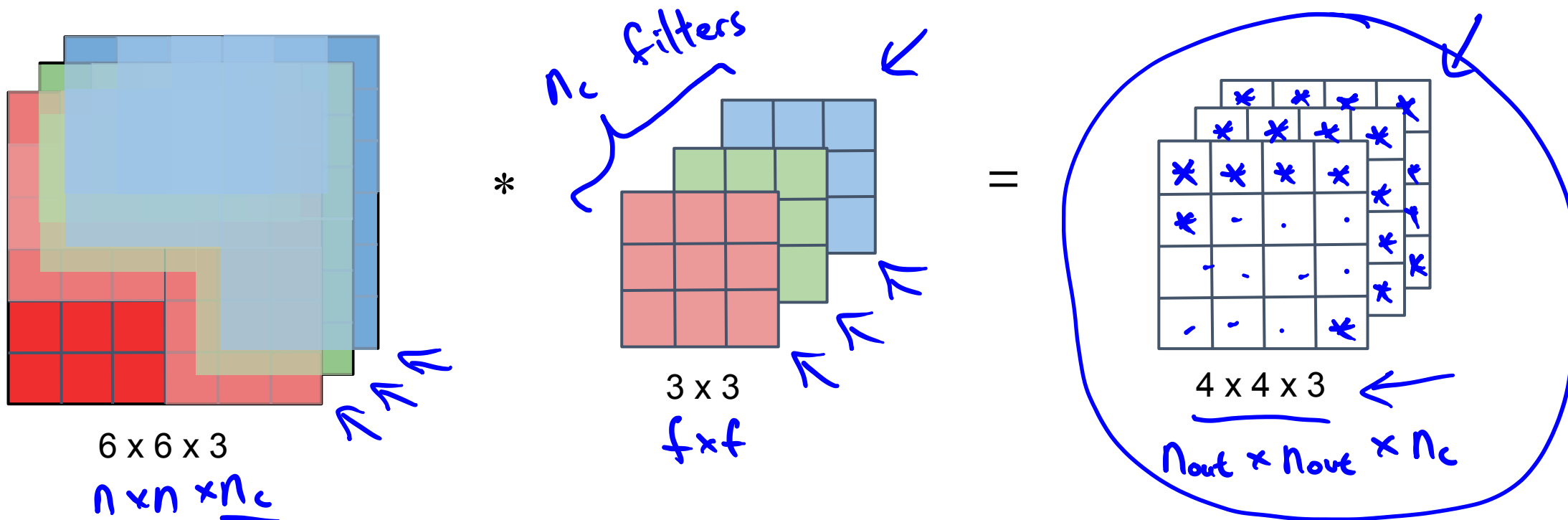
Normal Convolution



Depthwise Separable Convolution



Depthwise Convolution

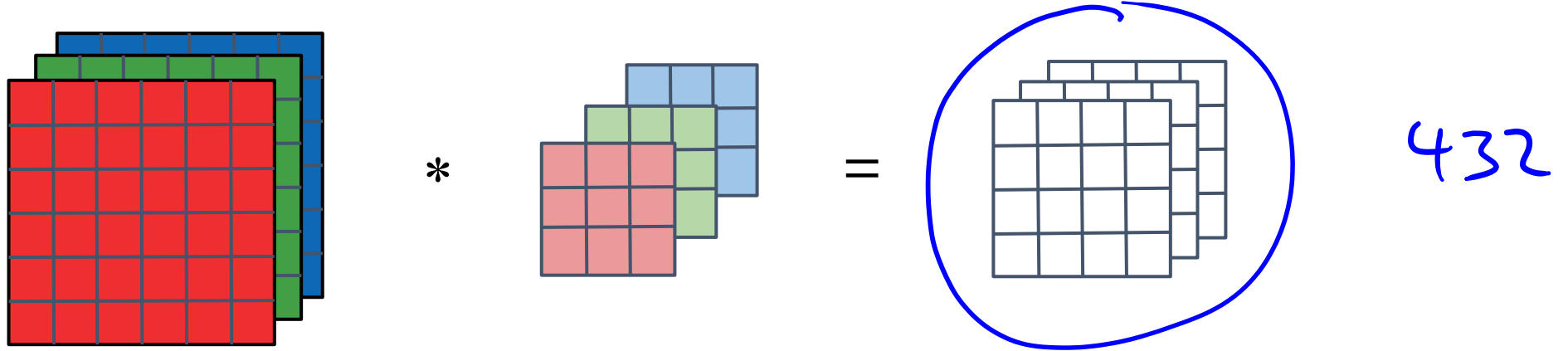


Computational cost = #filter params x # filter positions x # of filters

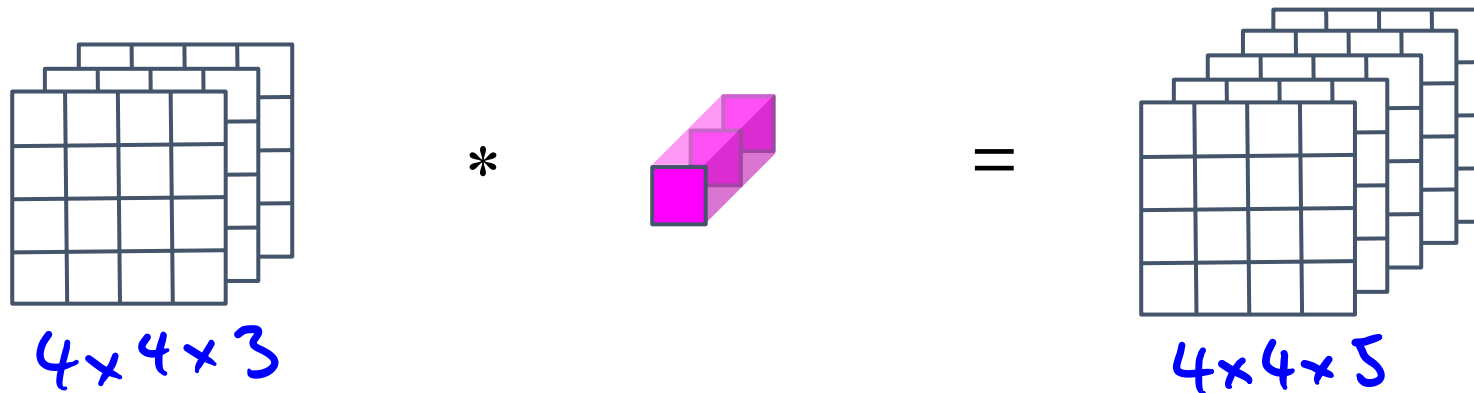
$$432 = \underbrace{3 \times 3}_{\text{#filter params}} \times \underbrace{4 \times 4}_{\text{# filter positions}} \times 3_{\text{# of filters}}$$

Depthwise Separable Convolution

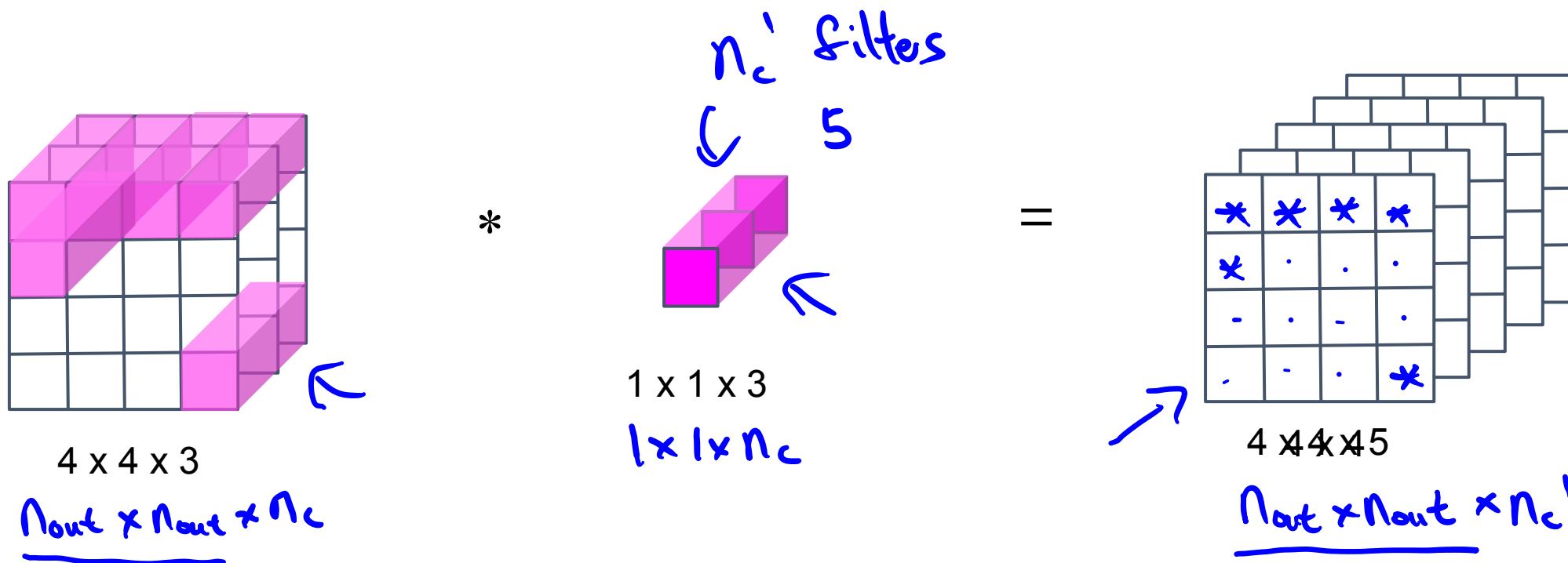
Depthwise Convolution



Pointwise Convolution



Pointwise Convolution

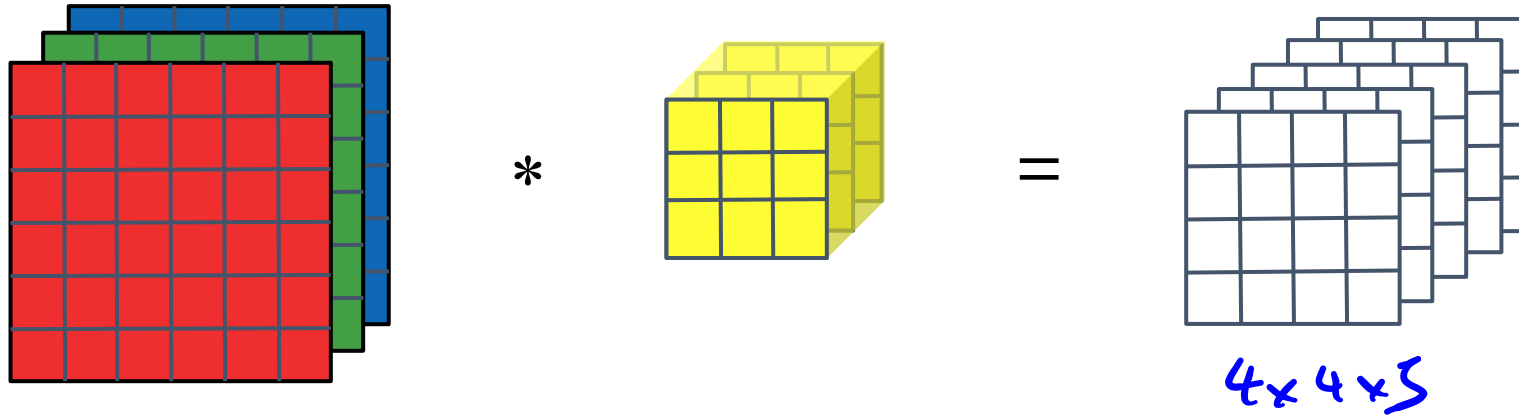


Computational cost = #filter params x # filter positions x # of filters

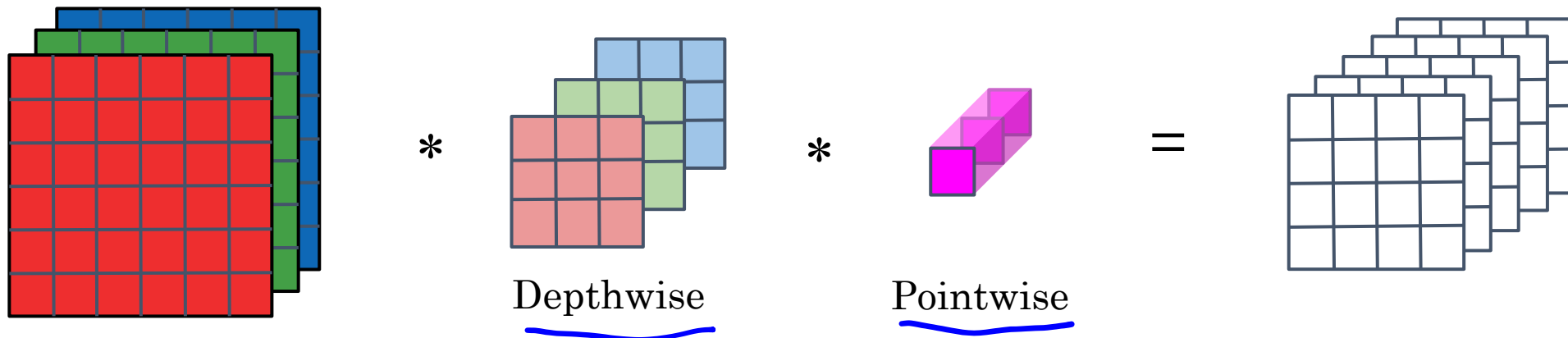
240 = $1 \times 1 \times 3$ x 4×4 x 5

Depthwise Separable Convolution

Normal Convolution



Depthwise Separable Convolution



Cost Summary

Cost of normal convolution

2160

Cost of depthwise separable convolution

depthwise + pointwise
432 + 240 = 672

$$\frac{672}{2160} = 0.31 \leftarrow$$

$$= \frac{1}{\textcircled{n_c}} + \frac{1}{f^2}$$
$$\frac{1}{5} + \frac{1}{9}$$

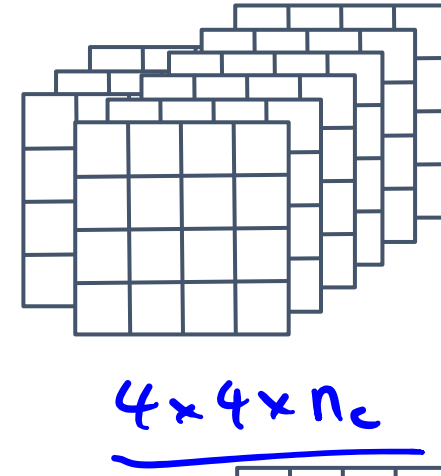
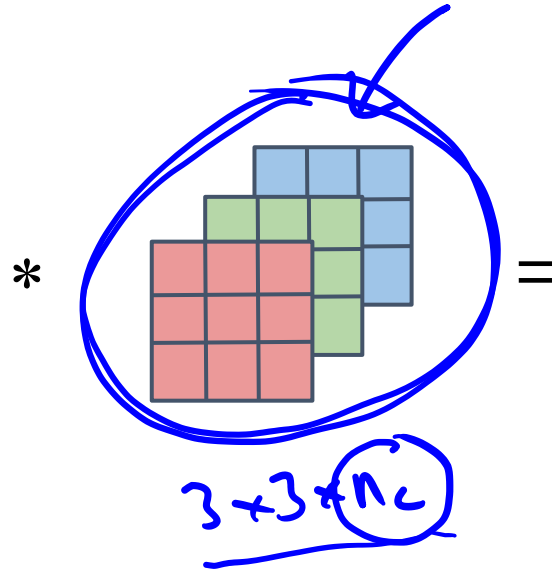
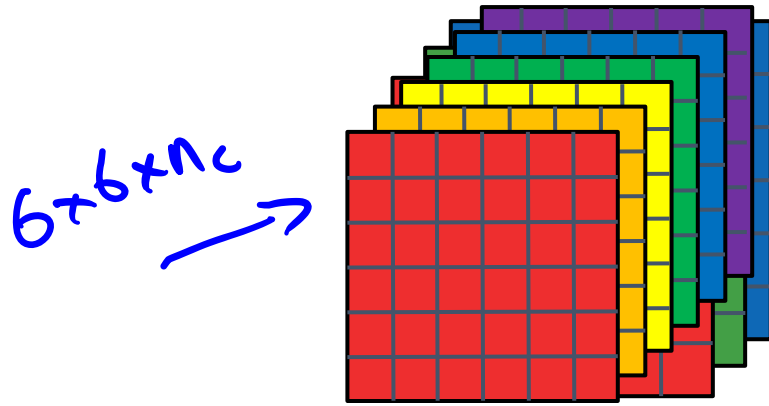
$$= \frac{1}{512} + \frac{1}{3^2}$$

↑ ↑

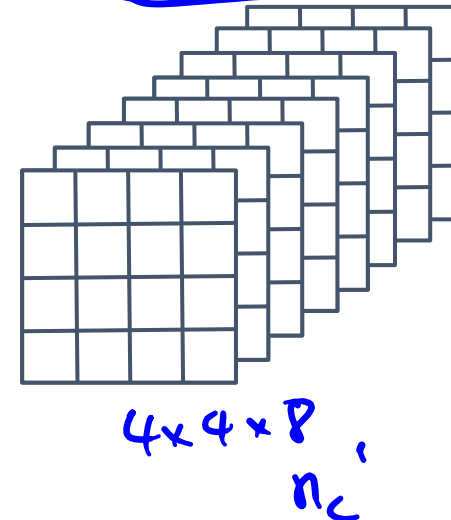
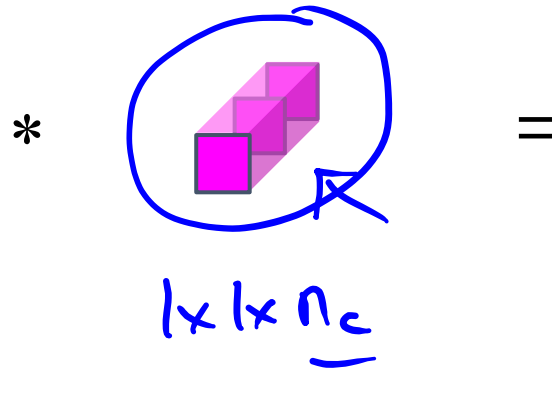
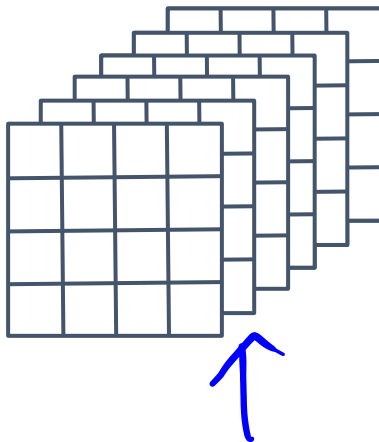
~10 times cheaper

Depthwise Separable Convolution

Depthwise Convolution



Pointwise Convolution





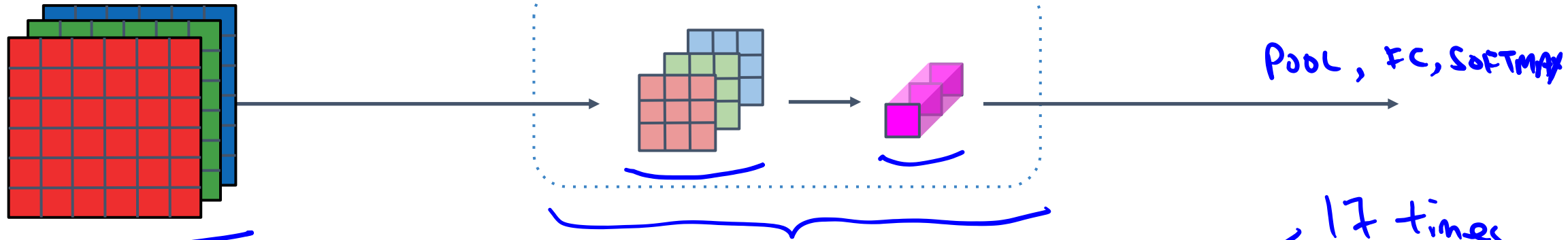
deeplearning.ai

Convolutional Neural Networks

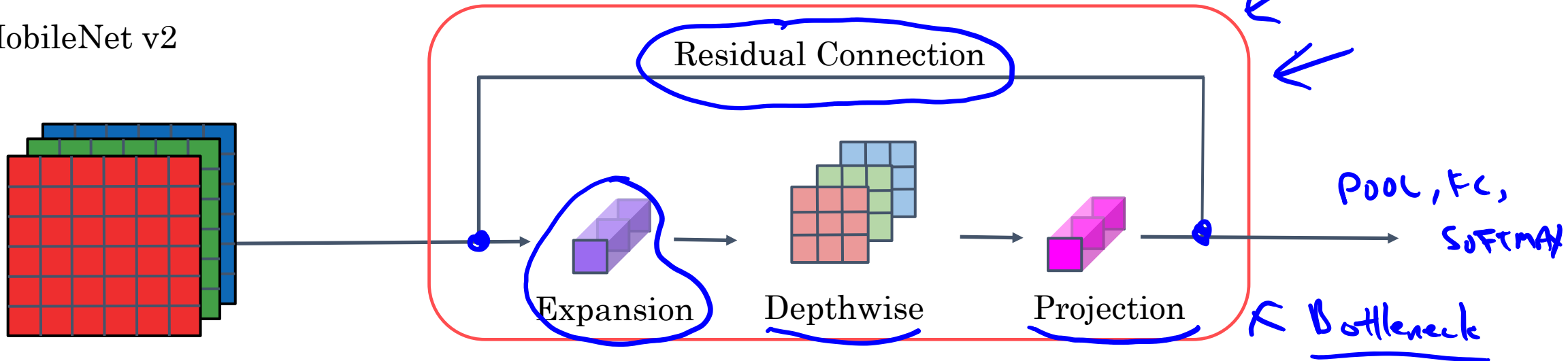
MobileNet Architecture

MobileNet

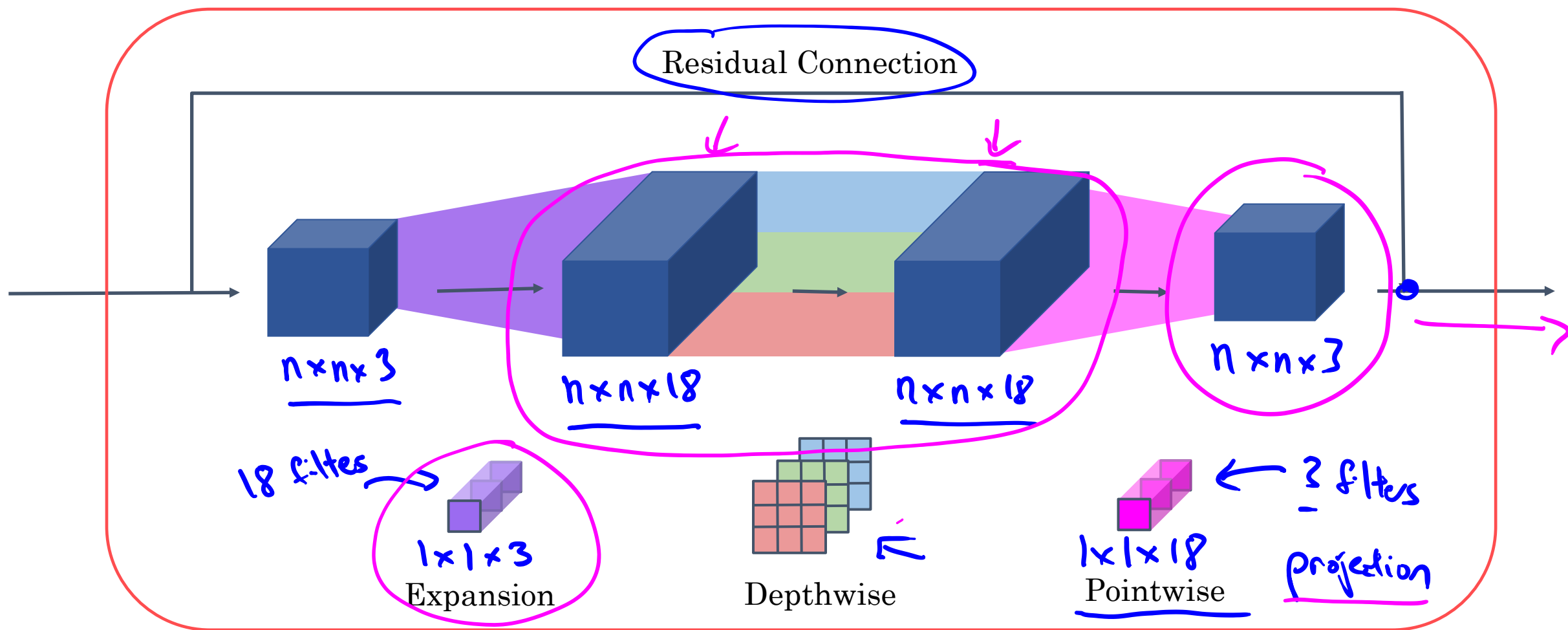
MobileNet v1



MobileNet v2

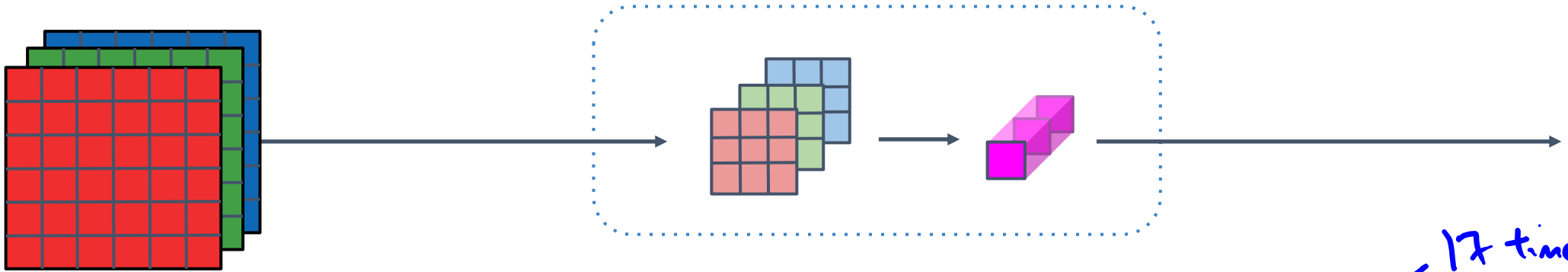


MobileNet v2 Bottleneck

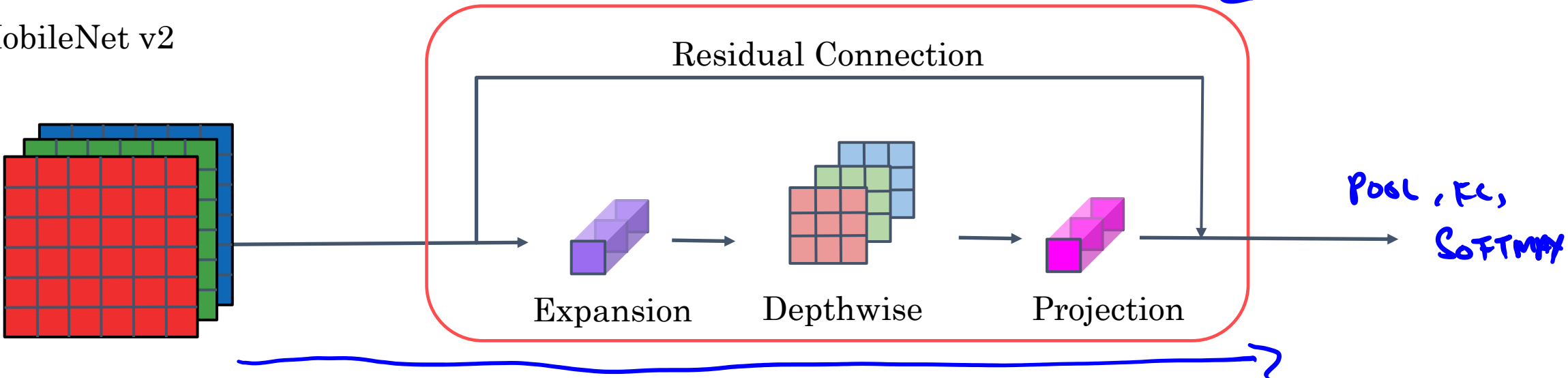


MobileNet

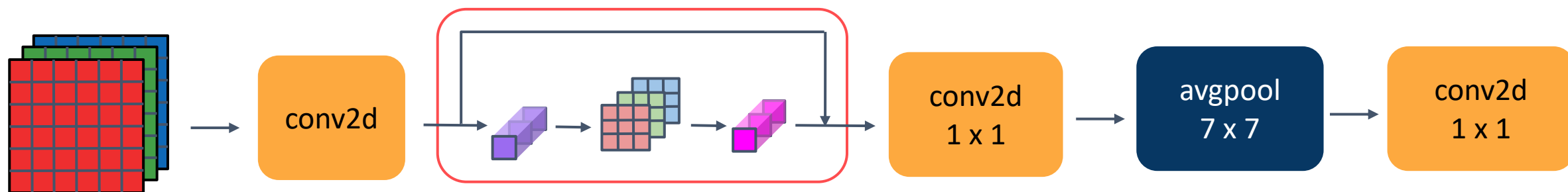
MobileNet v1



MobileNet v2



MobileNet v2 Full Architecture





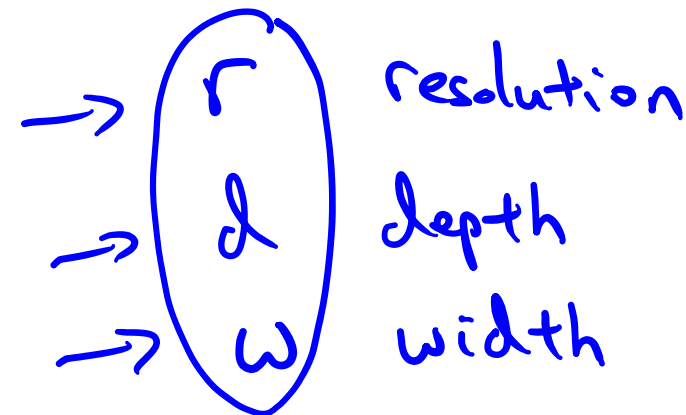
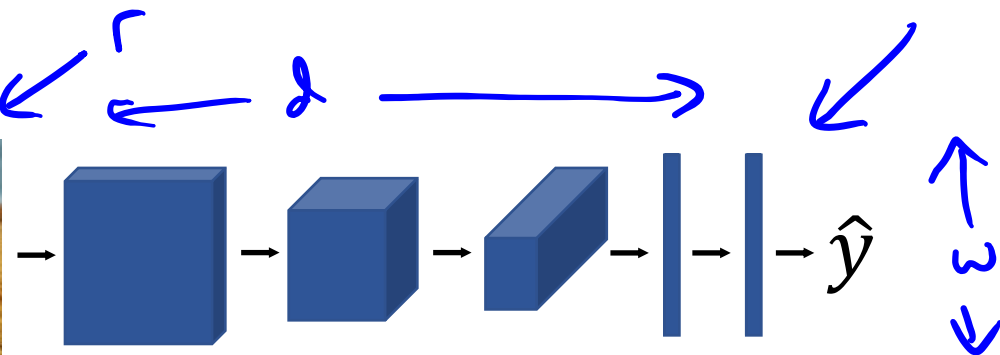
deeplearning.ai

Convolutional Neural Networks

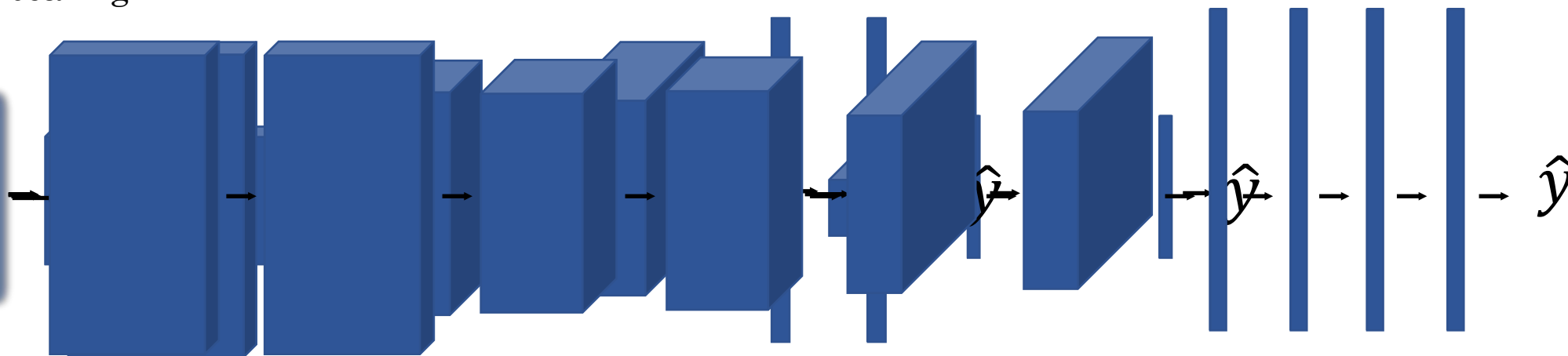
EfficientNet

EfficientNet

Baseline



Compound Scaling



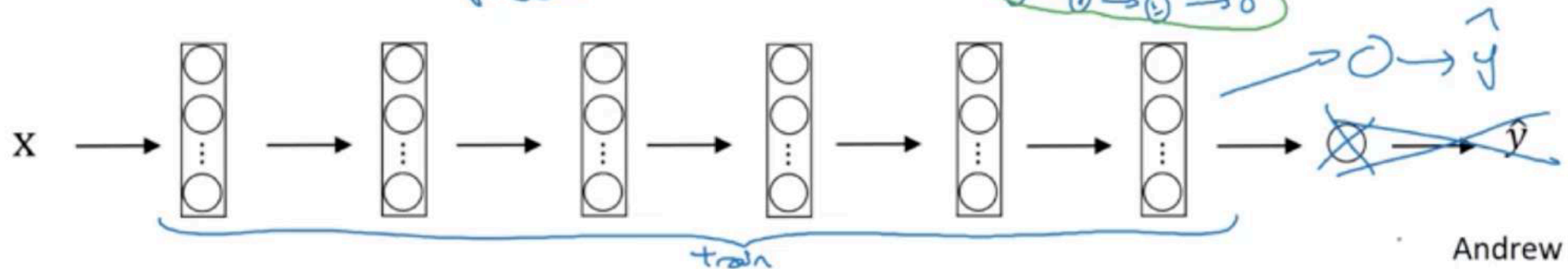
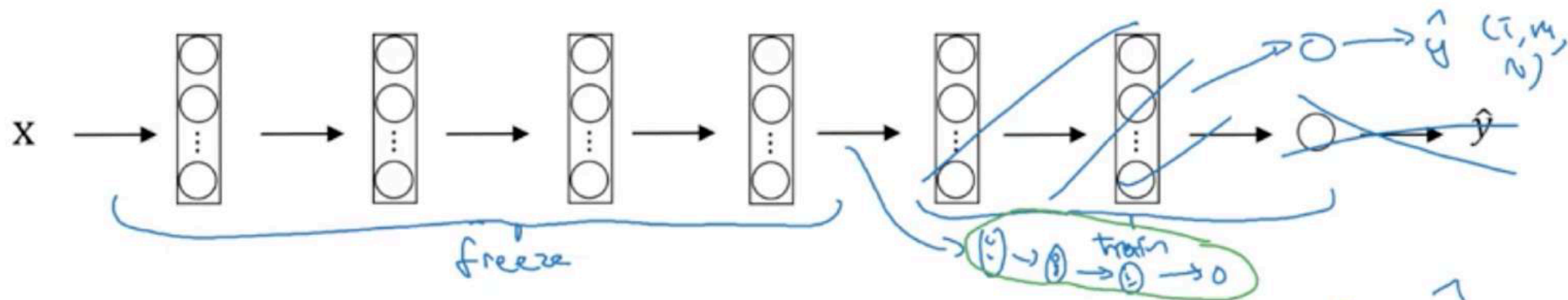
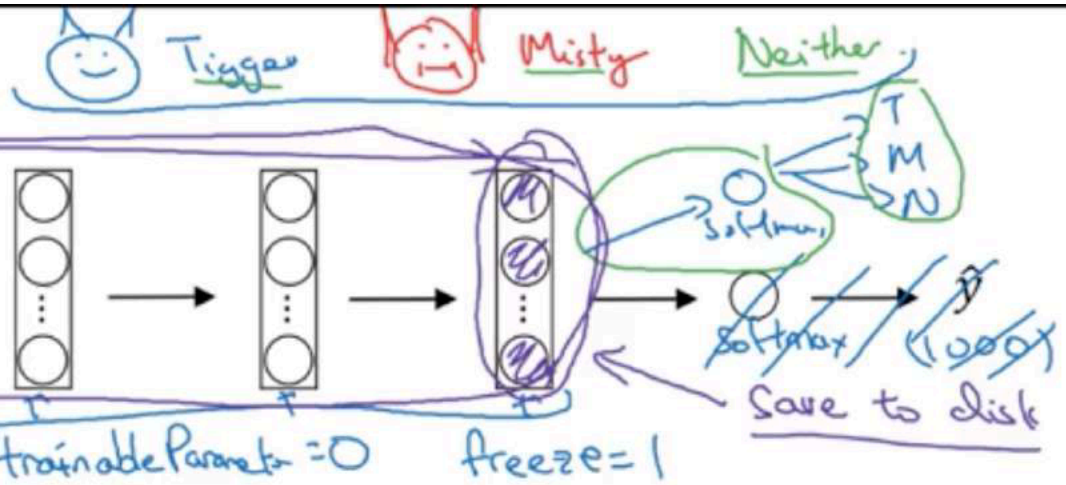


deeplearning.ai

Practical advice for
using ConvNets

Transfer Learning

Transfer Learning





deeplearning.ai

Practical advice for using ConvNets

Data augmentation

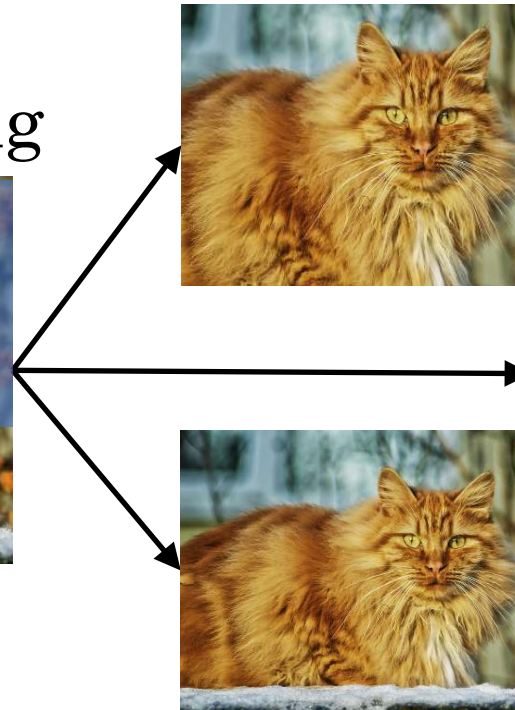
Common augmentation method

Mirroring



y

Random Cropping

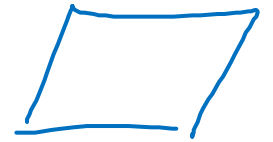


Rotation

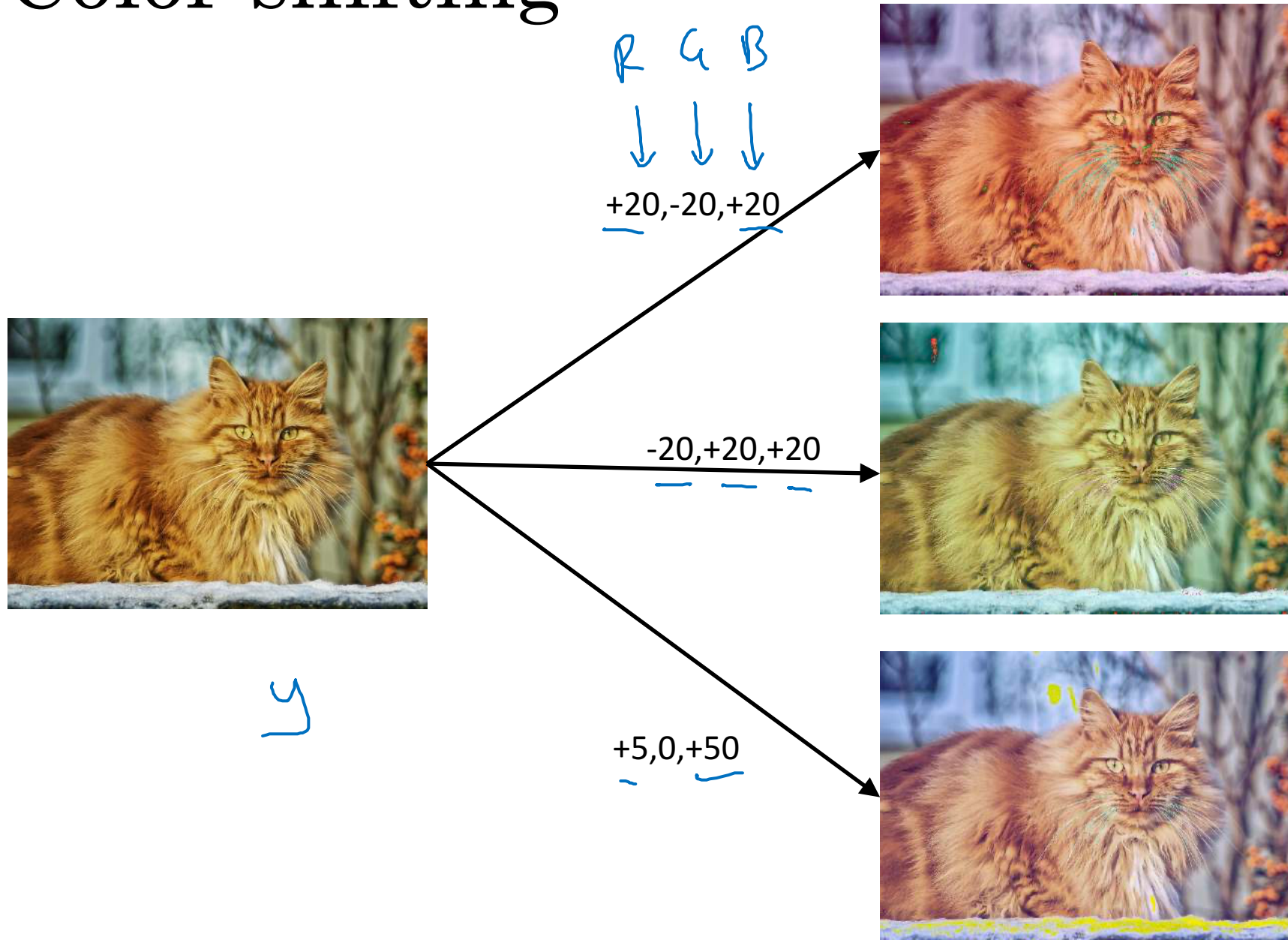
Shearing

Local warping

...



Color shifting



Advanced:

PCA

ml-class.org

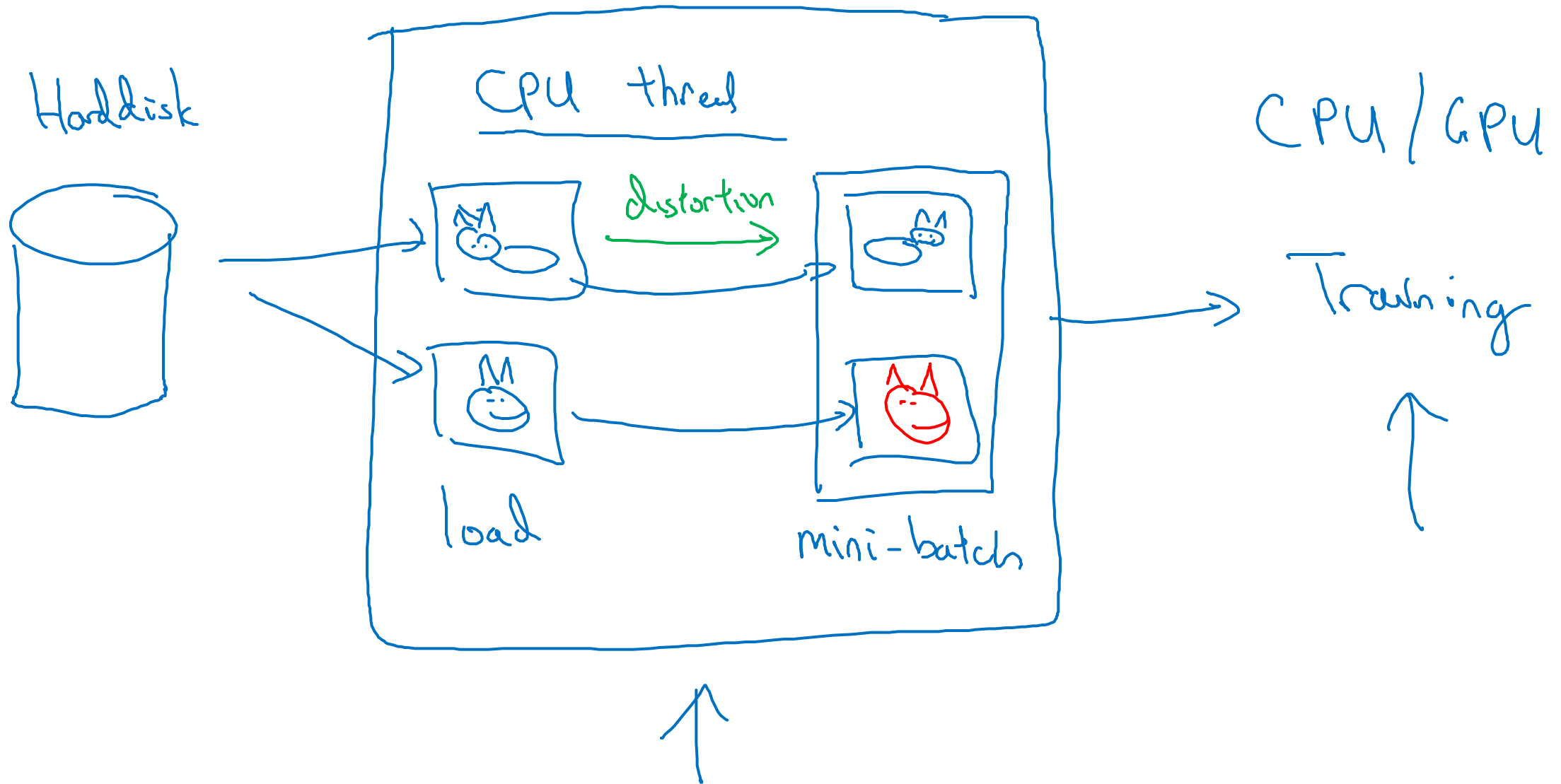
[AlexNet paper

["PCA color augmentation."

R B

G

Implementing distortions during training



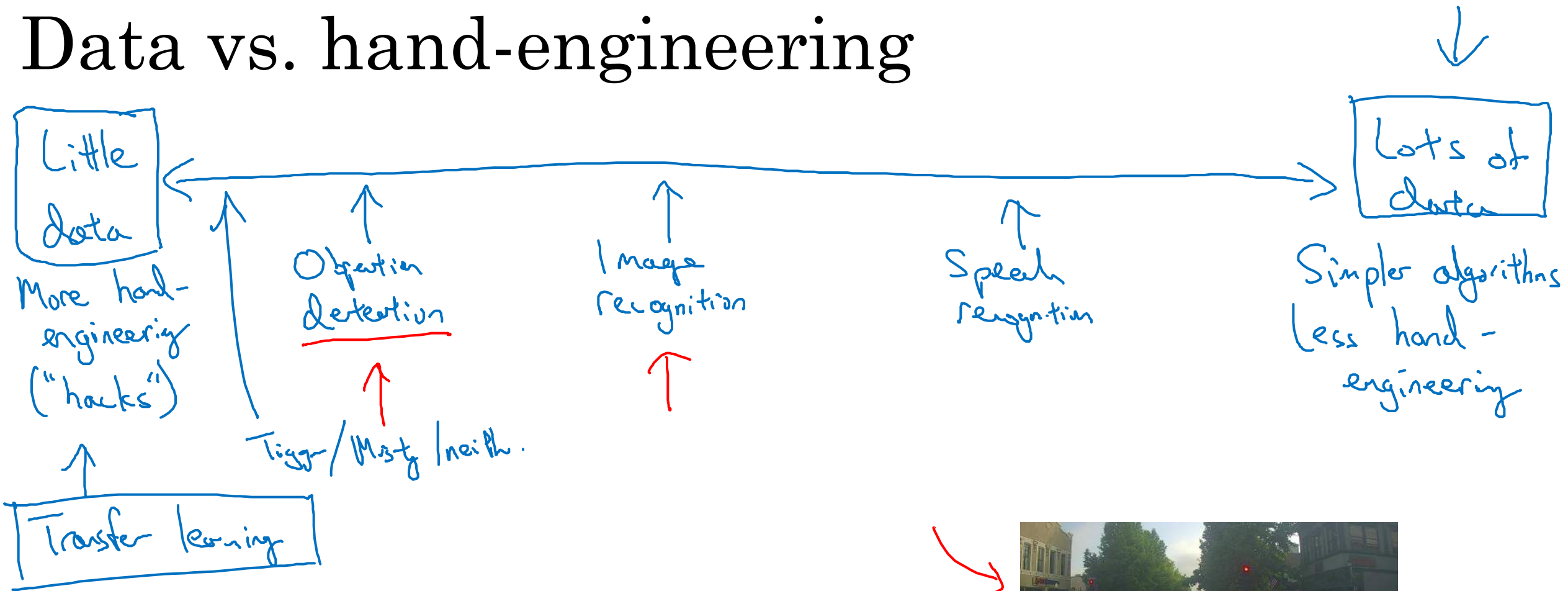


deeplearning.ai

Practical advice for using ConvNets

The state of computer vision

Data vs. hand-engineering



Two sources of knowledge

- • Labeled data (x, y)
- • Hand engineered features/network architecture/other components



Tips for doing well on benchmarks/winning competitions

3-15 networks

→ $\hat{y}$

Ensembling

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop



1

+



4

+



1

+



4

Use open source code

- Use architectures of networks published in the literature
- Use open source implementations if possible
- Use pretrained models and fine-tune on your dataset



deeplearning.ai

Object Detection

Object
localization

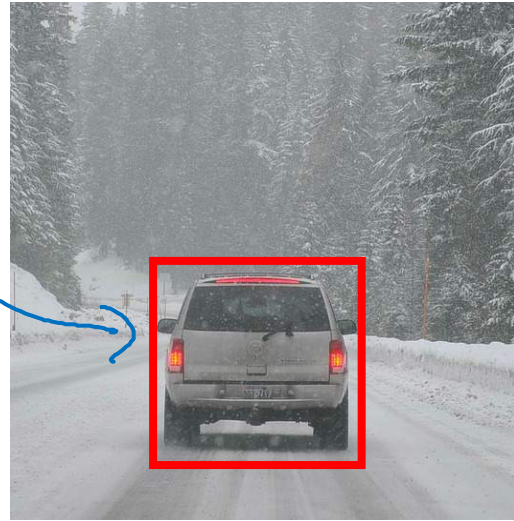
What are localization and detection?

Image classification



"Car"

Classification with
localization



"Car"

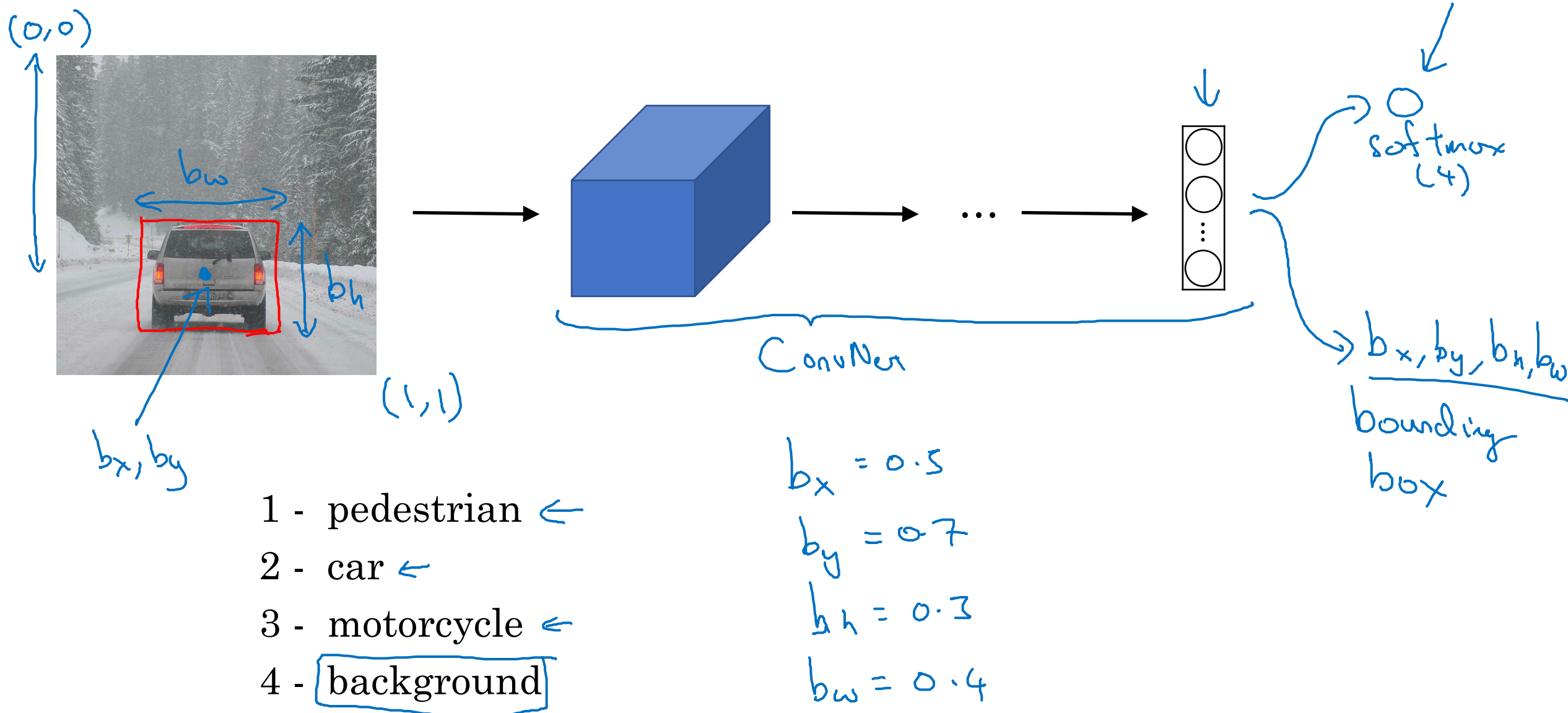
1 object

Detection



multiple
objects

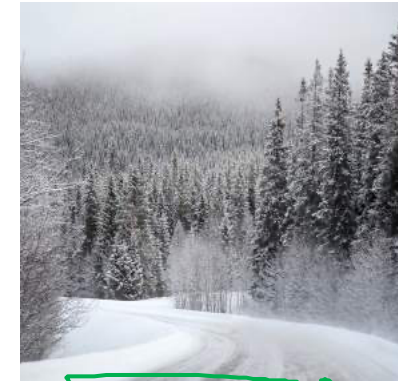
Classification with localization



Defining the target label y

- 1 - pedestrian
- 2 - car ←
- 3 - motorcycle
- 4 - background ←

Need to output b_x, b_y, b_h, b_w , class label (1-4)



$$L(\hat{y}, y) = \begin{cases} (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + \dots + (\hat{y}_8 - y_8)^2 & \text{if } \underline{y_1 = 1} \\ (\hat{y}_1 - y_1)^2 & \text{if } \underline{y_1 = 0} \end{cases}$$

$$\rightarrow y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad \left. \begin{array}{l} \text{is there any} \\ \text{object?} \end{array} \right\}$$

(x, y)

$$\begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ \vdots \end{bmatrix} \quad \left. \begin{array}{l} p_c \\ \text{"don't care"} \end{array} \right\}$$

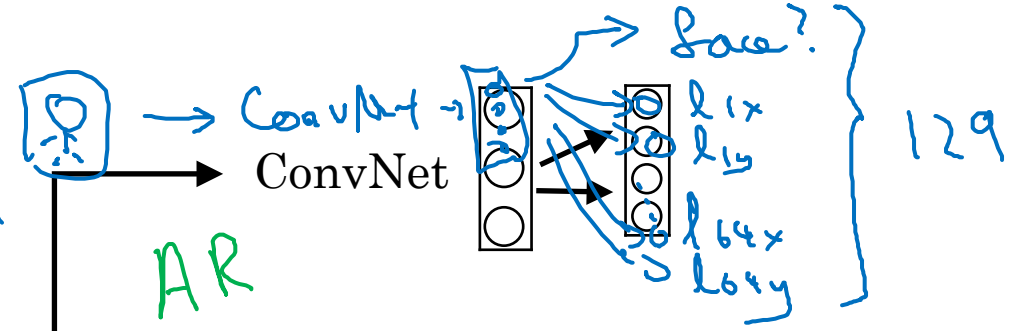


deeplearning.ai

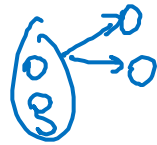
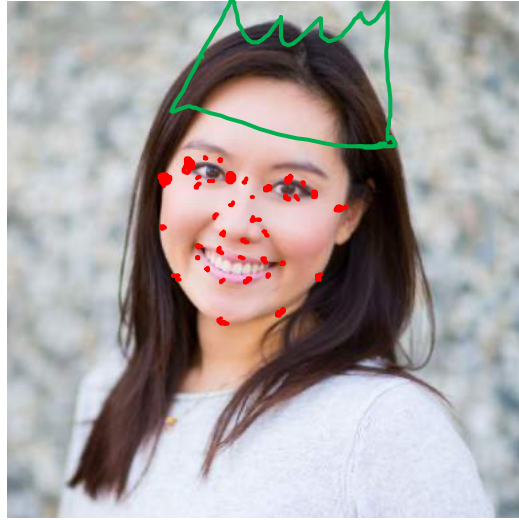
Object Detection

Landmark
detection

Landmark detection



b_x, b_y, b_h, b_w



$l_{1x}, l_{1y},$
 $l_{2x}, l_{2y},$
 $l_{3x}, l_{3y},$
 $l_{4x}, l_{4y},$
 $\vdots$
 l_{64x}, l_{64y}

x, y

$l_{1x}, l_{1y},$
 $\vdots$
 l_{32x}, l_{32y}



deeplearning.ai

Object Detection

Object
detection

Car detection example

Training set:

X

y



1



1



1



0



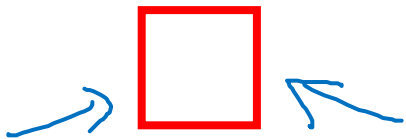
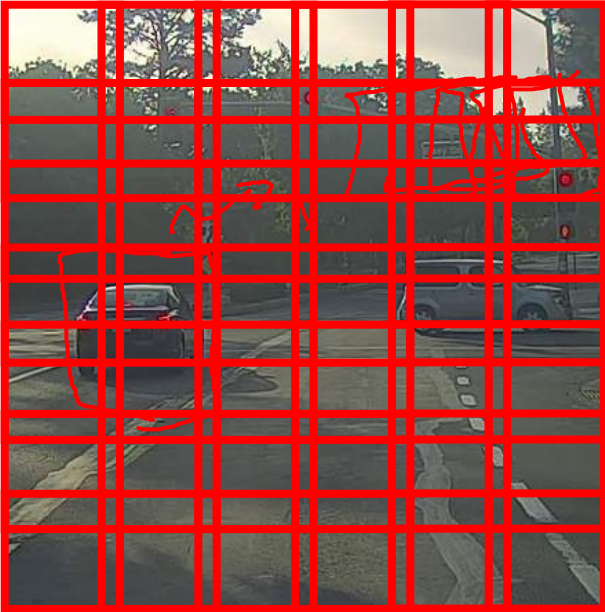
0



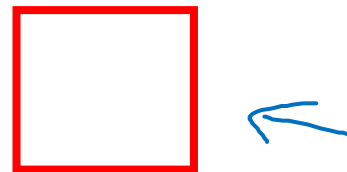
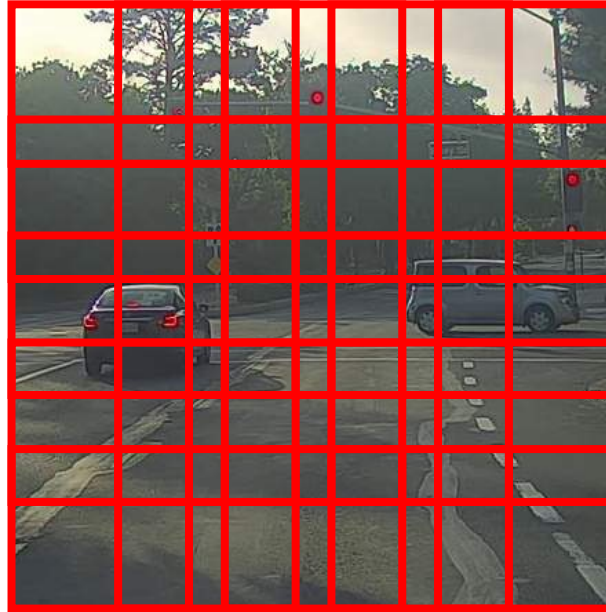
$\rightarrow$ ConvNet $\rightarrow y$

Sliding windows detection

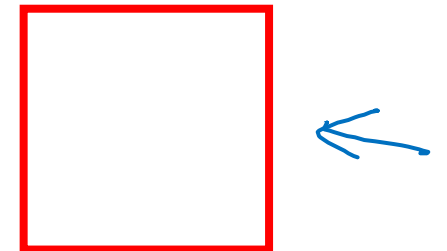
→ ConvNet → 0



→ ConvNet



Computation cost



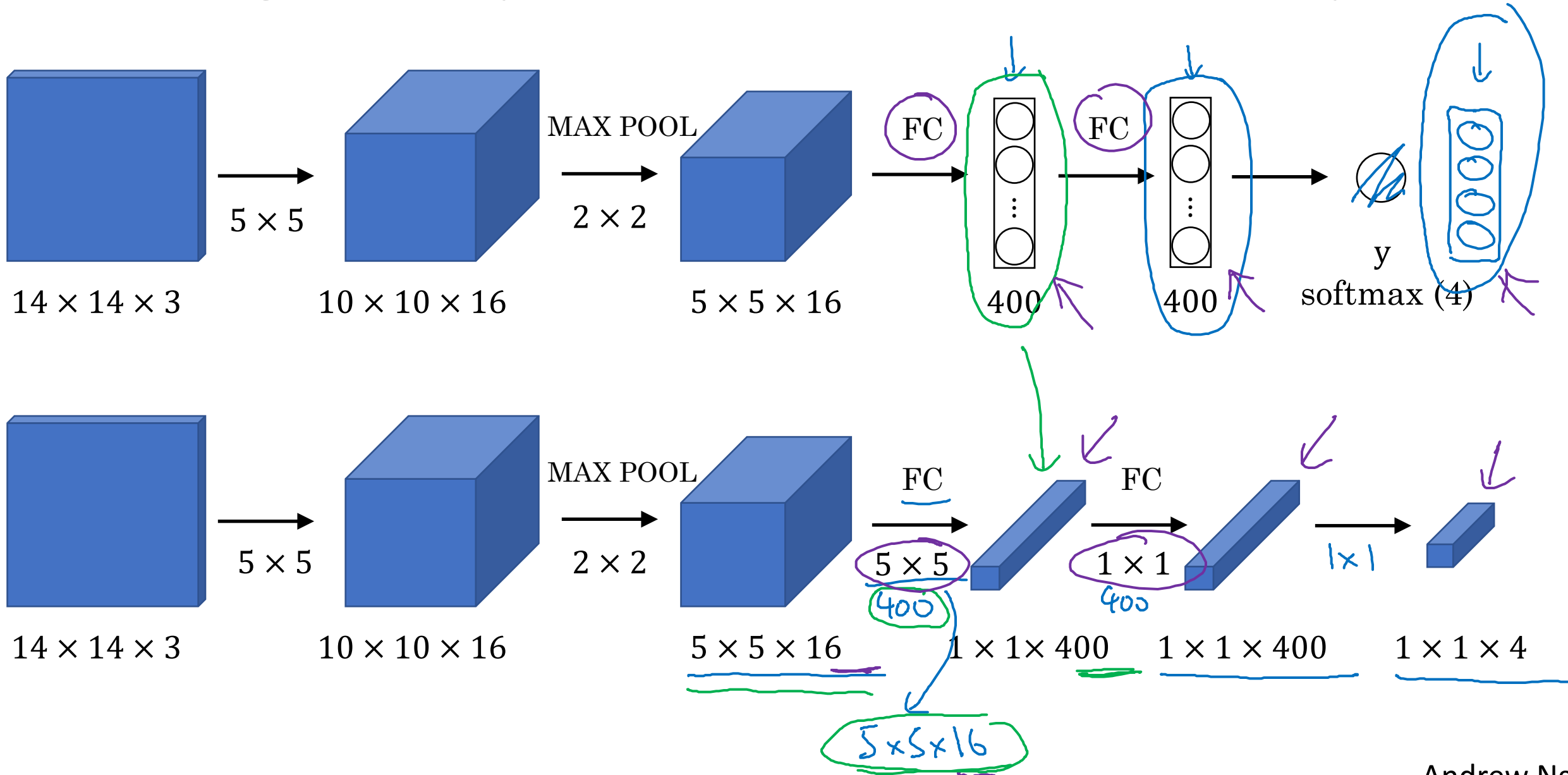


deeplearning.ai

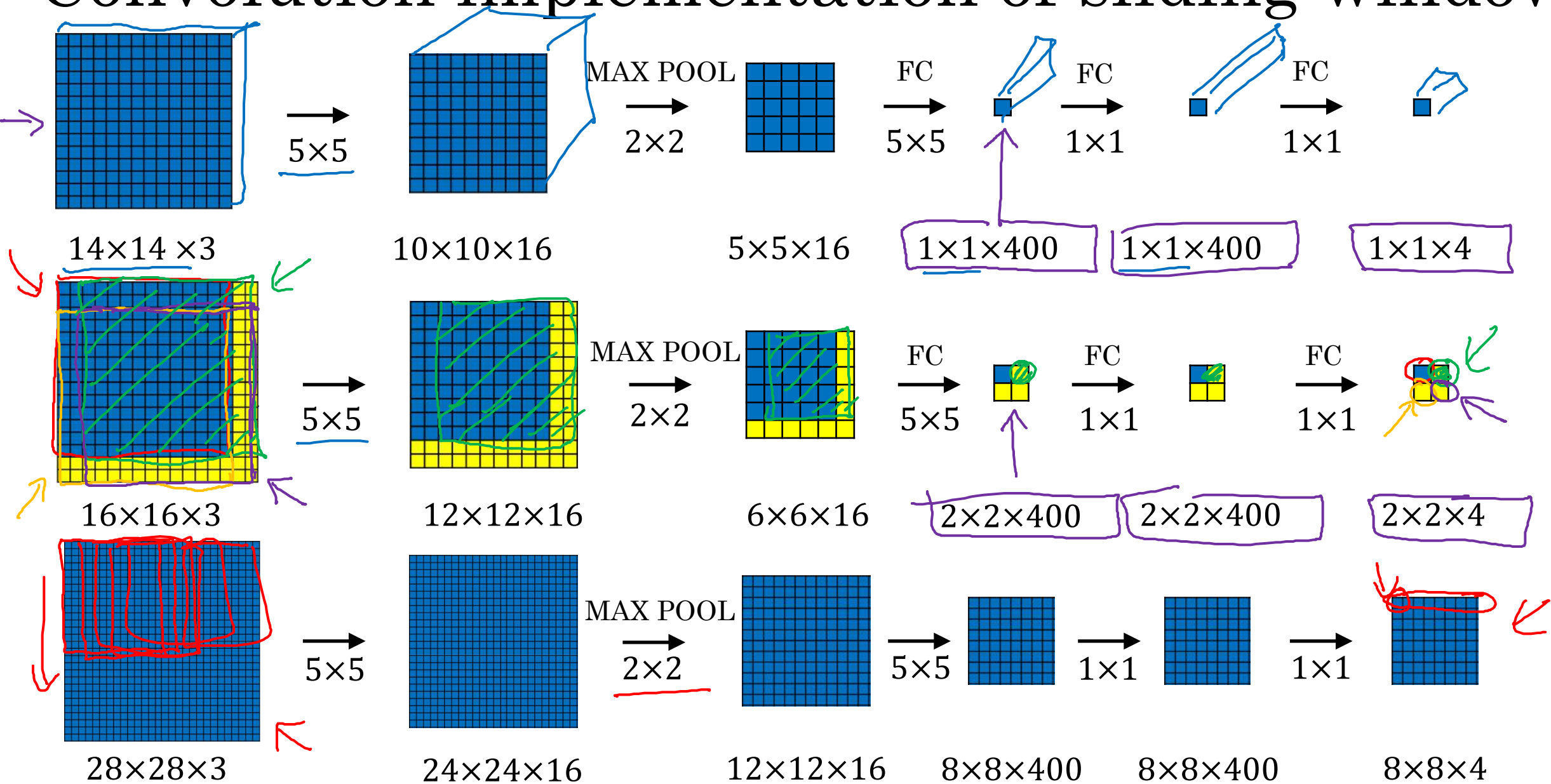
Object Detection

Convolutional
implementation of
sliding windows

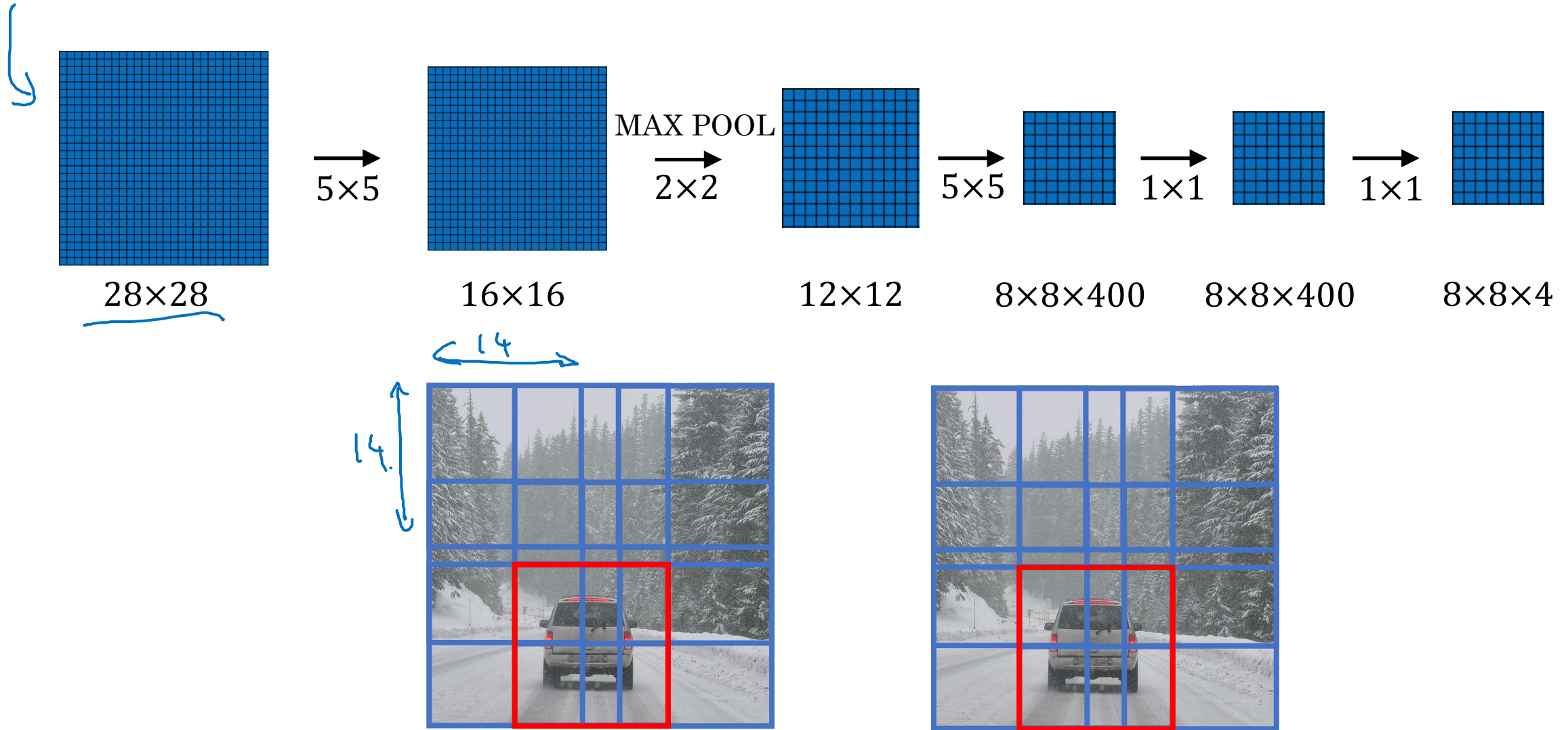
Turning FC layer into convolutional layers



Convolution implementation of sliding windows



Convolution implementation of sliding windows



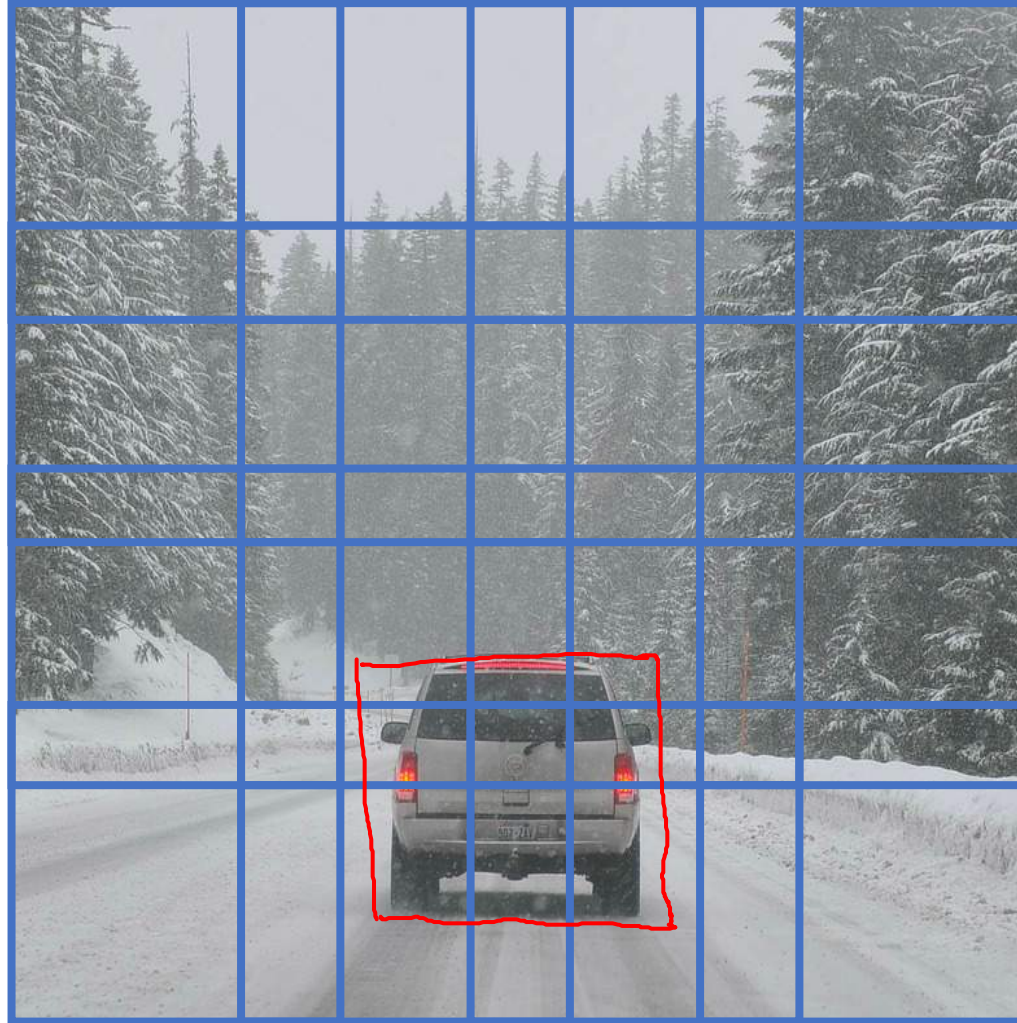


deeplearning.ai

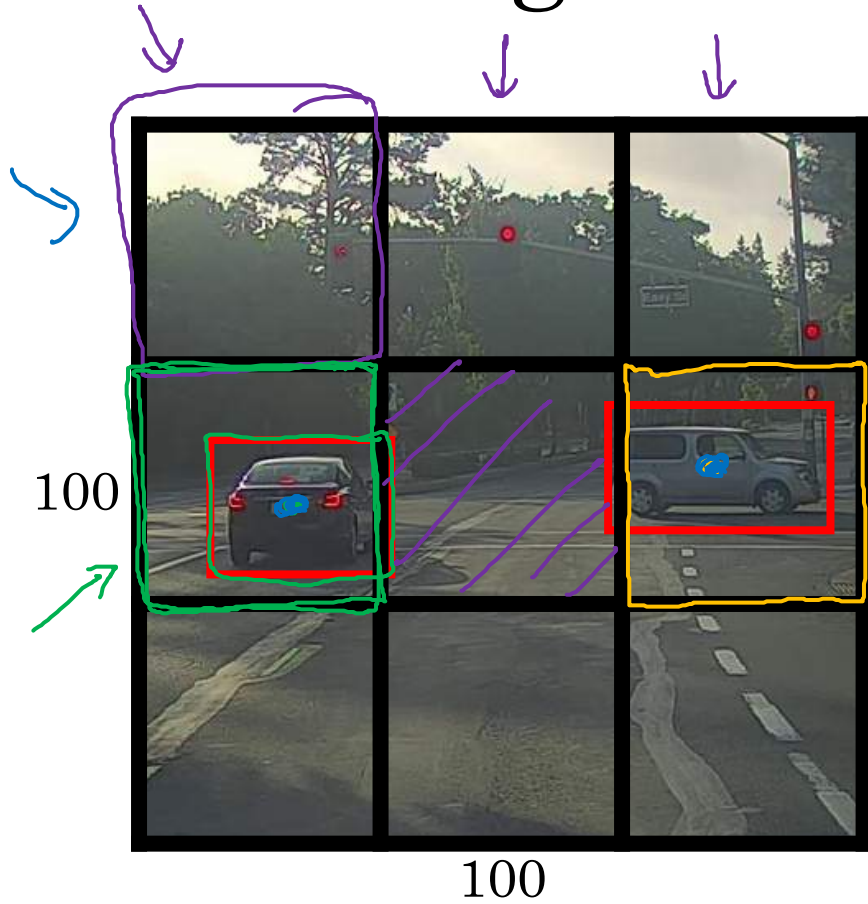
Object Detection

Bounding box
predictions

Output accurate bounding boxes



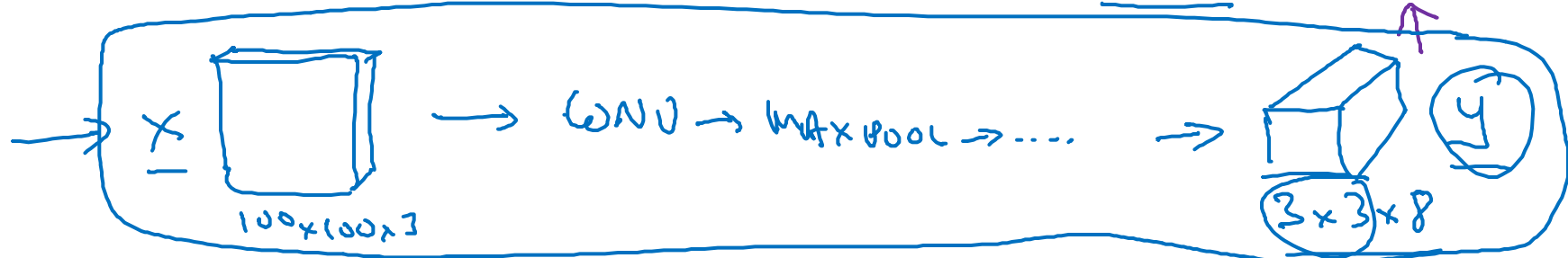
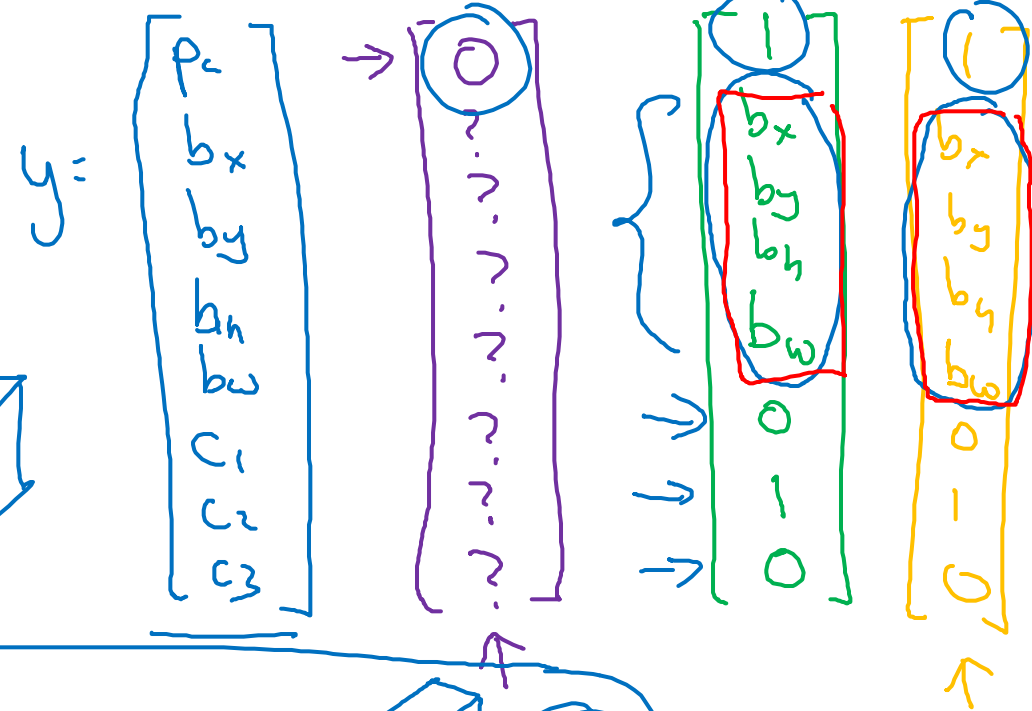
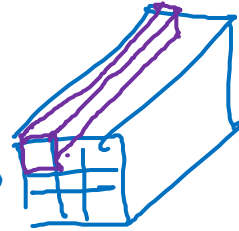
YOLO algorithm



Labels for training
For each grid cell:

Target output:

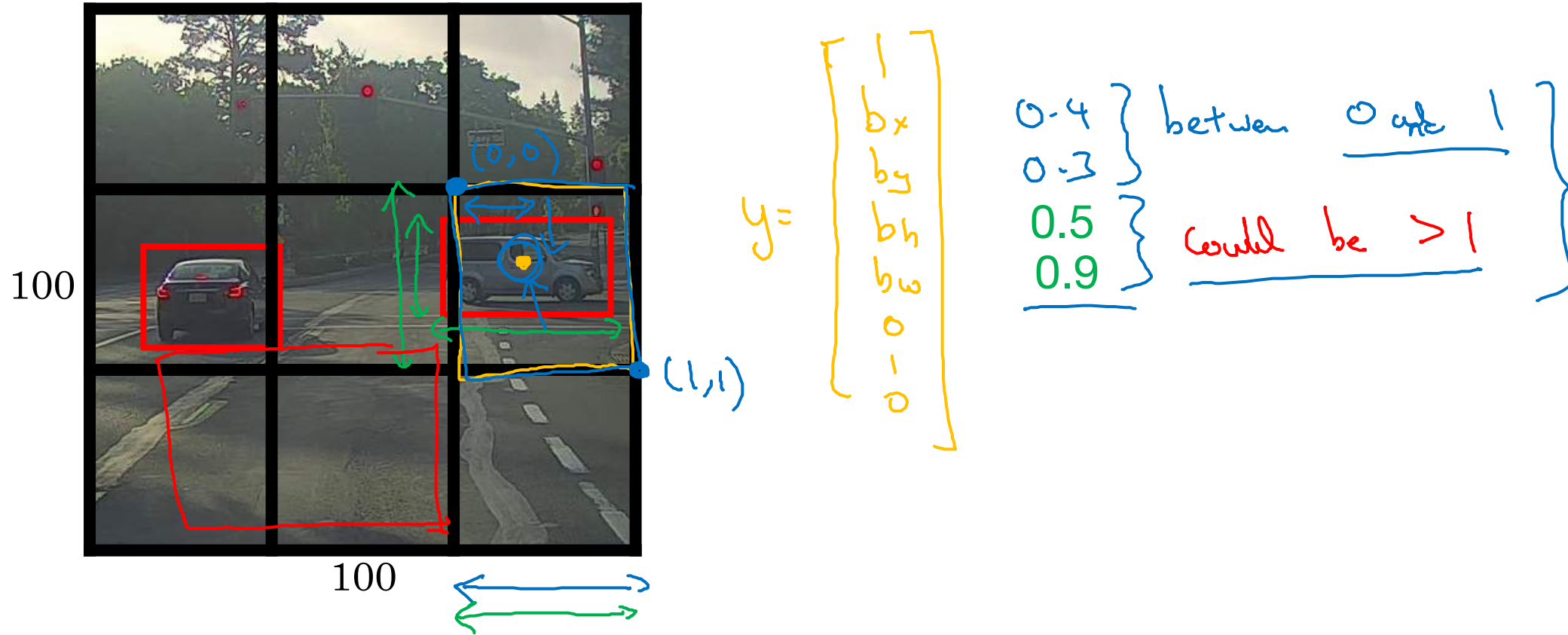
$3 \times 3 \times 8$



361

$\rightarrow 19 \times 19 \times 8$

Specify the bounding boxes



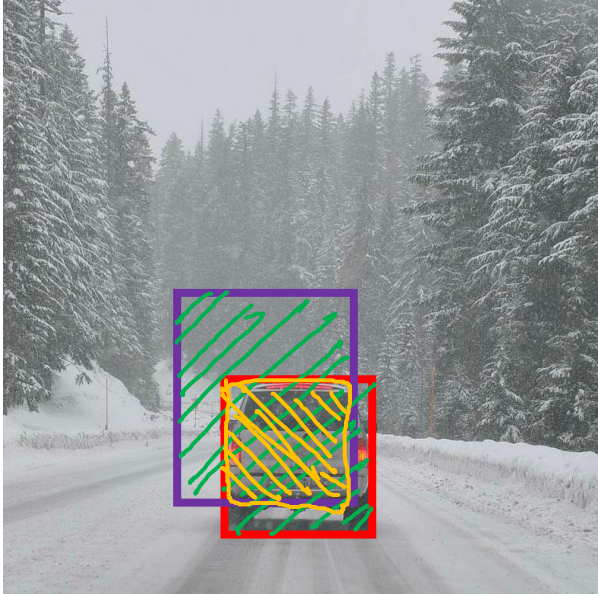


deeplearning.ai

Object Detection

Intersection
over union

Evaluating object localization



Intersection over Union (IoU)

$$= \frac{\text{size of } \text{[yellow hatched box]}}{\text{size of } \text{[green hatched box]}}$$

“Correct” if IoU $\geq$ 0.5 $\leftarrow$

0.6 $\leftarrow$

More generally, IoU is a measure of the overlap between two bounding boxes.



deeplearning.ai

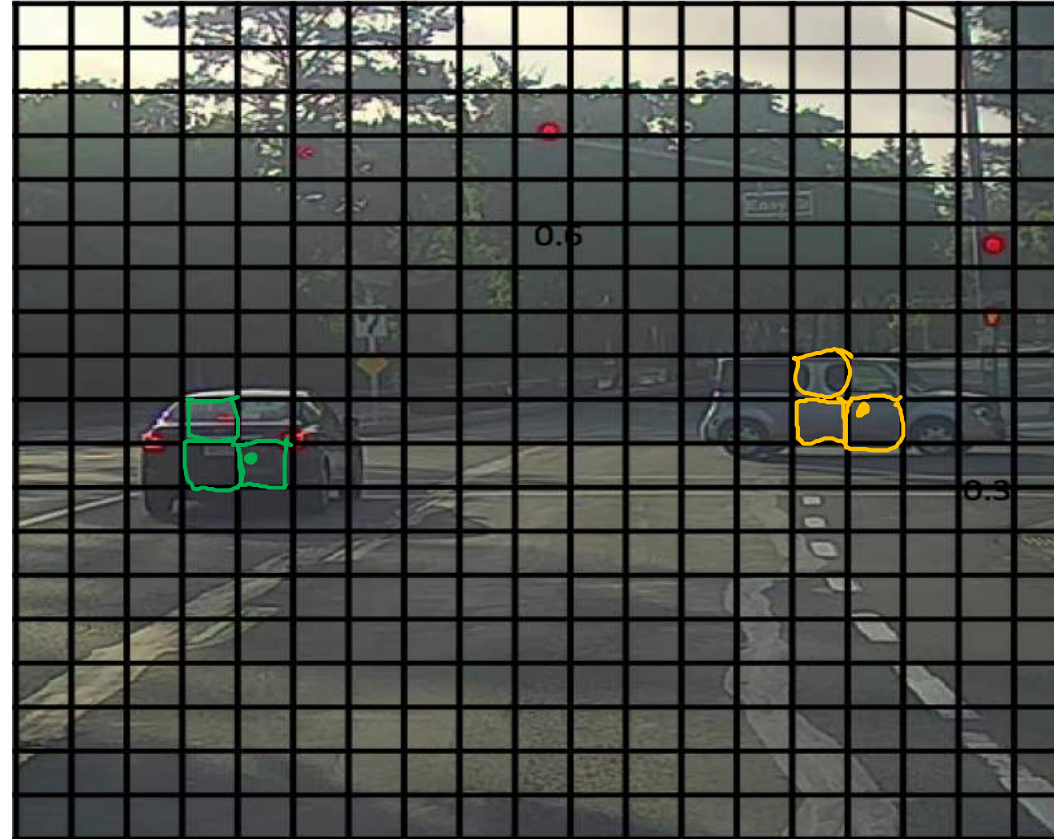
Object Detection

Non-max
suppression

Non-max suppression example

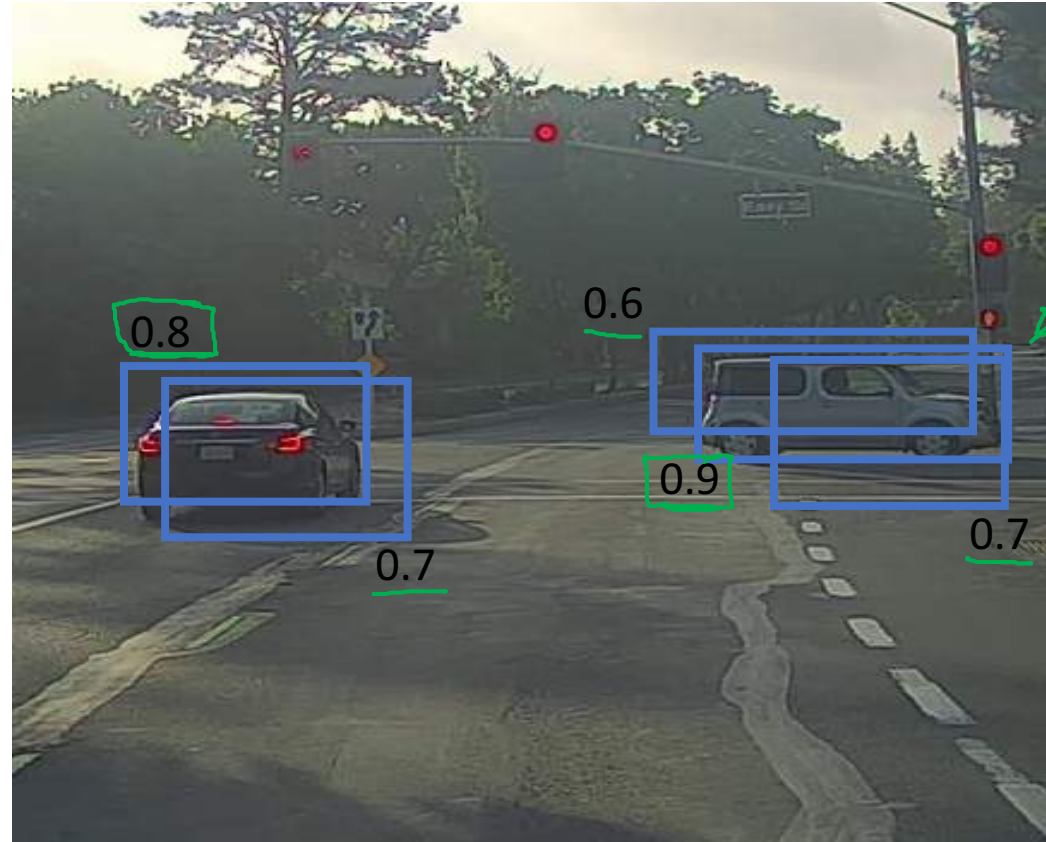


Non-max suppression example



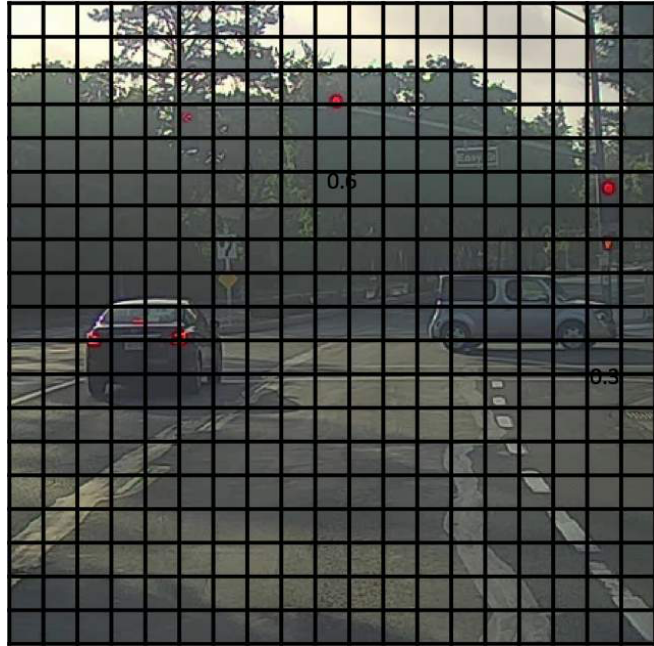
19x19

Non-max suppression example



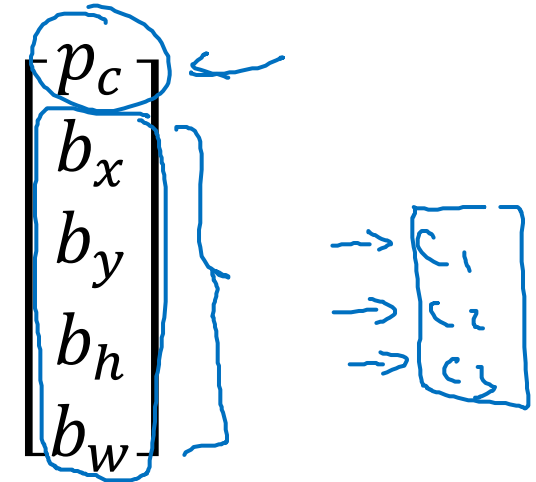
P_c

Non-max suppression algorithm



19x19

Each output prediction is:



Discard all boxes with $p_c \leq 0.6$

→ While there are any remaining boxes:

- Pick the box with the largest p_c
Output that as a prediction.
- Discard any remaining box with $\text{IoU} \geq 0.5$ with the box output in the previous step

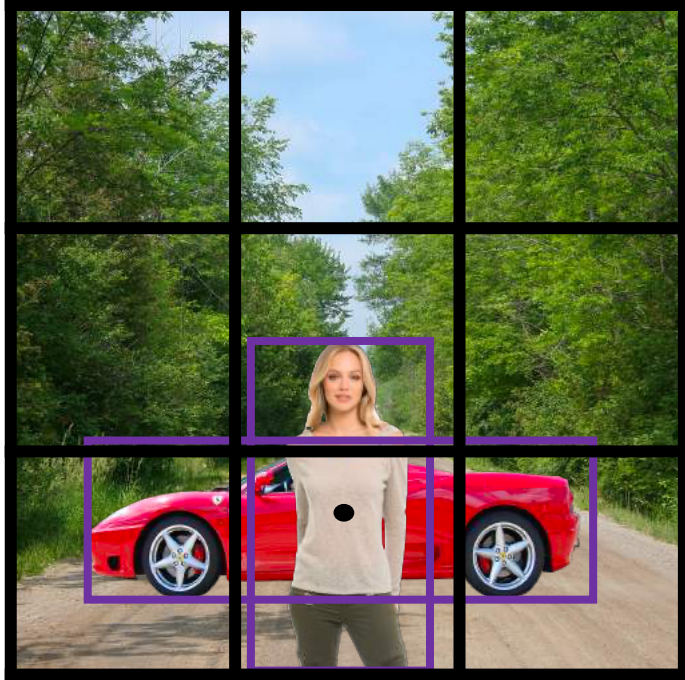


deeplearning.ai

Object Detection

Anchor boxes

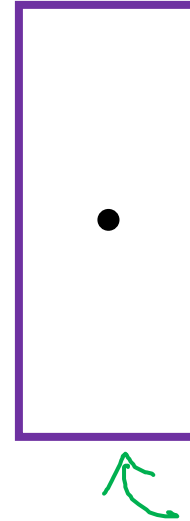
Overlapping objects:



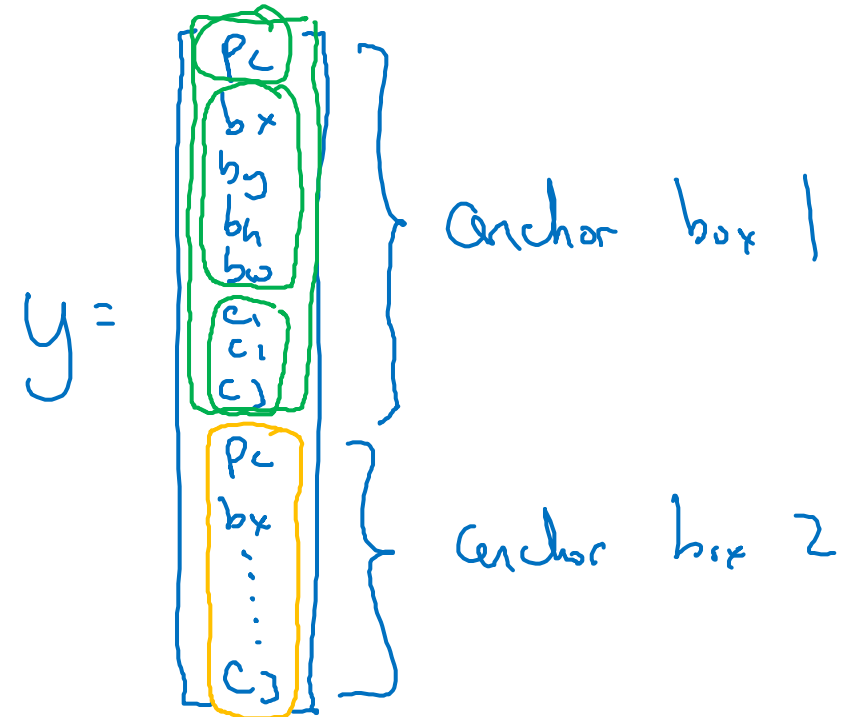
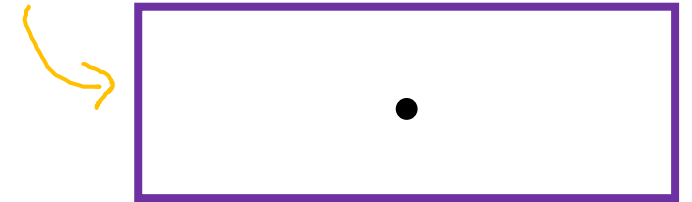
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Annotations: A green arrow points to p_c , a blue arrow points to b_x , and a blue bracket groups c_1, c_2, c_3 .

Anchor box 1:



Anchor box 2:

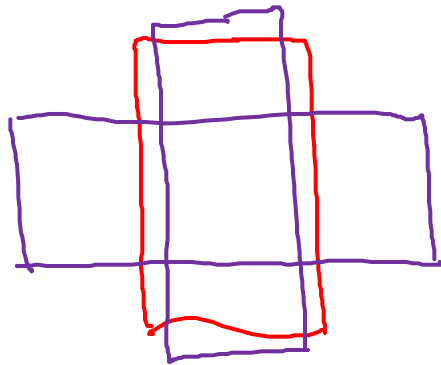


Anchor box algorithm

Previously:

Each object in training image is assigned to grid cell that contains that object's midpoint.

Output y :
 $3 \times 3 \times 8$



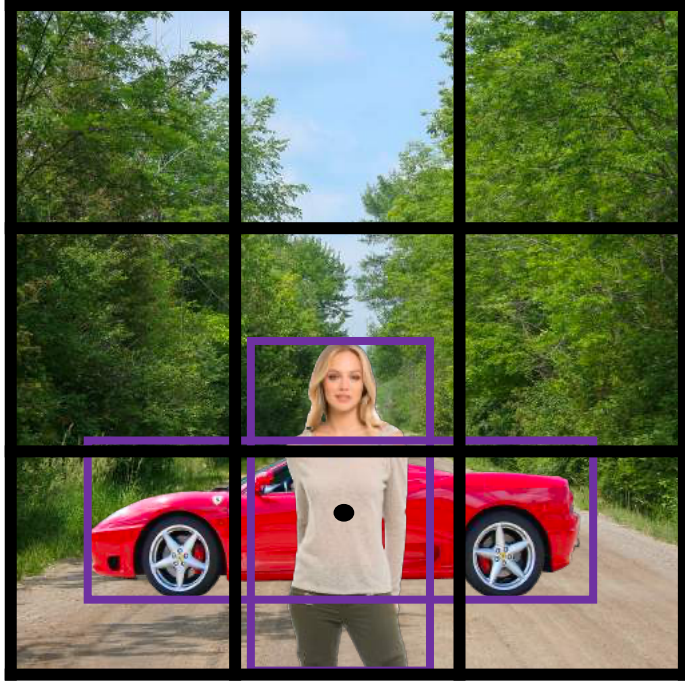
With two anchor boxes:

Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU.

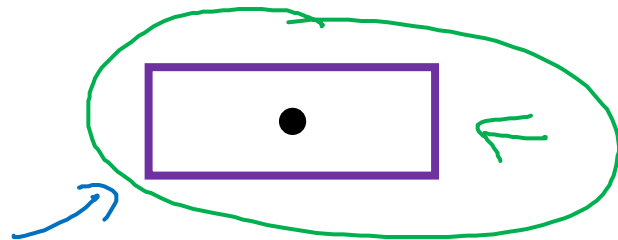
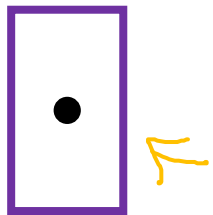
(grid cell, anchor box)

Output y :
 $3 \times 3 \times 16$
 $3 \times 3 \times 2 \times 8$

Anchor box example



Anchor box 1: Anchor box 2:



$y =$

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

1
bx
by
bh
bw
1
0
0
1
bx
by
bh
bw
0
0
0

car only?

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

anchor box 1

anchor box 2



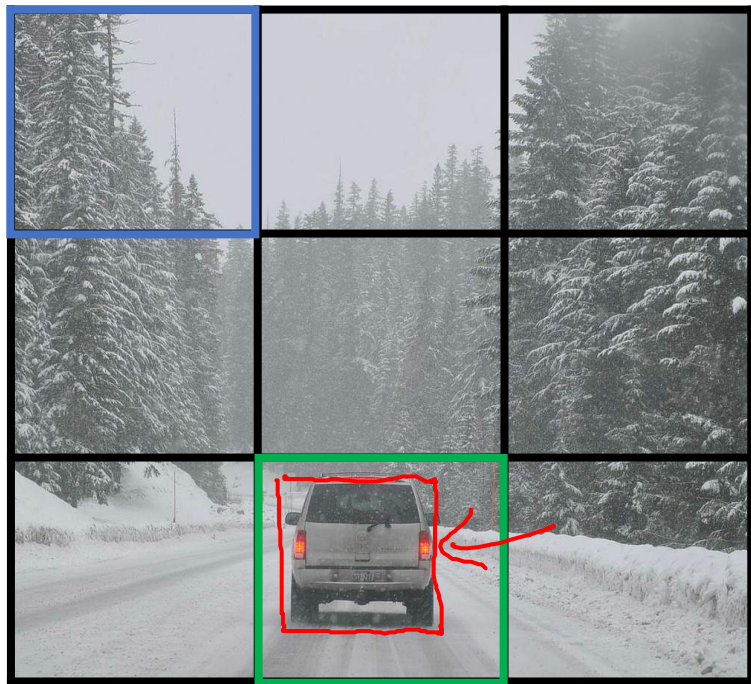
deeplearning.ai

Object Detection

Putting it together:
YOLO algorithm

Training

- 1 - pedestrian
- 2 - car ←
- 3 - motorcycle



$y =$



$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

y is $3 \times 3 \times 2 \times 8$

$19 \times 19 \times 16$

$19 \times 19 \times 40$

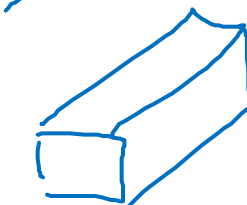
↑
#anchors

↑
 $5 + \#classes$



$100 \times 100 \times 3$

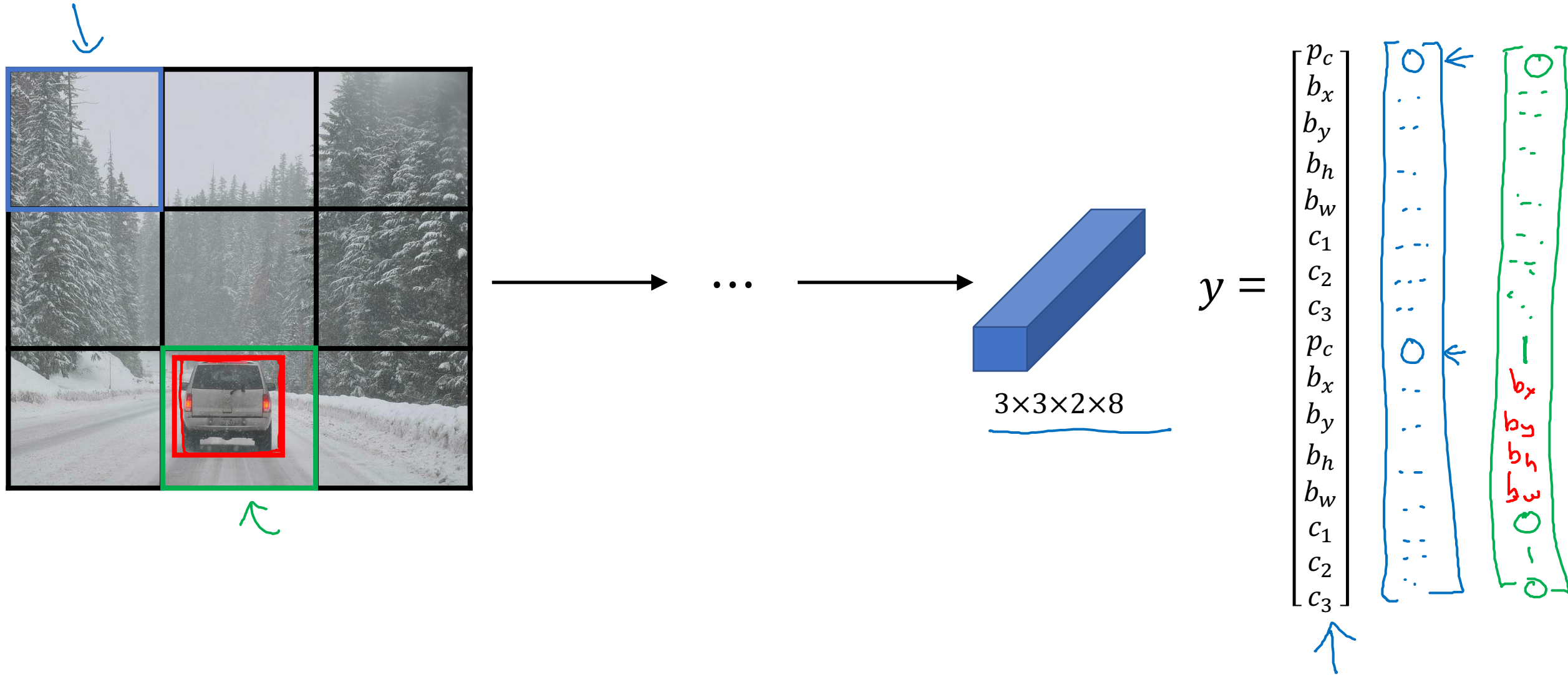
→ ConvNet →



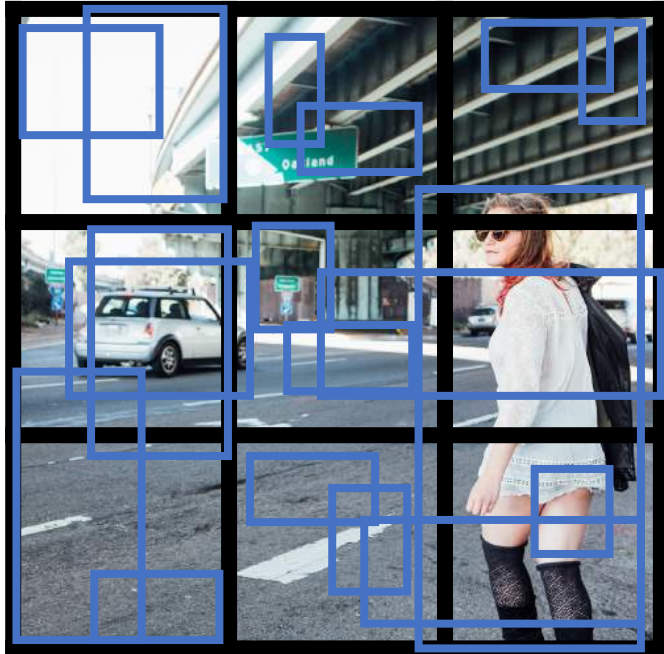
$3 \times 3 \times 16$

Andrew Ng

Making predictions



Outputting the non-max suppressed outputs



- For each grid cell, get 2 predicted bounding boxes.
- Get rid of low probability predictions.
- For each class (pedestrian, car, motorcycle) use non-max suppression to generate final predictions.

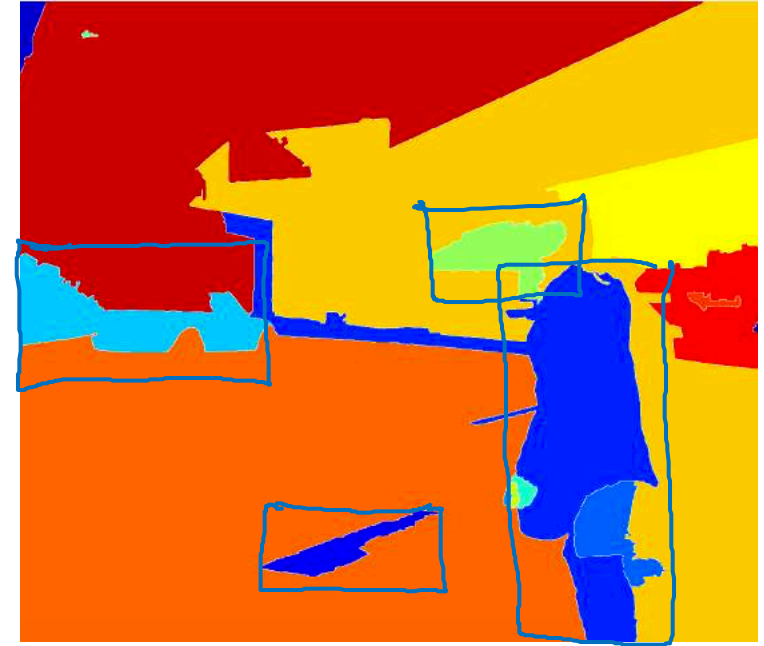
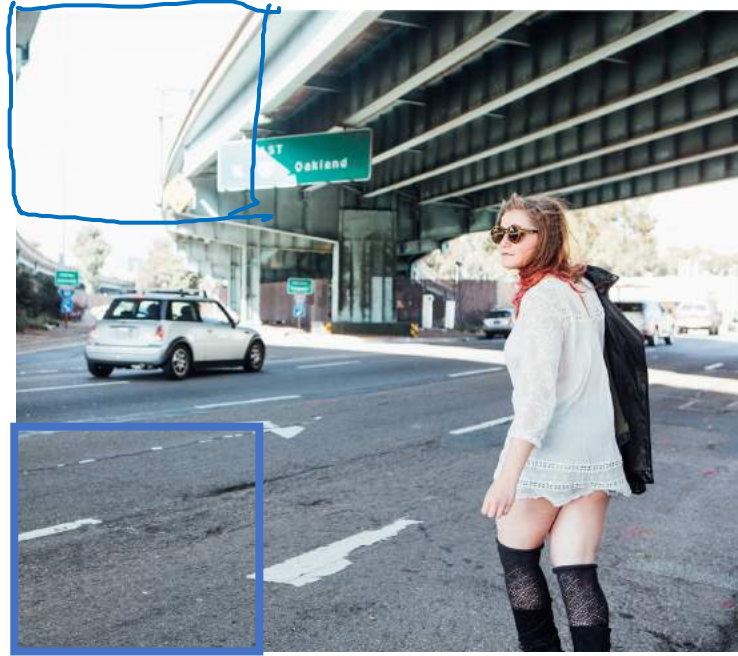
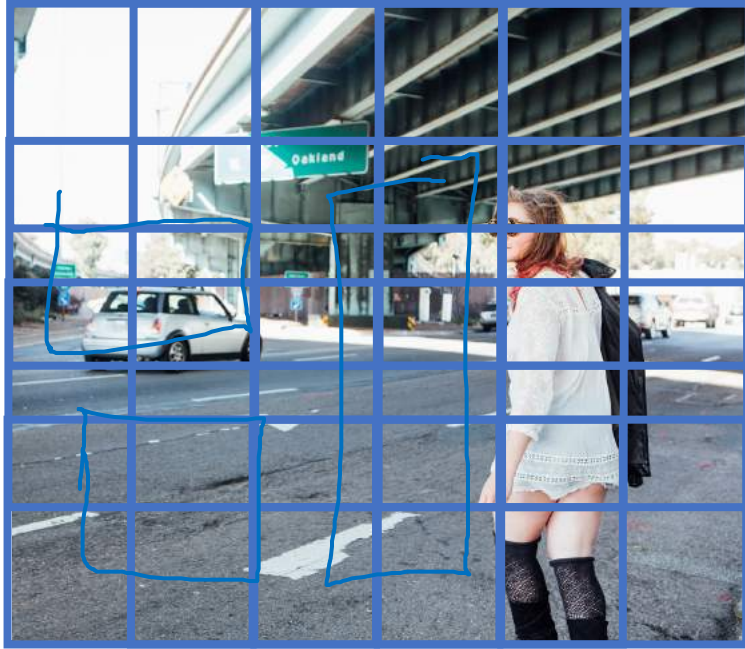


deeplearning.ai

Object Detection

Region proposals
(Optional)

Region proposal: R-CNN



Segmentation algorithm
 $\sim 2,000$

Faster algorithms

→ R-CNN: Propose regions. Classify proposed regions one at a time. Output label + bounding box. ←

Fast R-CNN: Propose regions. Use convolution implementation of sliding windows to classify all the proposed regions. ←

Faster R-CNN: Use convolutional network to propose regions.

[Girshik et. al, 2013. Rich feature hierarchies for accurate object detection and semantic segmentation]

[Girshik, 2015. Fast R-CNN]

[Ren et. al, 2016. Faster R-CNN: Towards real-time object detection with region proposal networks]

Andrew Ng



deeplearning.ai

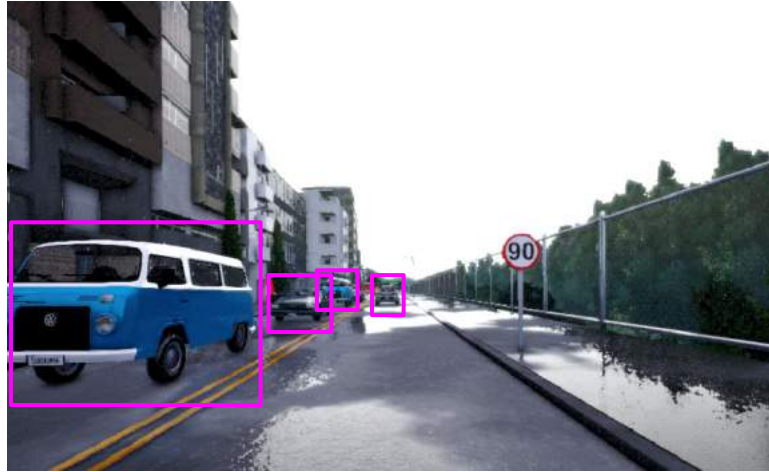
Convolutional Neural Networks

Semantic segmentation with U-Net

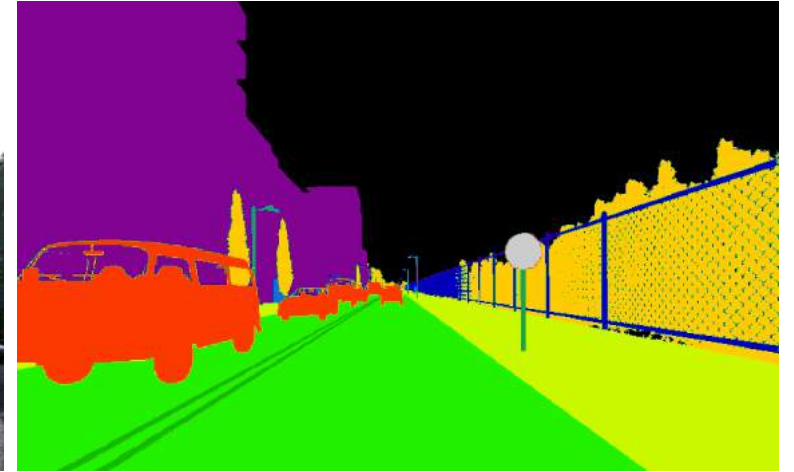
Object Detection vs. Semantic Segmentation



Input image

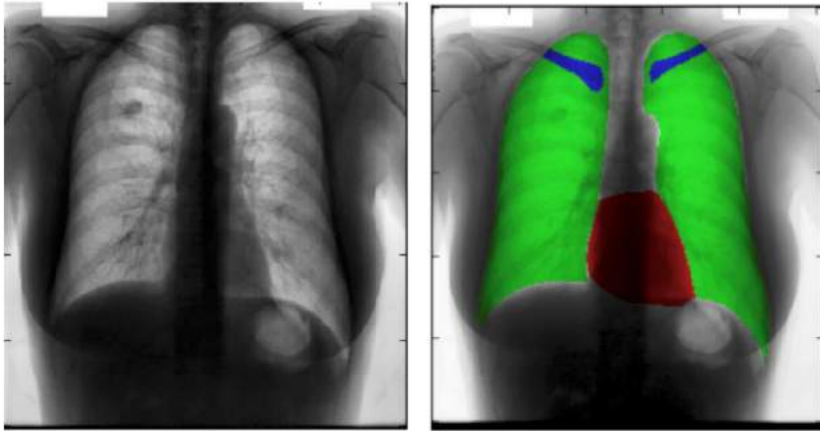


Object Detection

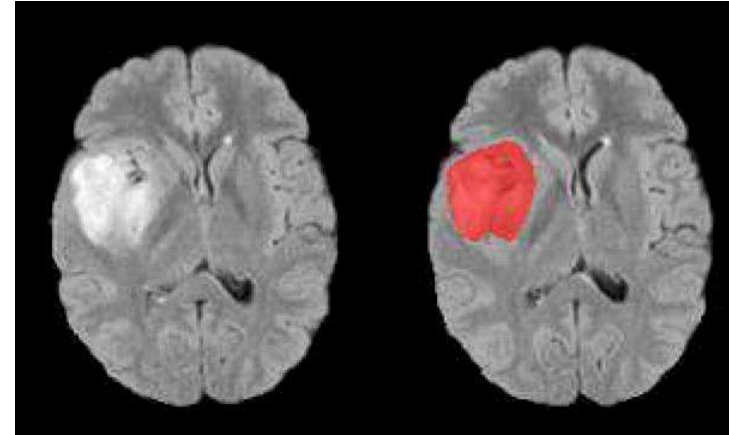


Semantic Segmentation

Motivation for U-Net



Chest X-Ray

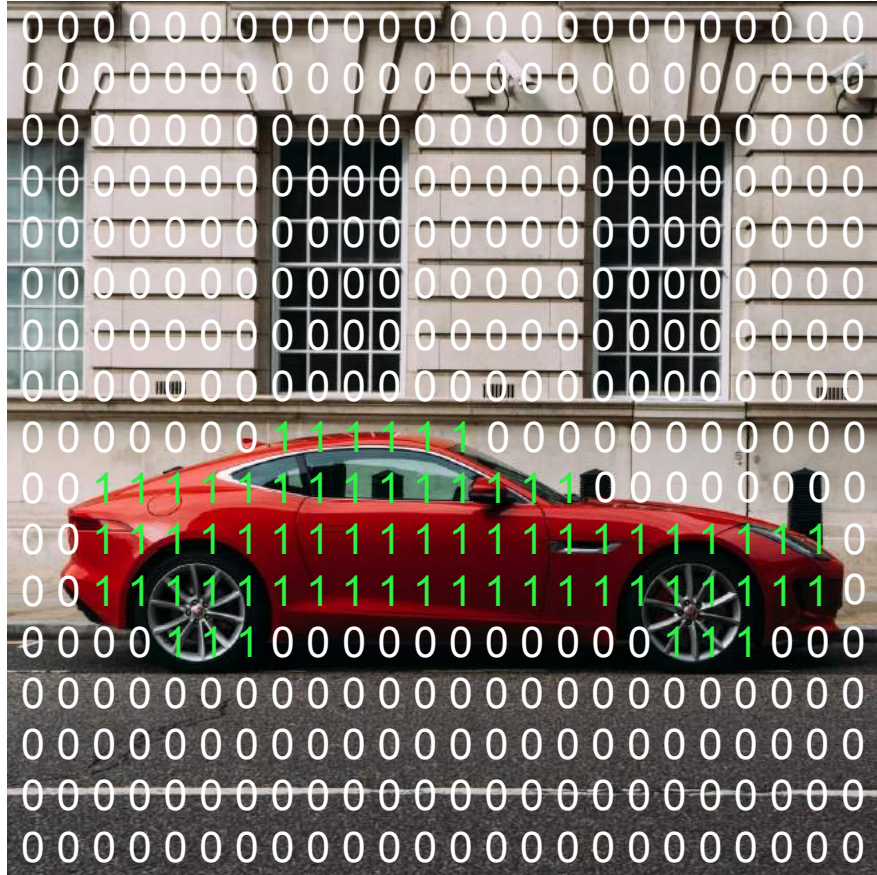


Brain MRI

[Novikov et al., 2017, Fully Convolutional Architectures for Multi-Class Segmentation in Chest Radiographs]

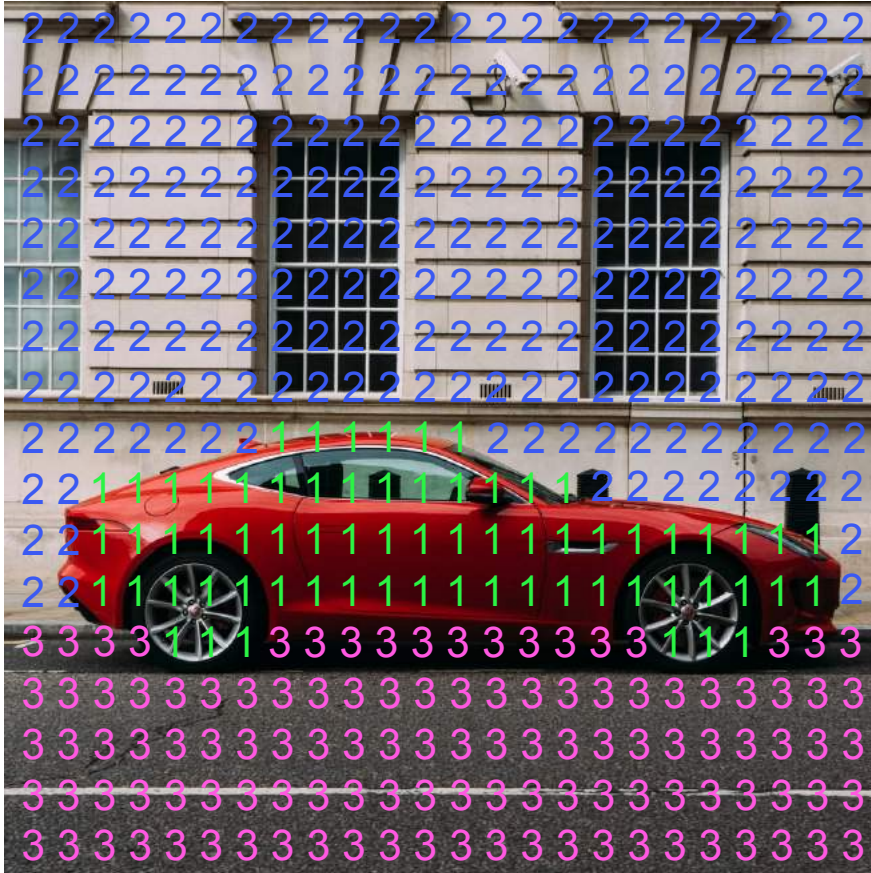
[Dong et al., 2017, Automatic Brain Tumor Detection and Segmentation Using U-Net Based Fully Convolutional Networks]

Per-pixel class labels

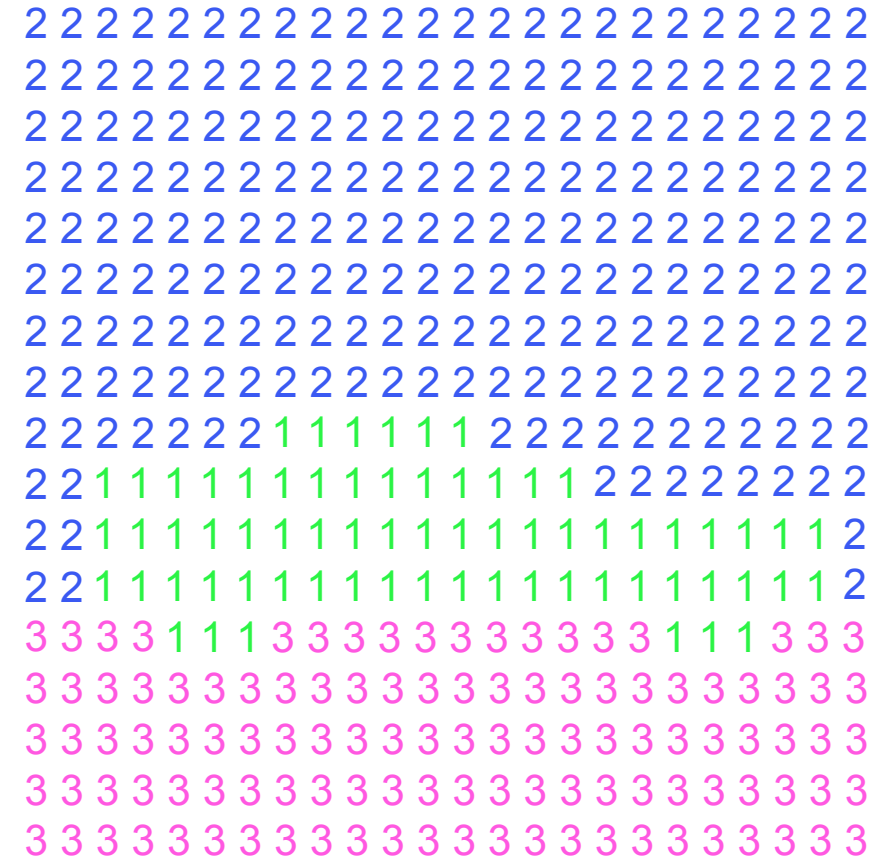


1. Car
0. Not Car

Per-pixel class labels

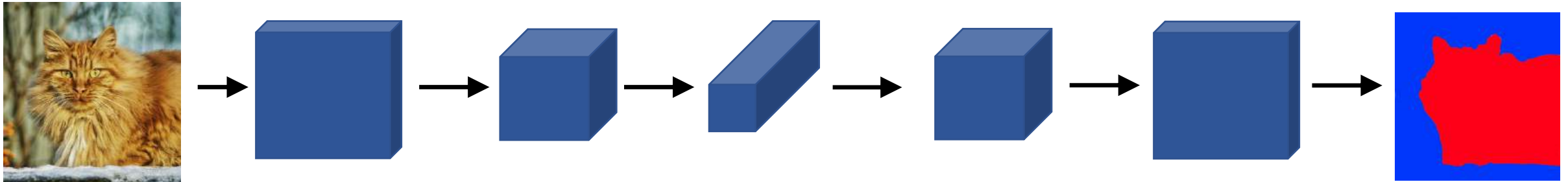


1. Car
2. Building
3. Road



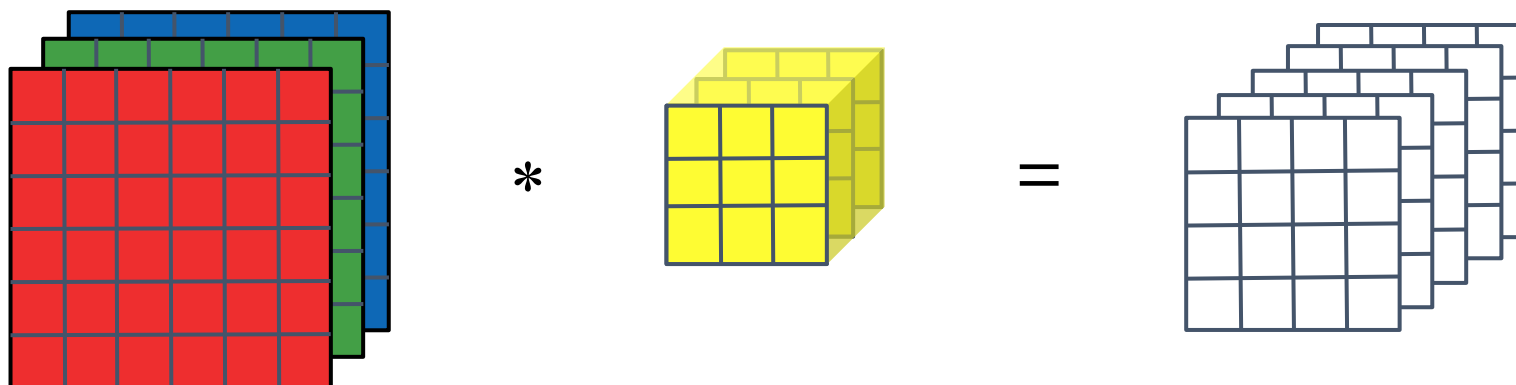
Segmentation Map

Deep Learning for Semantic Segmentation

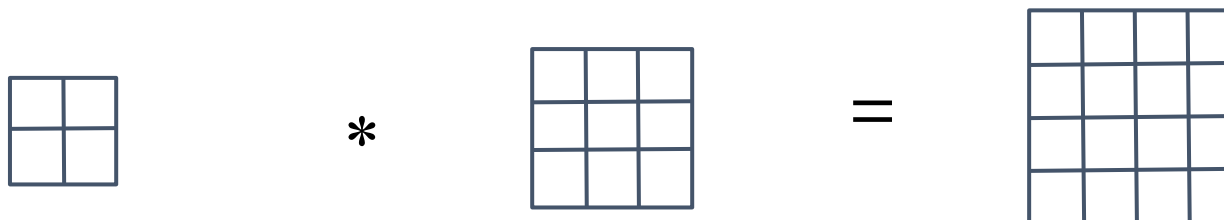


Transpose Convolution

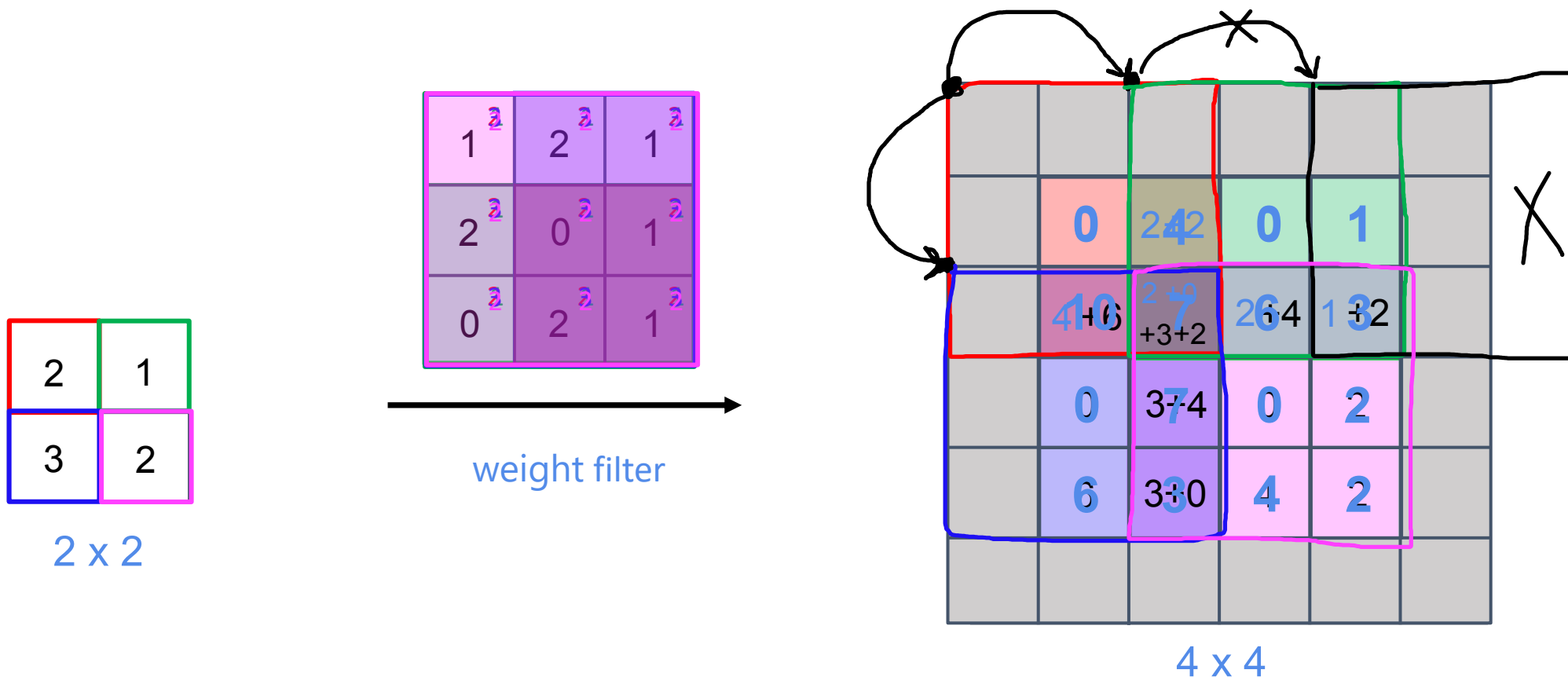
Normal Convolution



Transpose Convolution



Transpose Convolution

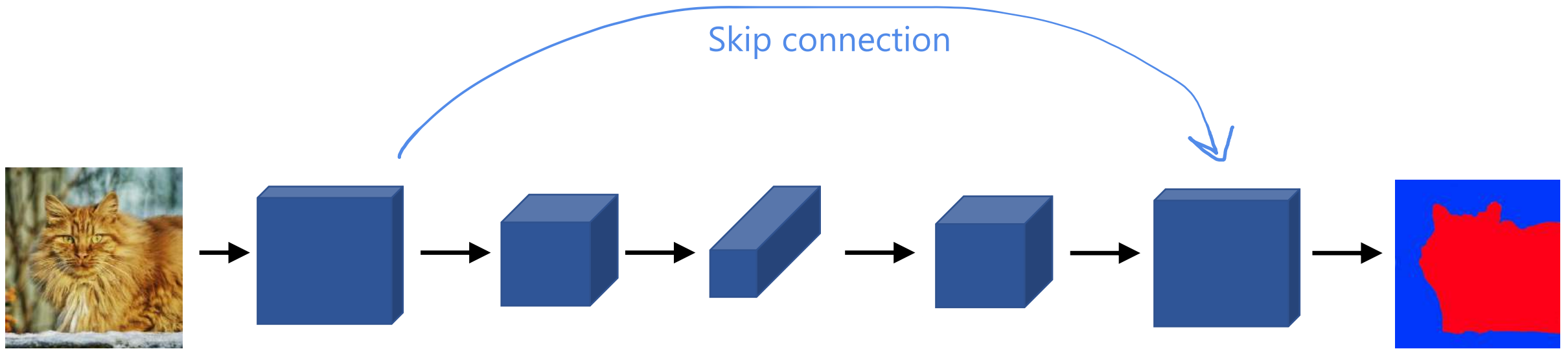


filter $f \times f = 3 \times 3$

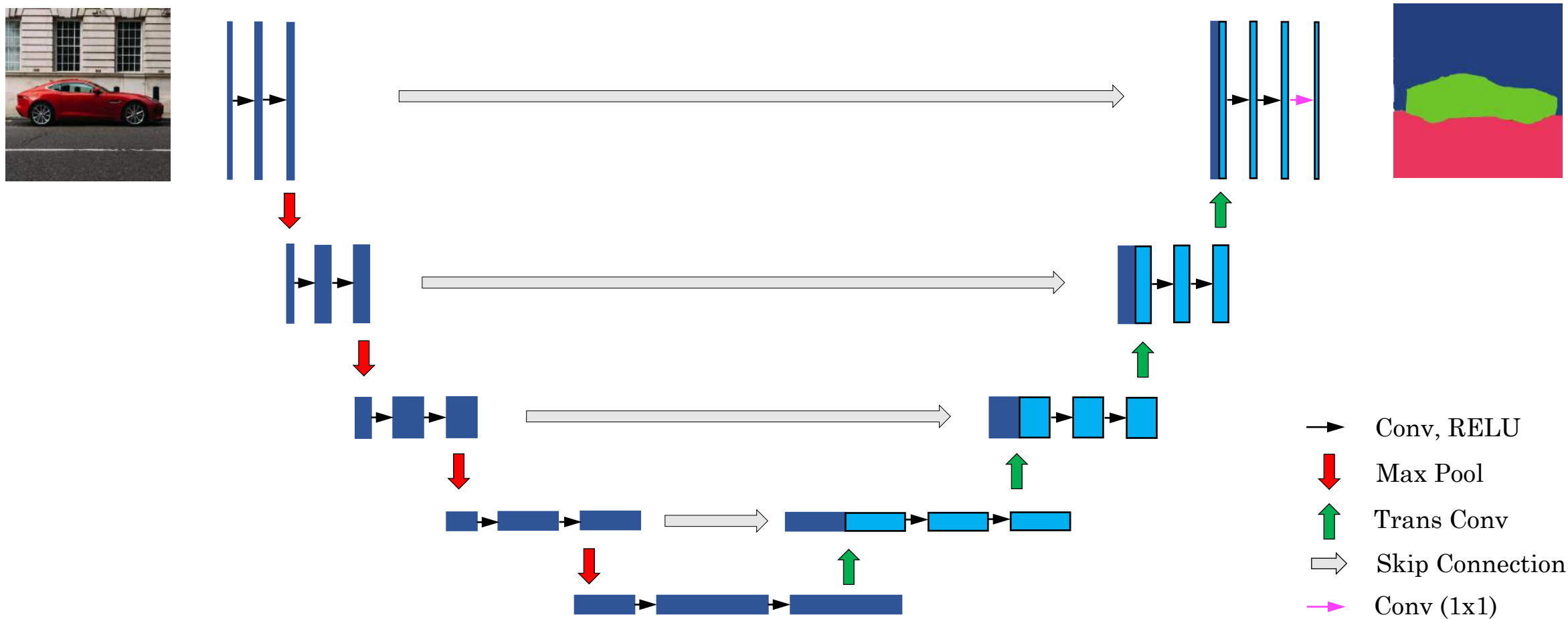
padding $p = 1$

stride $s = 2$

Deep Learning for Semantic Segmentation



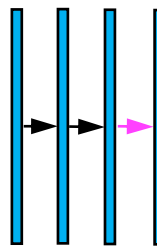
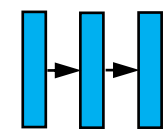
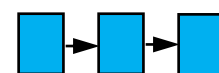
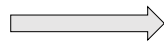
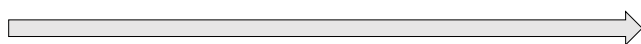
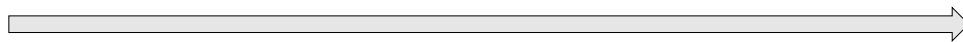
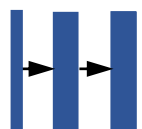
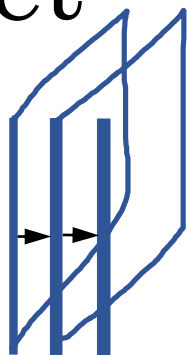
U-Net



U-Net



$h \times w \times 3$



$h \times w \times \# \text{ classes}$

- Conv, RELU
- ↓ Max Pool
- ↑ Trans Conv
- Skip Connection
- Conv (1x1)

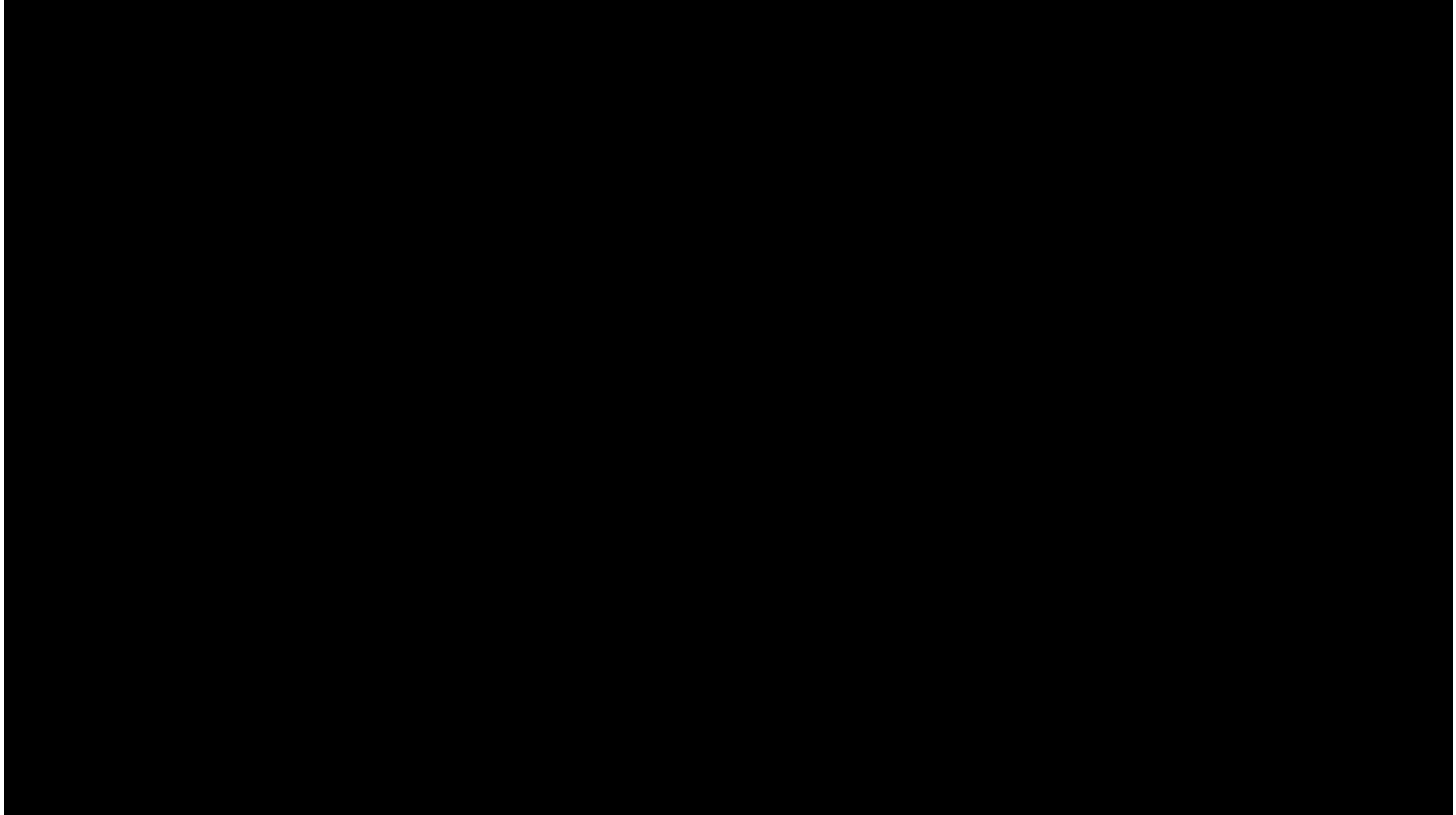


deeplearning.ai

Face recognition

What is face
recognition?

Face recognition



Face verification vs. face recognition

→ Verification

- Input image, name/ID
- Output whether the input image is that of the claimed person

1:1

99.0%

99.9

→ Recognition

- Has a database of K persons
- Get an input image
- Output ID if the image is any of the K persons (or “not recognized”)

1:K

K=100 ←

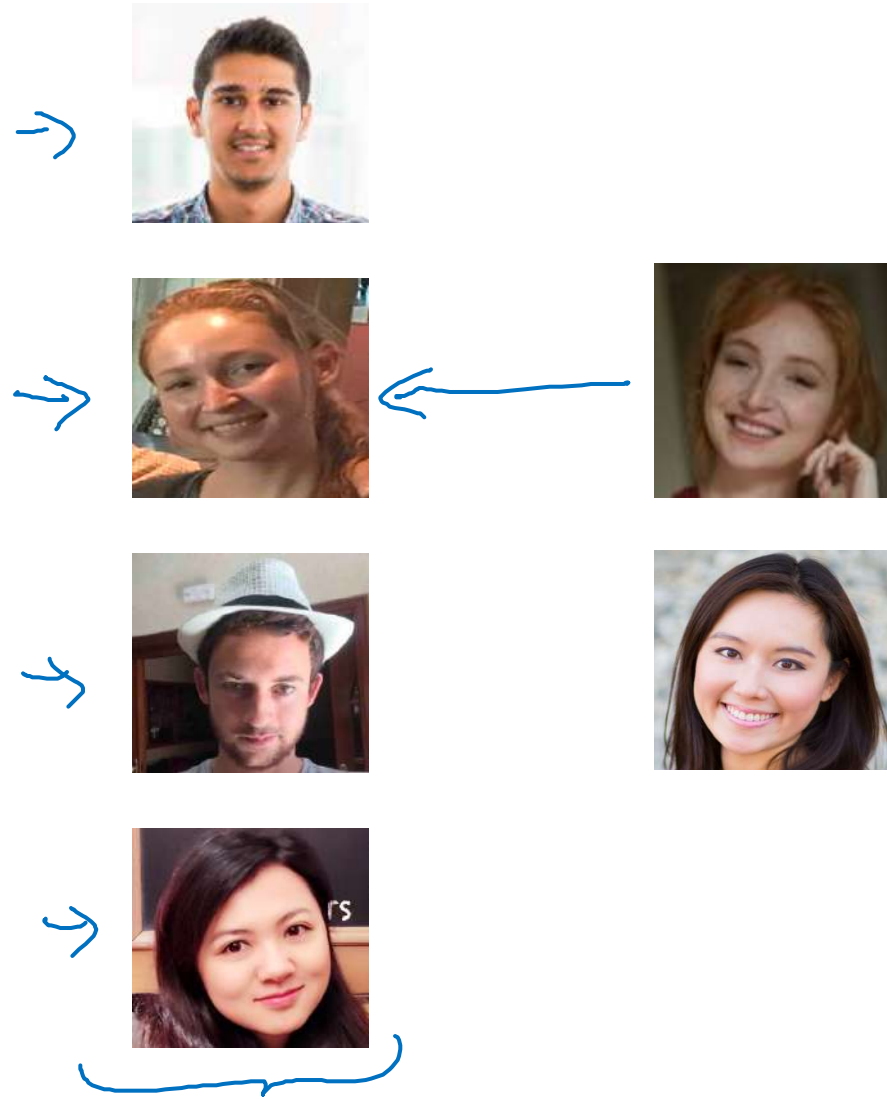


deeplearning.ai

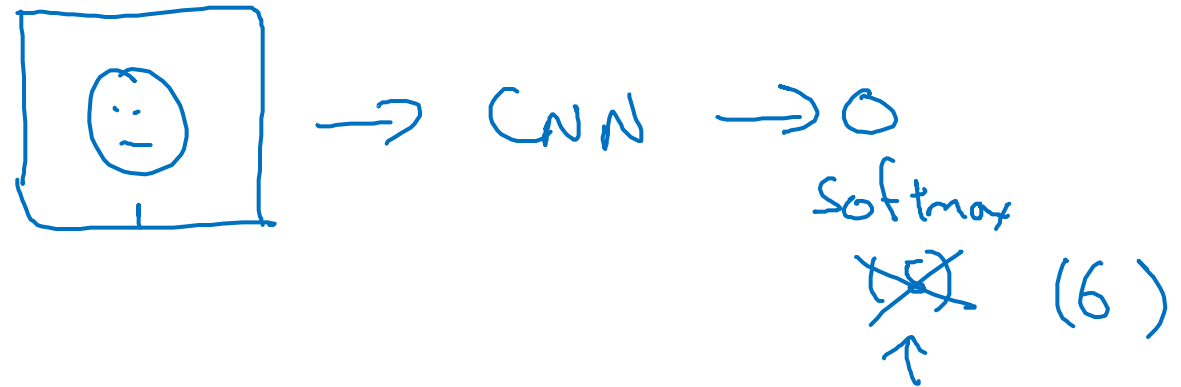
Face recognition

One-shot learning

One-shot learning



Learning from one example to recognize the person again



Learning a “similarity” function

→ $d(\text{img1}, \text{img2})$ = degree of difference between images

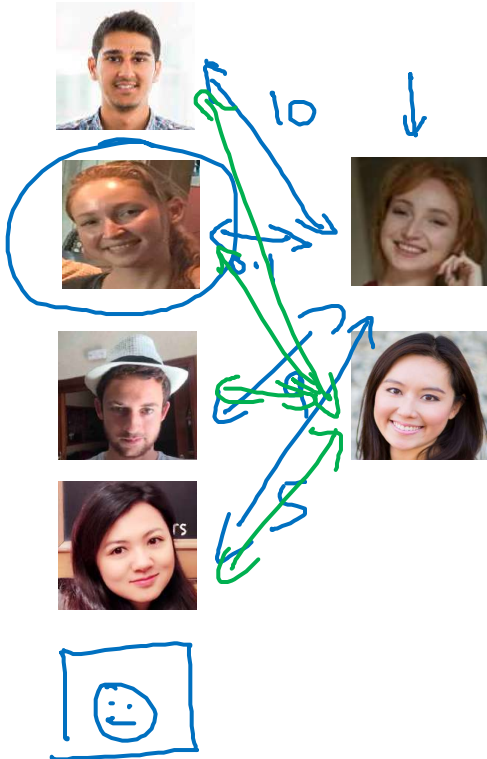
If $d(\text{img1}, \text{img2}) \leq \tau$

$> \tau$

“same”

“different”

} Verification.



$d(\text{img1}, \text{img2})$

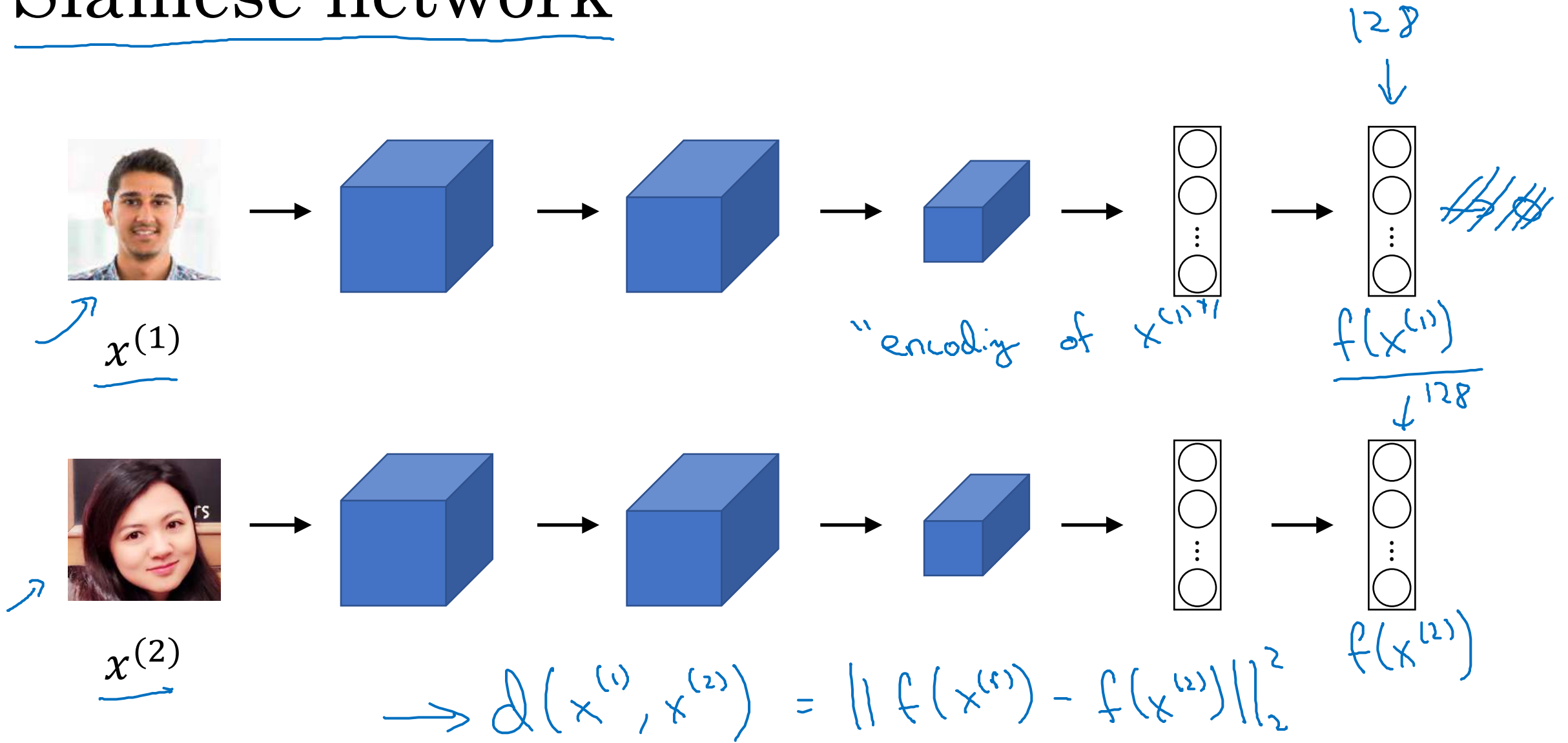


deeplearning.ai

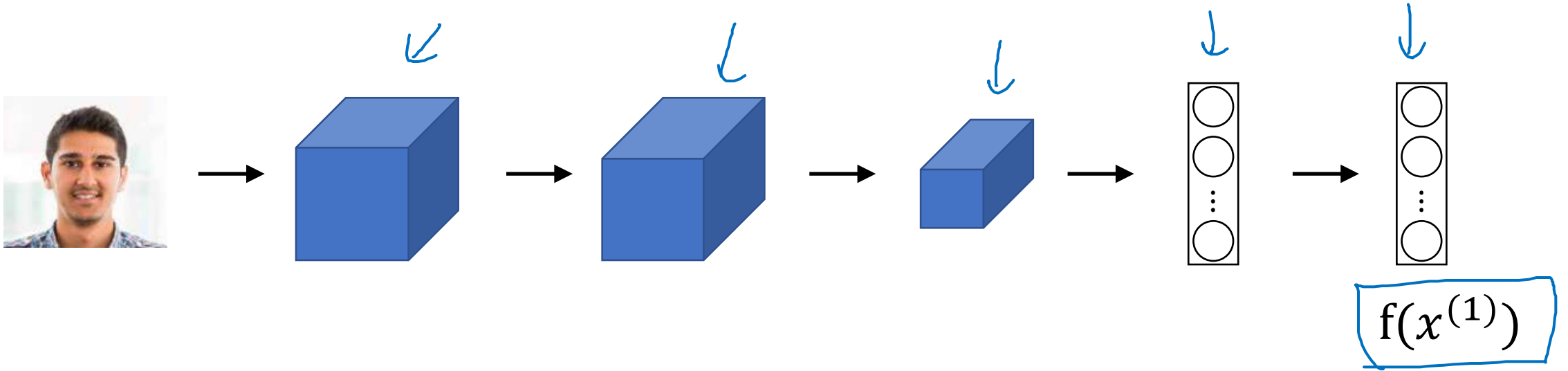
Face recognition

Siamese network

Siamese network



Goal of learning



Parameters of NN define an encoding $f(x^{(i)})$

Learn parameters so that:

If $x^{(i)}, x^{(j)}$ are the same person, $\|f(x^{(i)}) - f(x^{(j)})\|^2$ is small.

If $x^{(i)}, x^{(j)}$ are different persons, $\|f(x^{(i)}) - f(x^{(j)})\|^2$ is large.

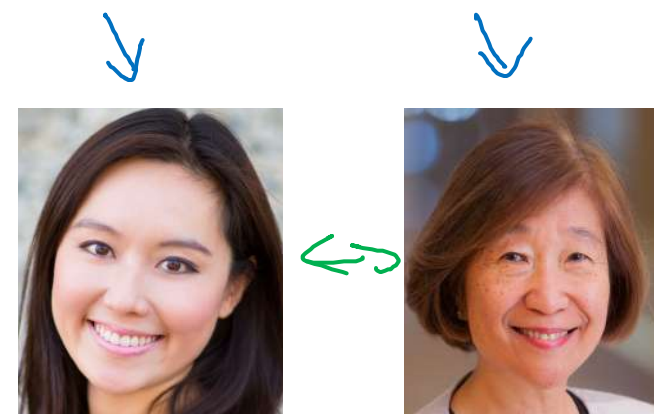
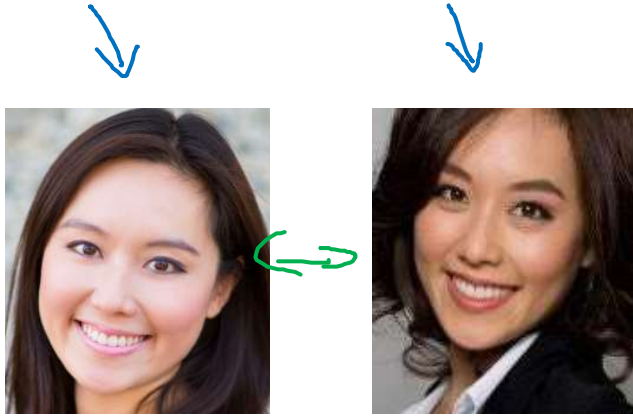


deeplearning.ai

Face recognition

Triplet loss

Learning Objective



Anchor

Positive

Anchor

Negative

A

$$d(A, P) = 0.5$$

Want:

$$\underbrace{\|f(A) - f(P)\|^2}_{d(A, P)} + \underline{\alpha} \leq$$

A

$$d(A, N) = \cancel{0.5} \quad 0.7$$

$$\underbrace{\|f(A) - f(N)\|^2}_{d(A, N)}$$

$$\underbrace{\|f(A) - f(P)\|^2}_0 - \underbrace{\|f(A) - f(N)\|^2}_0 + \underline{\alpha} \leq \underline{0} \quad \text{margin}$$

$$f(\text{img}) = \vec{0}$$

Loss function

Given 3 images

A, P, N :

$$\underline{L(A, P, N)} = \max \left(\underbrace{\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha}_{\geq 0}, 0 \right)$$

$$J = \sum_{i=1}^m L(A^{(i)}, P^{(i)}, N^{(i)})$$

A, P
↑ ↑

Training set: 10k pictures of 1k persons

Choosing the triplets A,P,N

During training, if A,P,N are chosen randomly,
 $d(A, P) + \alpha \leq d(A, N)$ is easily satisfied.

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$$

Choose triplets that're "hard" to train on.

$$\frac{d(A, P)}{d(A, P)} + \alpha \leq \frac{d(A, N)}{d(A, N)}$$

↓ ↑

Face Net
Deep Face

Training set using triplet loss

Anchor



⋮



Positive



⋮



Negative



⋮



$$d(x^{(i)}, x^{(j)})$$

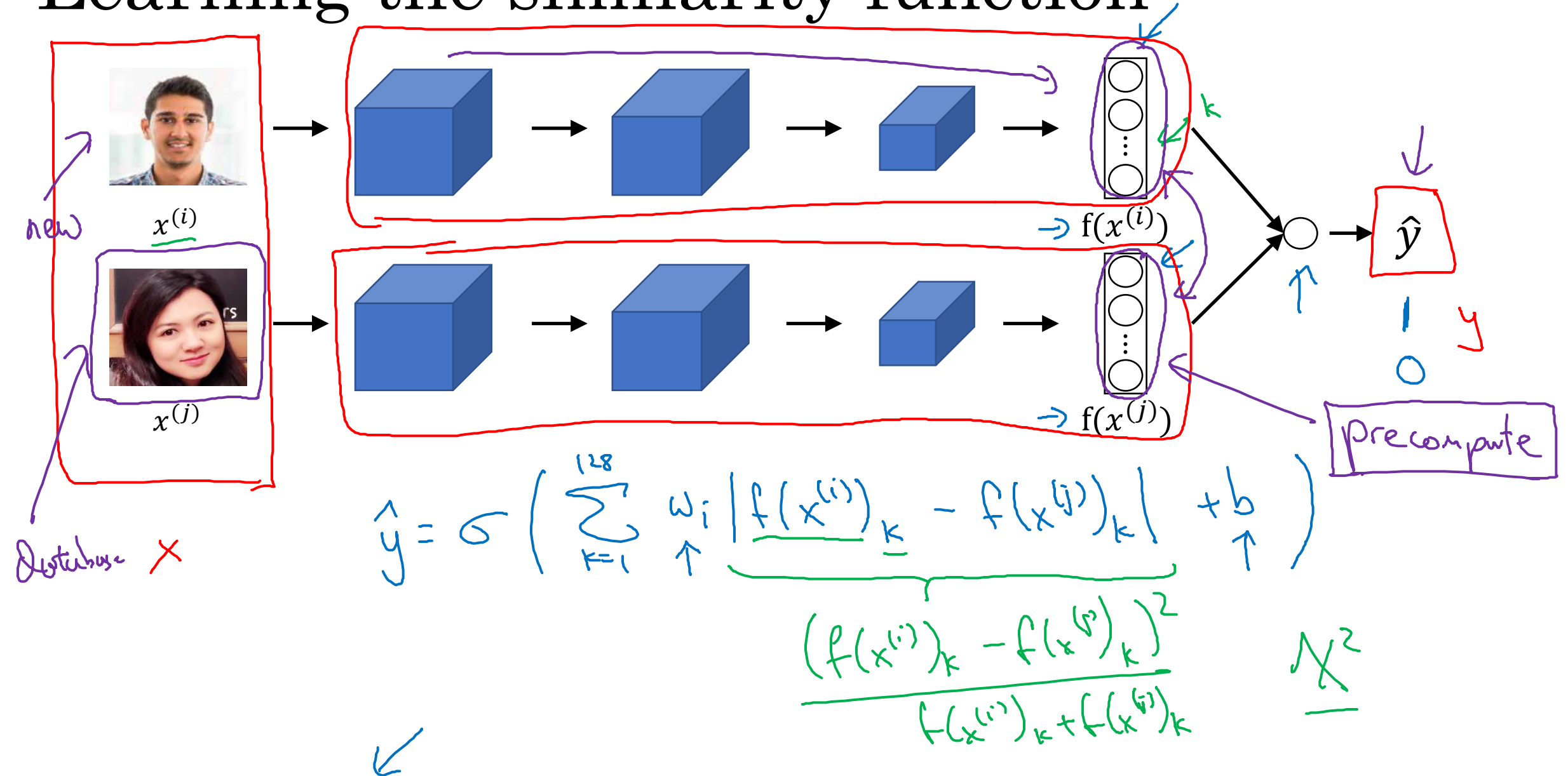


deeplearning.ai

Face recognition

Face verification and binary classification

Learning the similarity function



Face verification supervised learning

x		y	
		1	"Same"
		0	"Different"
		0	
		1	



deeplearning.ai

Neural Style Transfer

What is neural style
transfer?

Neural style transfer



Content (c)



Style (s)



Generated image (G)



Content (c)



Style (s)



Generated image (G)

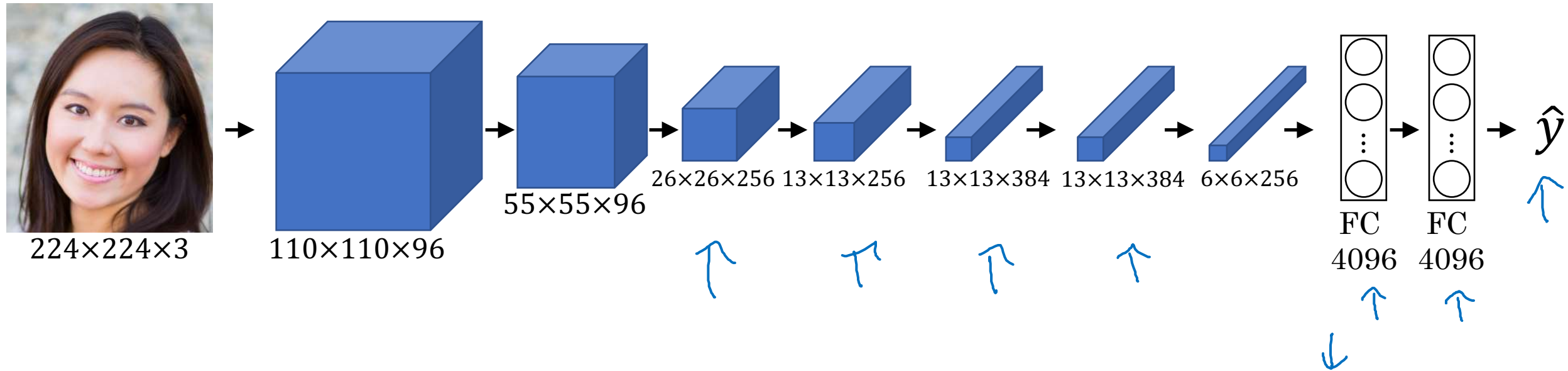


deeplearning.ai

Neural Style Transfer

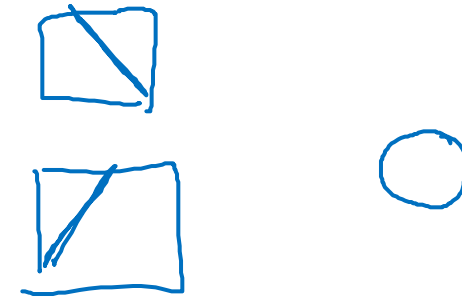
What are deep
ConvNets learning?

Visualizing what a deep network is learning

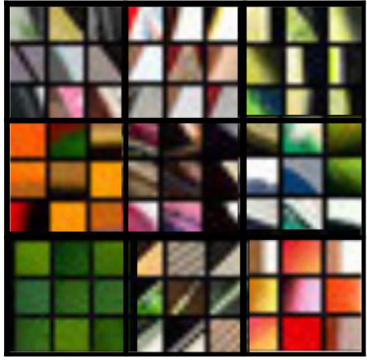


Pick a unit in layer 1. Find the nine image patches that maximize the unit's activation.

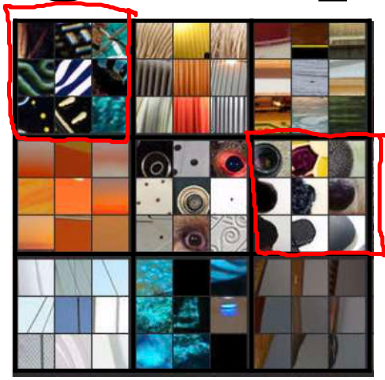
Repeat for other units.



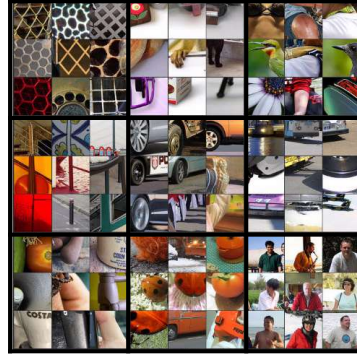
Visualizing deep layers



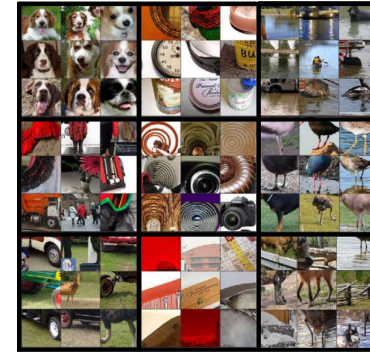
Layer 1



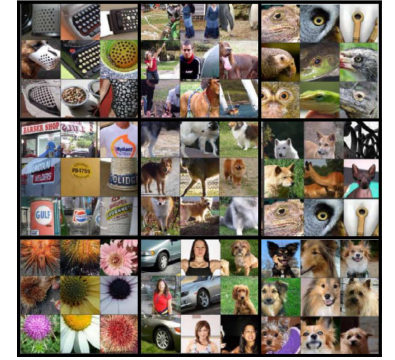
Layer 2



Layer 3

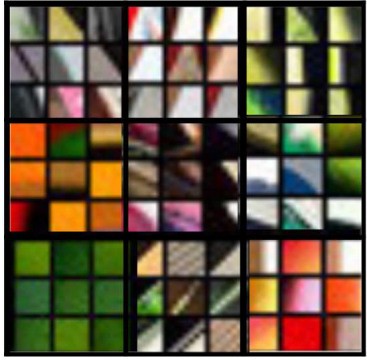


Layer 4



Layer 5

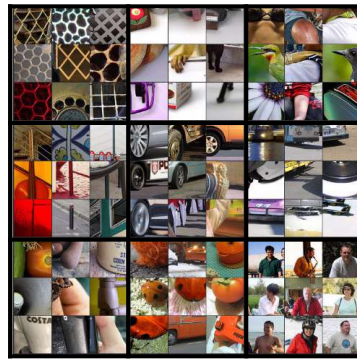
Visualizing deep layers: Layer 1



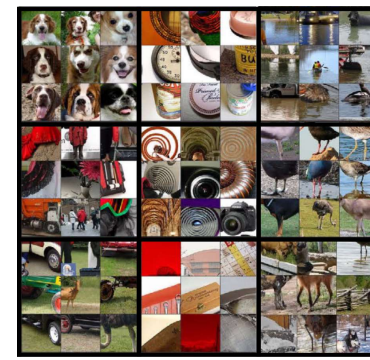
Layer 1



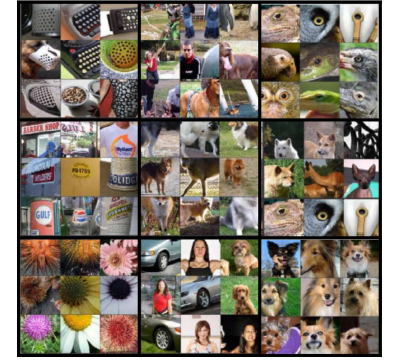
Layer 2



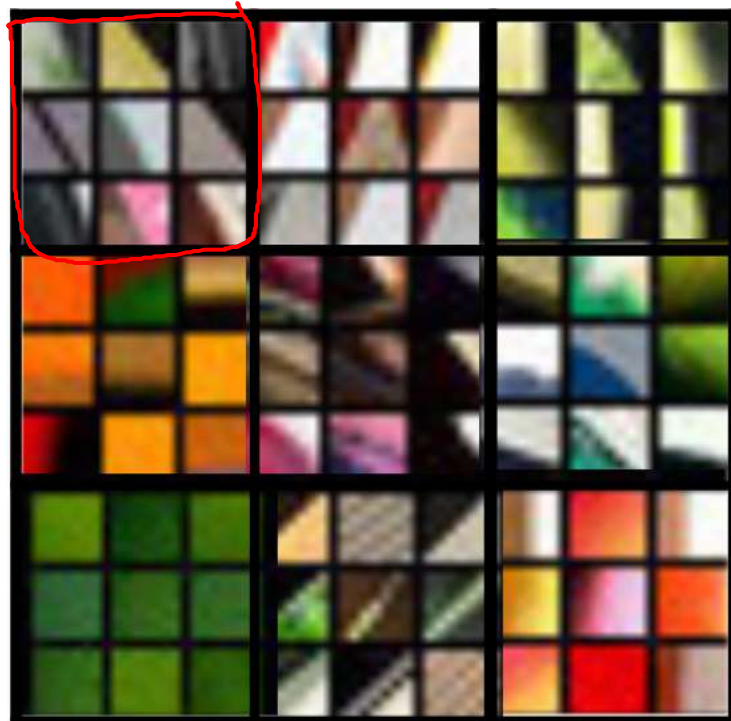
Layer 3



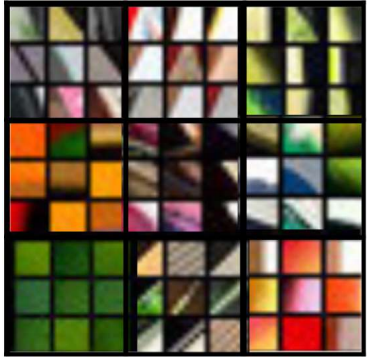
Layer 4



Layer 5



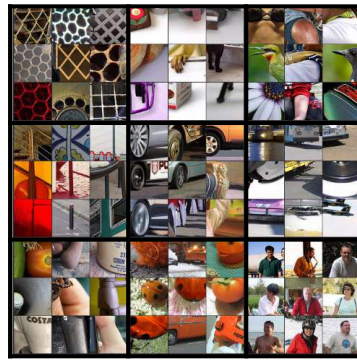
Visualizing deep layers: Layer 2



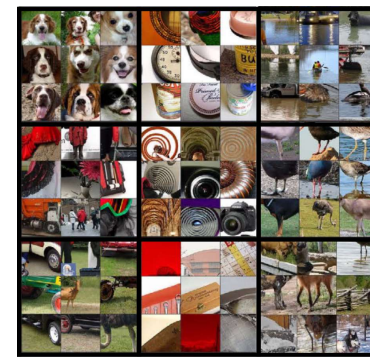
Layer 1



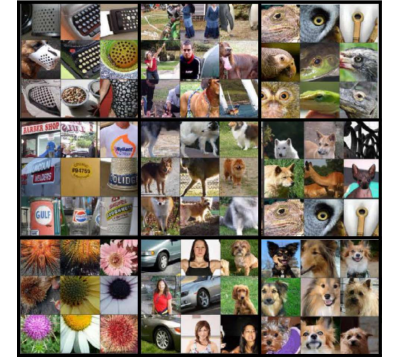
Layer 2



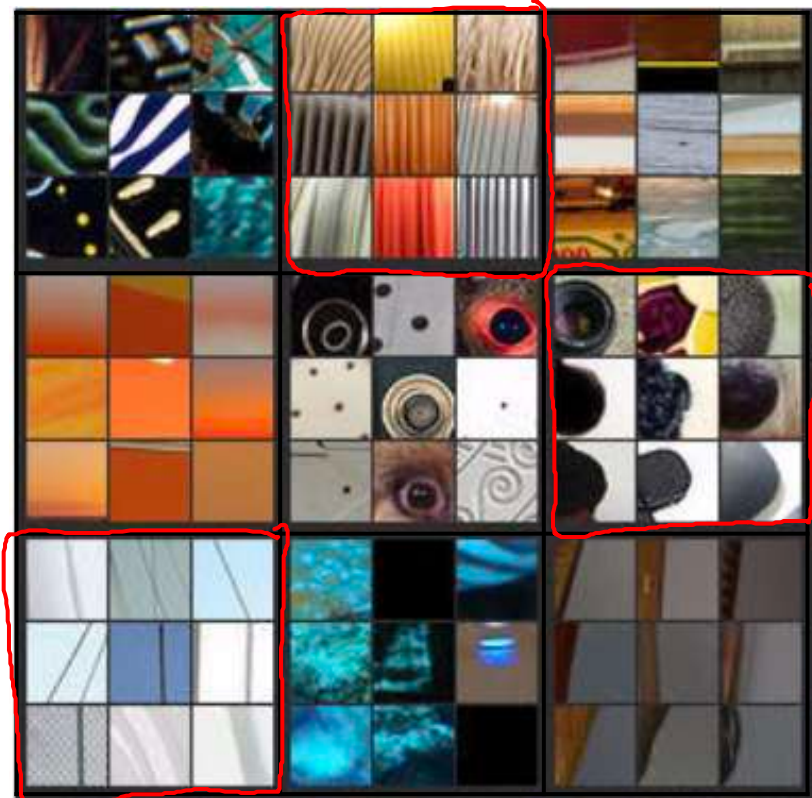
Layer 3



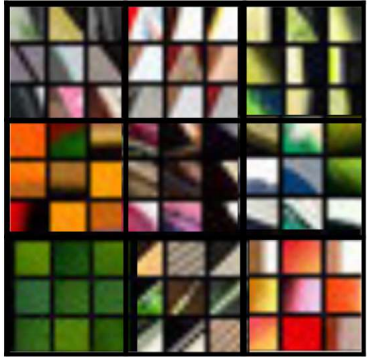
Layer 4



Layer 5



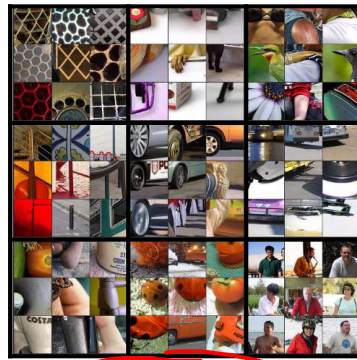
Visualizing deep layers: Layer 3



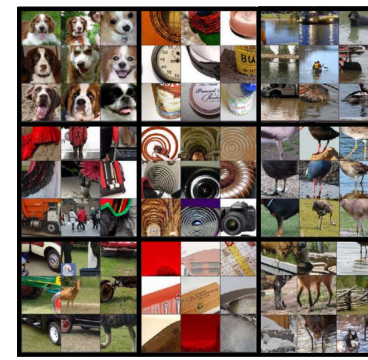
Layer 1



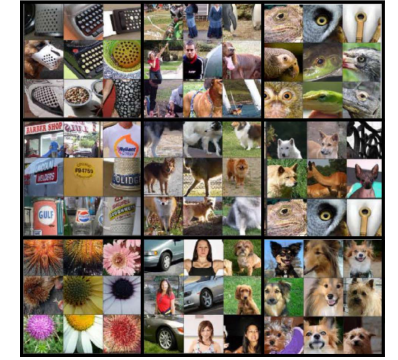
Layer 2



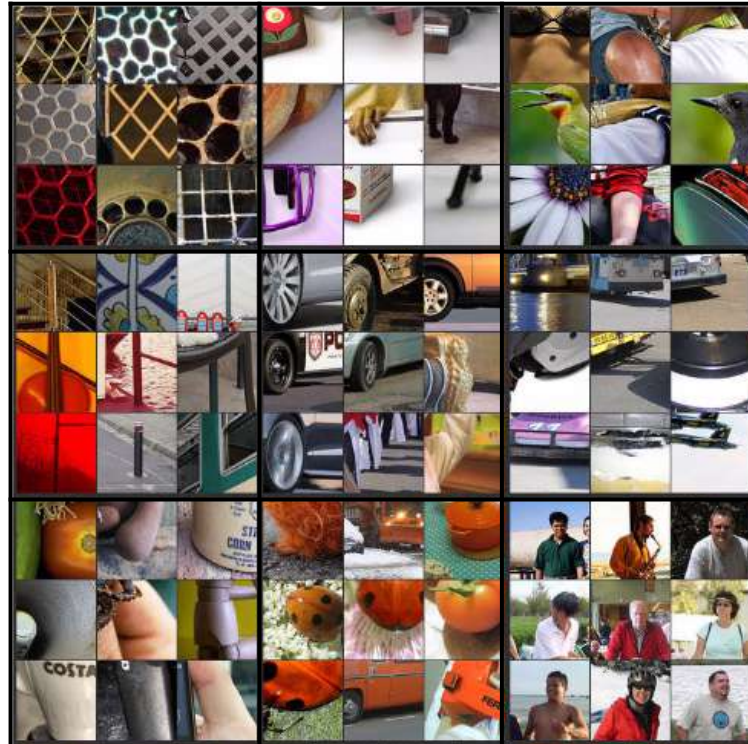
Layer 3



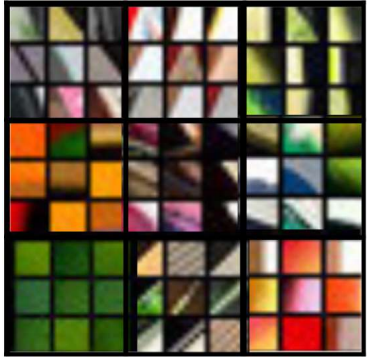
Layer 4



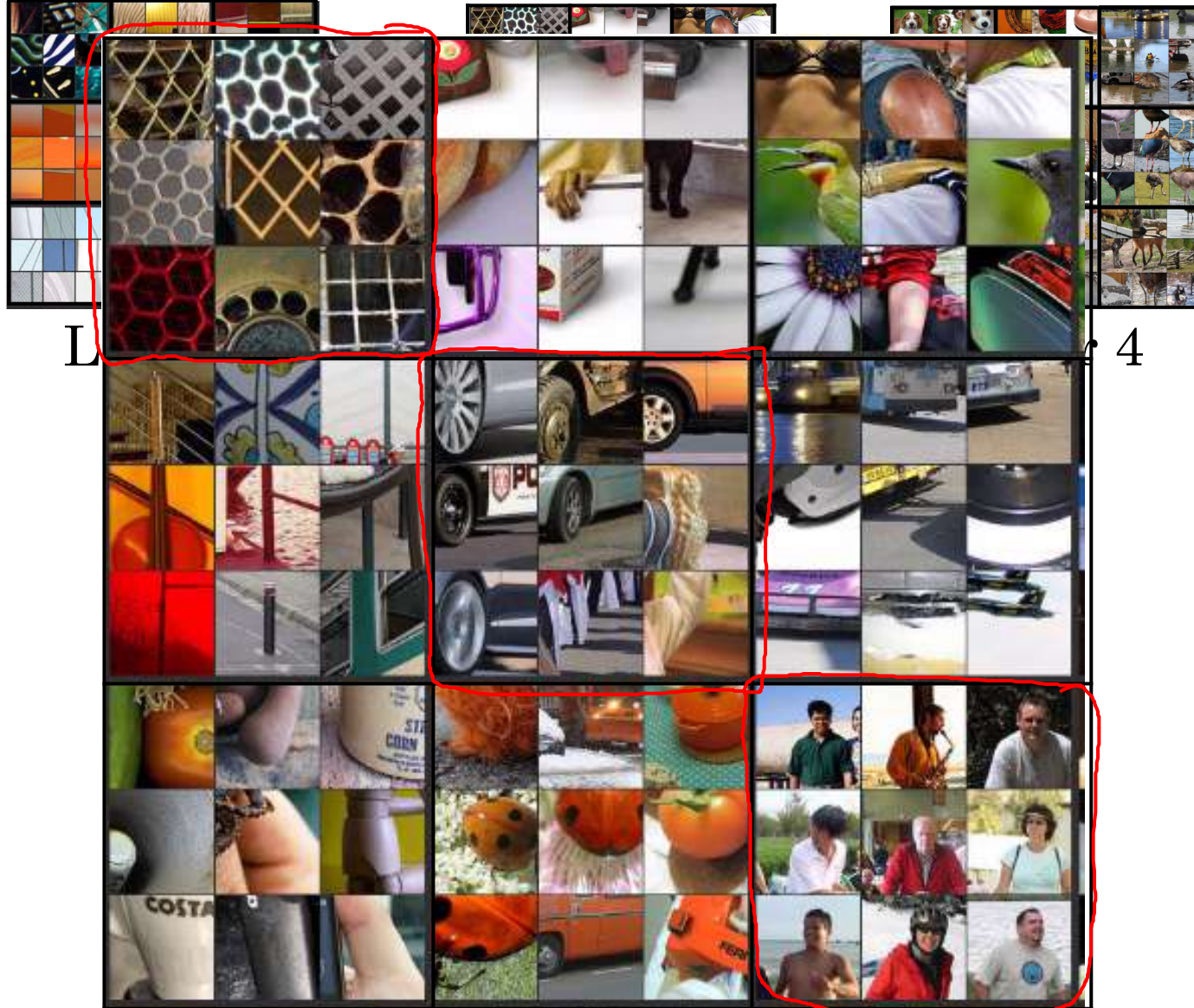
Layer 5



Visualizing deep layers: Layer 3

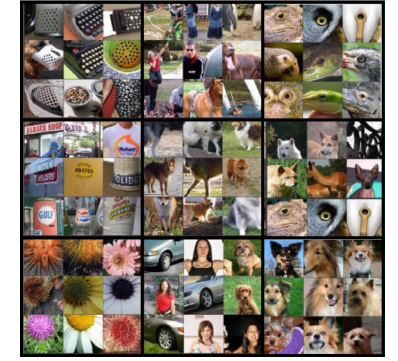


Layer 1



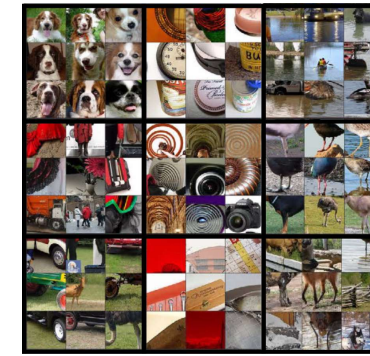
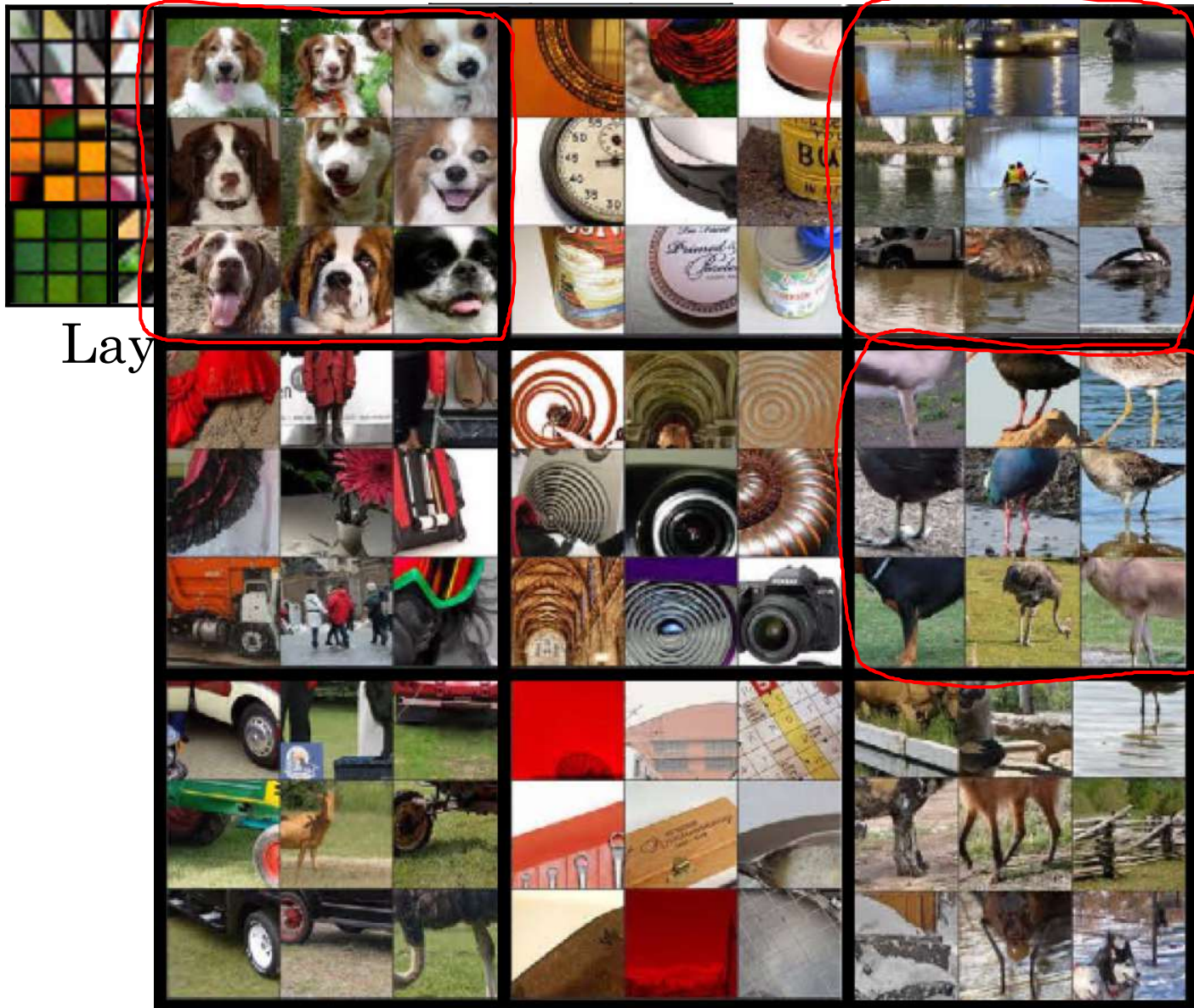
Layer 3

Layer 4

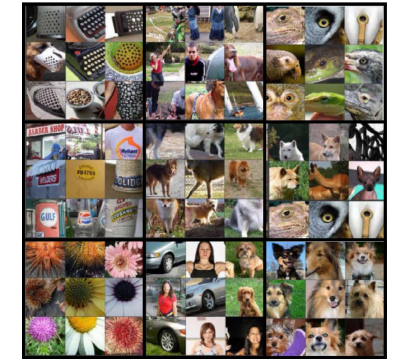


Layer 5

Visualizing deep layers: Layer 4

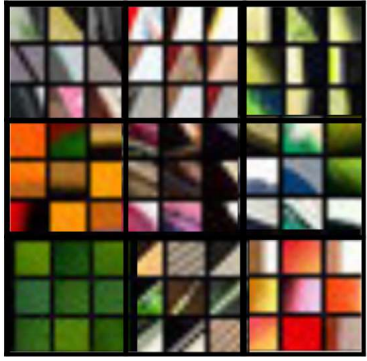


Layer 4

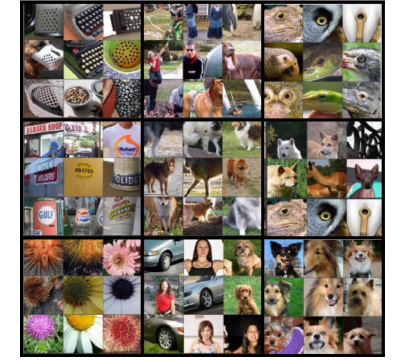
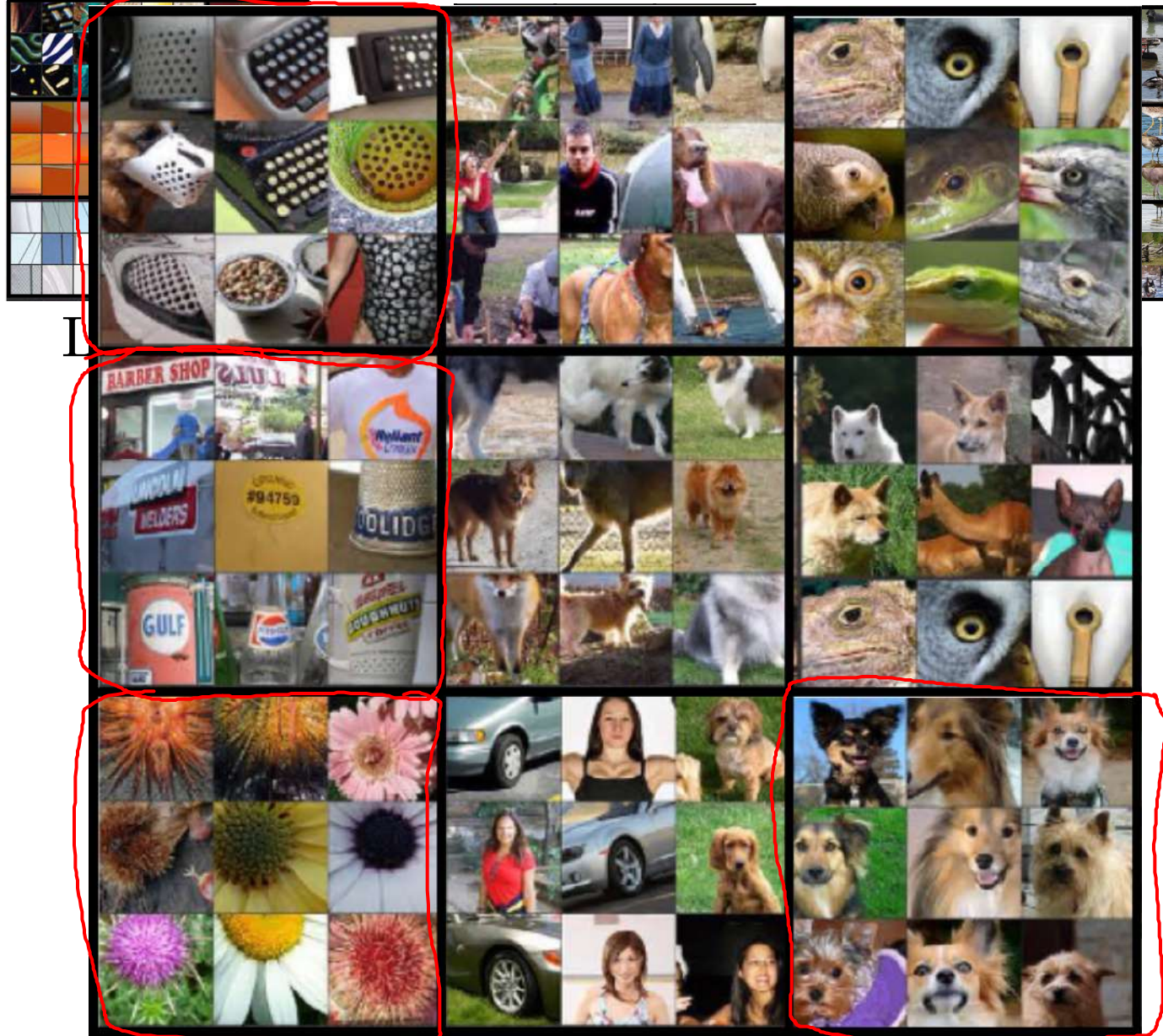


Layer 5

Visualizing deep layers: Layer 5



Layer 1



Layer 5



deeplearning.ai

Neural Style Transfer

Cost function

Neural style transfer cost function



Content C

Style S



Generated image G

$$\mathcal{J}(G) = \alpha \mathcal{J}_{\text{Content}}(C, G) + \beta \mathcal{J}_{\text{Style}}(S, G)$$

Find the generated image G

1. Initiate G randomly

G : $100 \times 100 \times 3$

↑
RGB

2. Use gradient descent to minimize $J(G)$

$$G := G - \frac{d}{dG} J(G)$$





deeplearning.ai

Neural Style Transfer

Content cost function

Content cost function

$$\underline{J(G)} = \alpha \underline{J_{content}(C, G)} + \beta J_{style}(S, G)$$

- Say you use hidden layer l to compute content cost.
- Use pre-trained ConvNet. (E.g., VGG network)
- Let $a^{[l](C)}$ and $a^{[l](G)}$ be the activation of layer l on the images
- If $a^{[l](C)}$ and $a^{[l](G)}$ are similar, both images have similar content

$$J_{content}(C, G) = \frac{1}{2} \left\| \underbrace{a^{[l](C)}}_{\text{activation of layer } l \text{ on } C} - \underbrace{a^{[l](G)}}_{\text{activation of layer } l \text{ on } G} \right\|^2$$

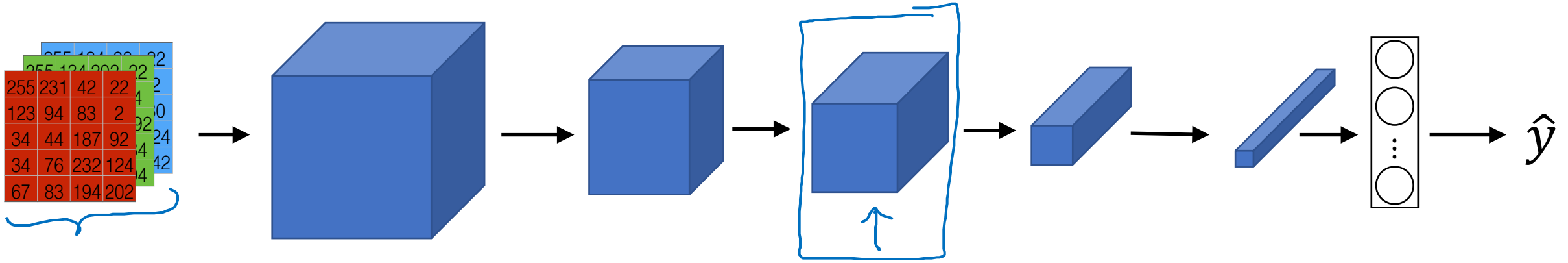


deeplearning.ai

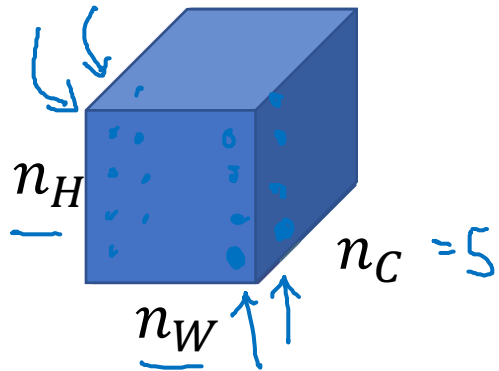
Neural Style Transfer

Style cost function

Meaning of the “style” of an image



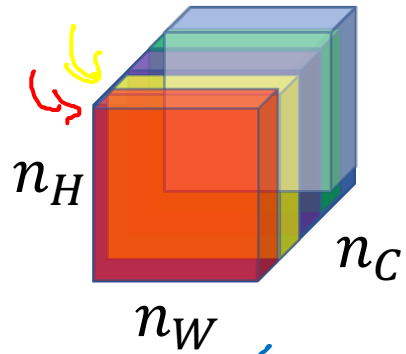
Say you are using layer l 's activation to measure “style.”
Define style as correlation between activations across channels.



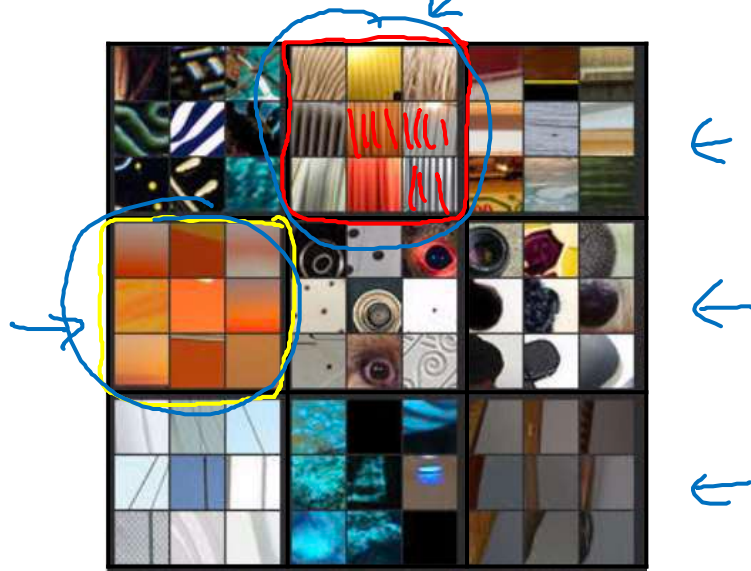
How correlated are the activations
across different channels?

Intuition about style of an image

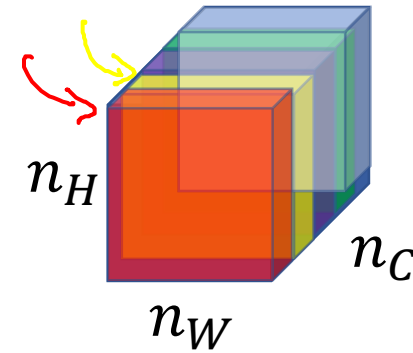
Style image



Correlated?
Uncorrelated



Generated Image



Style matrix

Let $a_{i,j,k}^{[l]}$ = activation at (i, j, k) . $\underline{G}^{[l]}$ is $\underline{n_c}^{[l]} \times \underline{n_c}^{[l]}$

$$\begin{aligned} \rightarrow \underline{G_{kk'}^{[l](S)}} &= \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l](S)} a_{ijk'}^{[l](S)} \\ \rightarrow \underline{G_{kk'}^{[l](G)}} &= \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l](G)} a_{ijk}^{[l](G)} \end{aligned}$$

$$\begin{aligned} & n_c \\ & G_{kk'}^{[l]} \\ & \uparrow \uparrow \\ & k = 1, \dots, n_c \end{aligned}$$

"Gram matrix"

$$\begin{aligned} \beta \uparrow J_{\text{style}}^{[l]}(S, G) &= \frac{1}{(\dots)} \left\| G^{[l](S)} - G^{[l](G)} \right\|_F^2 \\ &= \frac{1}{(2 n_H^{[l]} n_W^{[l]} n_c^{[l]})^2} \sum_k \sum_{k'} \left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)} \right)^2 \end{aligned}$$

Style cost function

$$\|G^{[l](S)} - G^{[l](G)}\|_F^2$$

$$J_{style}^{[l]}(S, G) = \frac{1}{\left(2n_H^{[l]}n_W^{[l]}n_C^{[l]}\right)^2} \sum_k \sum_{k'} (G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)})^2$$

$$J_{style}(S, G) = \sum_l \lambda_l J_{style}^{[l]}(S, G)$$

$$\min_G J(G) = \alpha J_{content}(G) + \beta J_{style}(S, G)$$

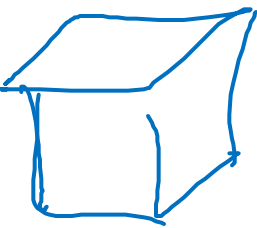


deeplearning.ai

Convolutional Networks in 1D or 3D

1D and 3D
generalizations of
models

Convolutions in 2D and 1D



$$14 \times 14 \times \underline{3} * 5 \times 5 \times \underline{3}$$

$$\rightarrow \underline{10 \times 10 \times 16}$$

$$\underline{10 \times 10 \times 16} * \underline{5 \times 5 \times 16}$$

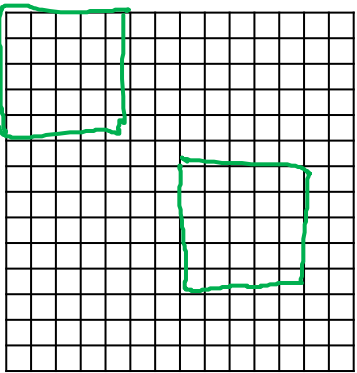
$$\rightarrow \underline{6 \times 6 \times 32}$$

$$14 \times \underline{1} * 5 \times \underline{1}$$

$$\rightarrow \underline{10 \times 16}$$

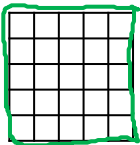
$$\underline{10 \times 16} * \underline{5 \times 16}$$

$$\rightarrow \underline{6 \times 32}$$

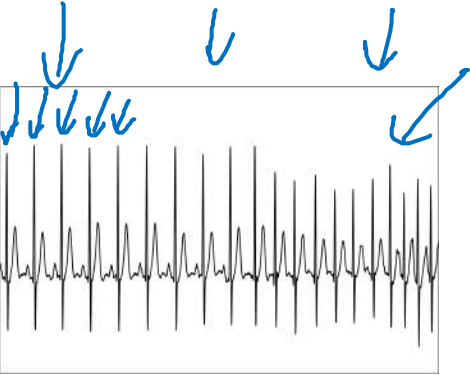


2D input image
14x14

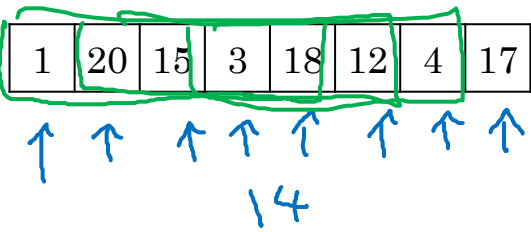
*



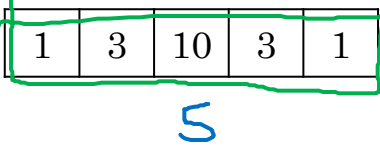
2D filter
5x5



*

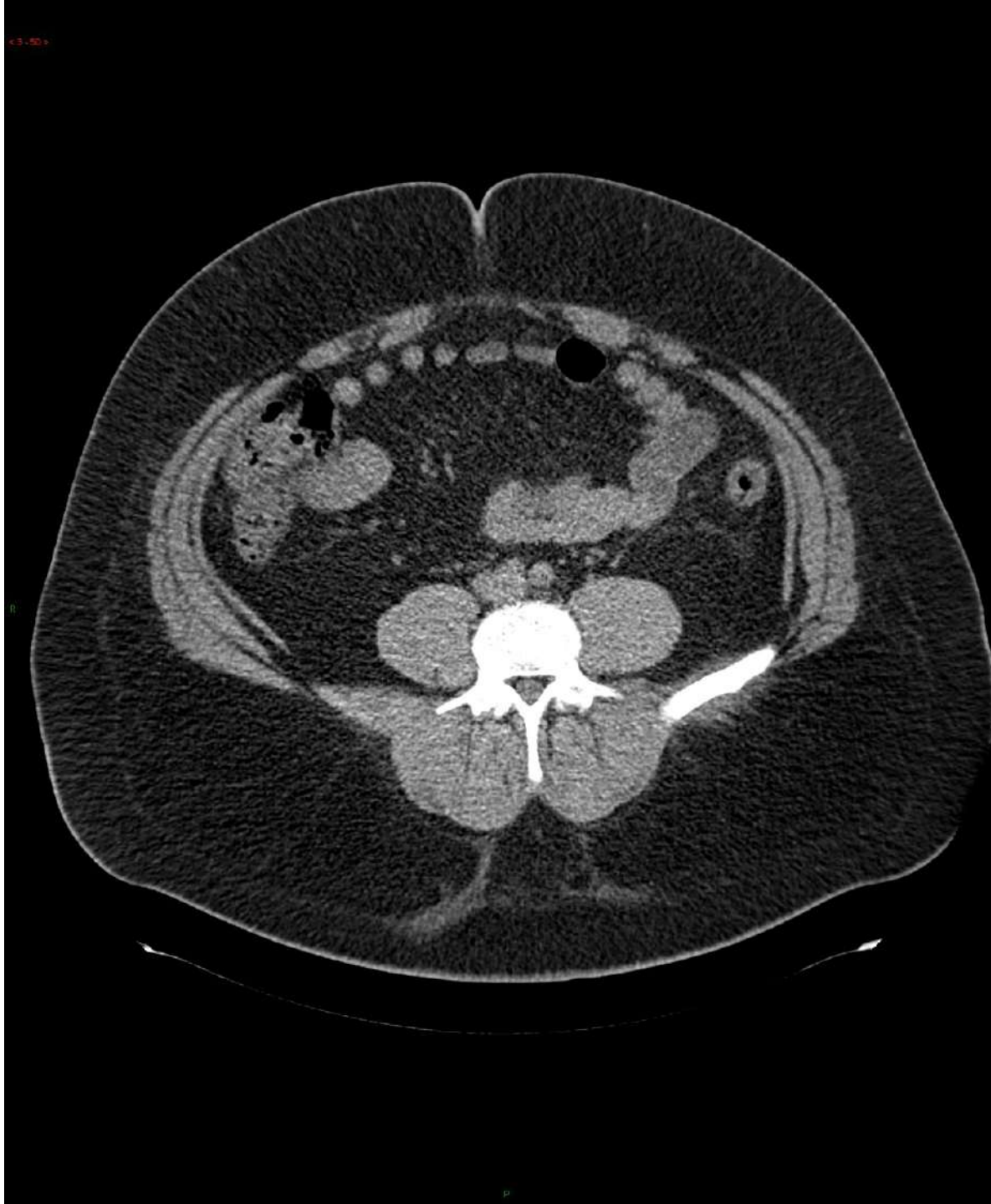


←



←

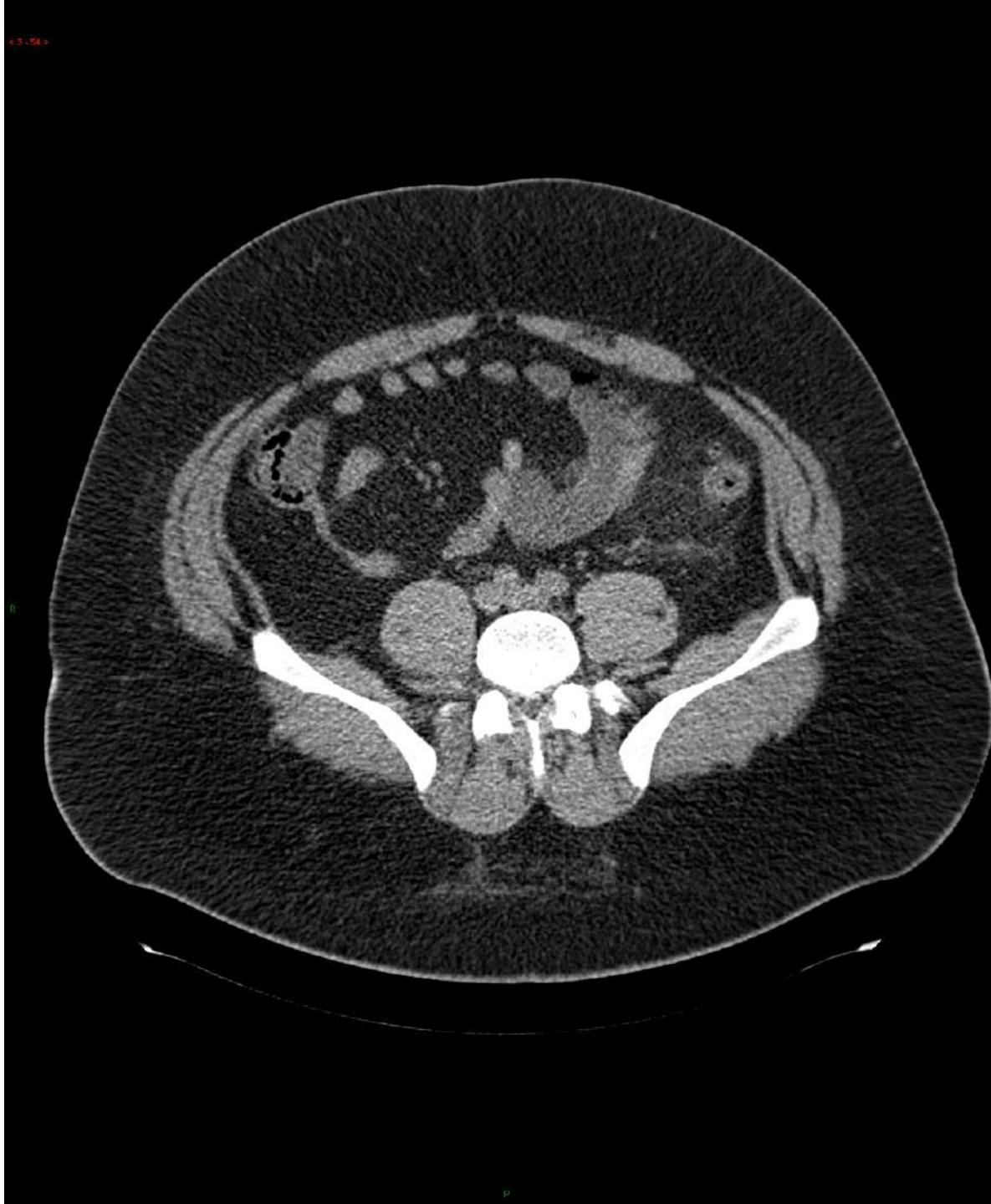
3D data



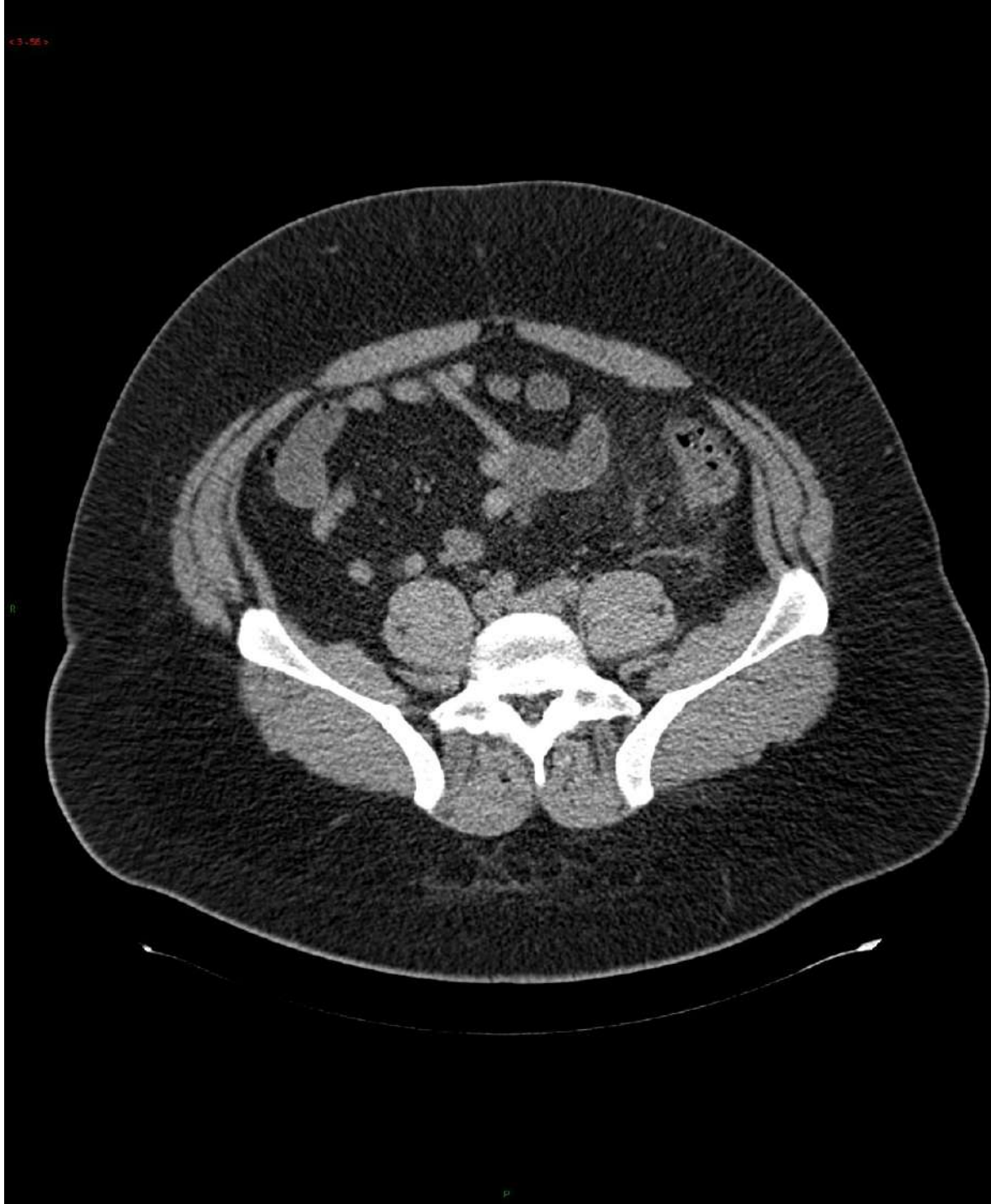
3D data



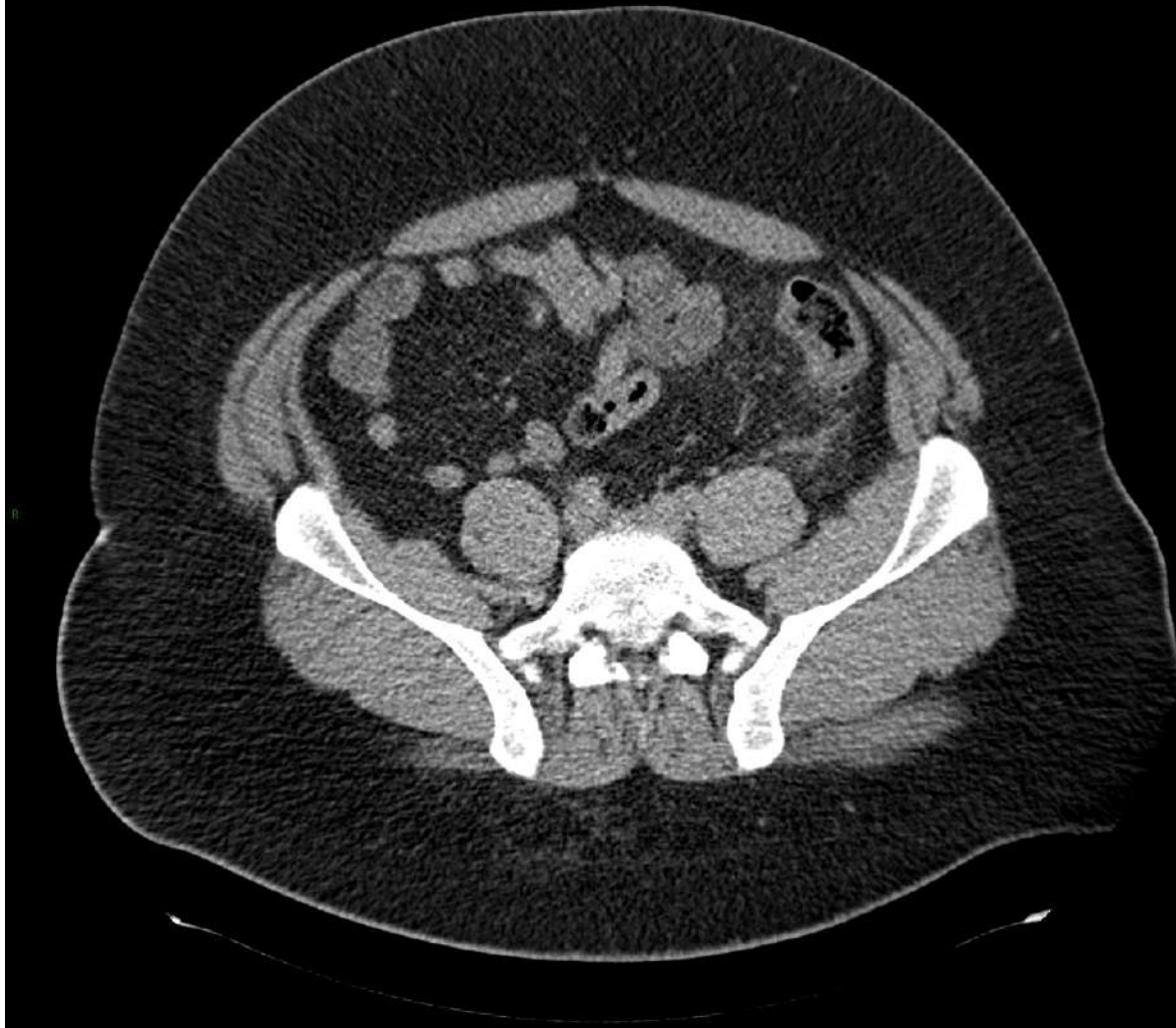
3D data



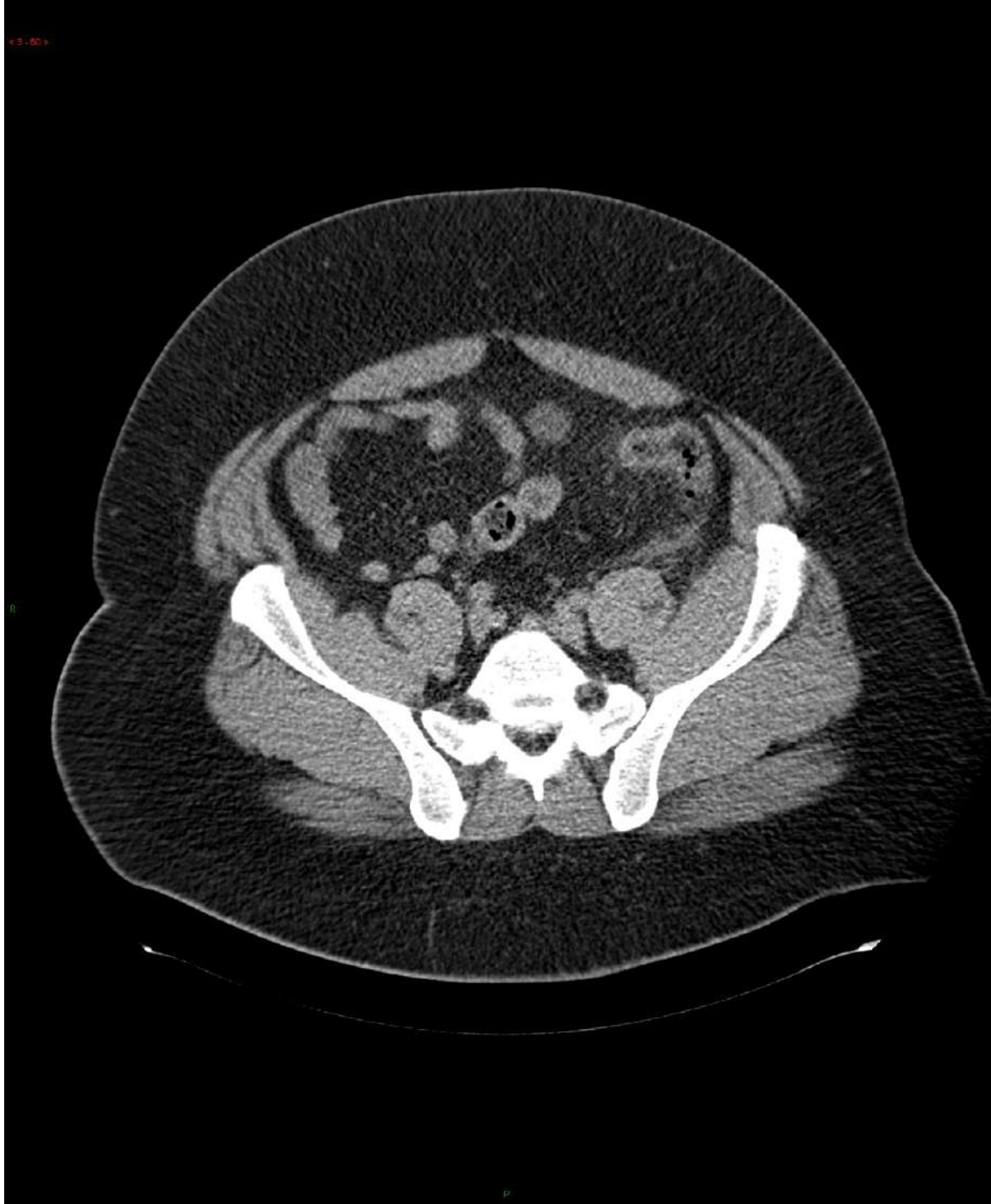
3D data



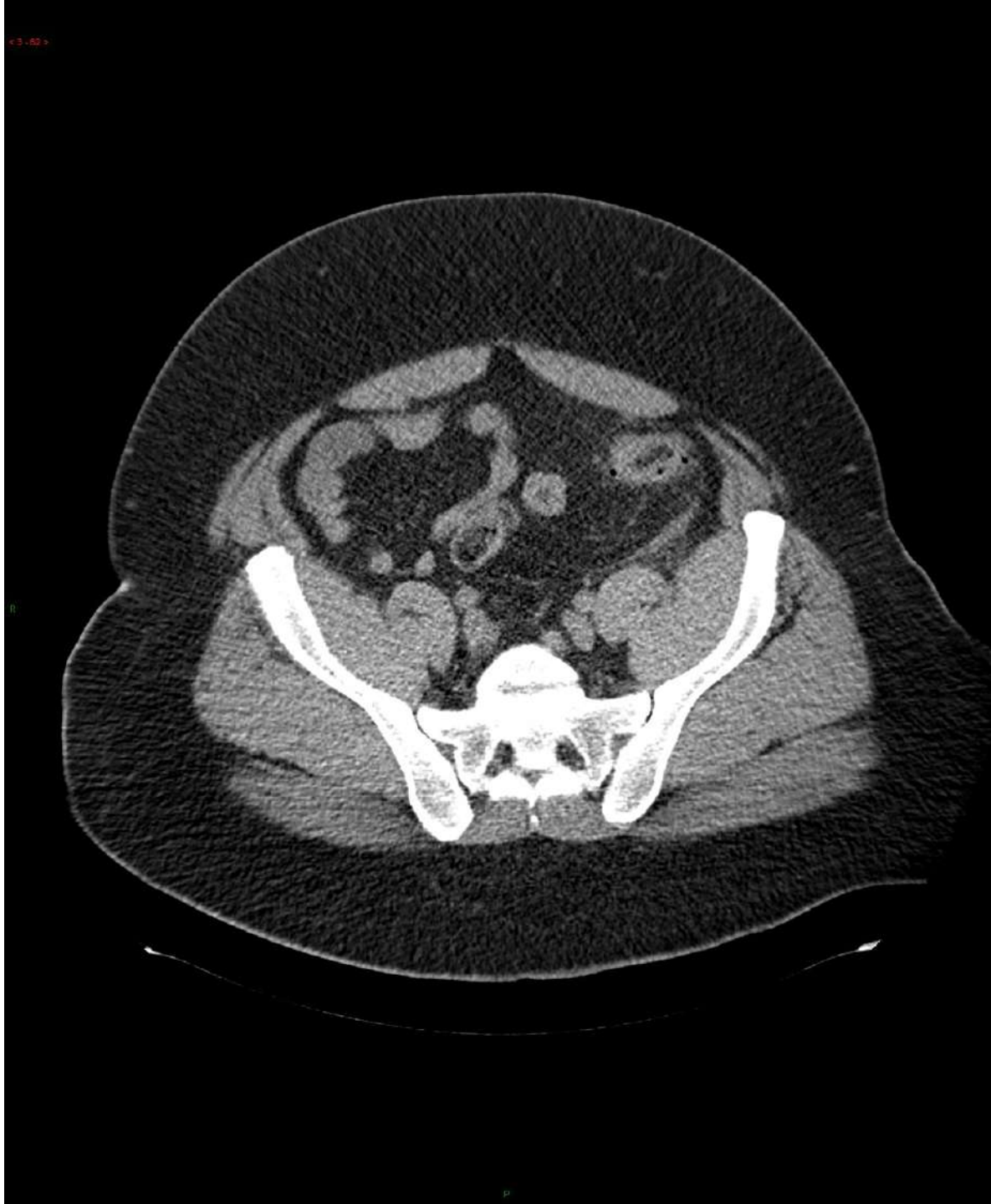
3D data



3D data



3D data



3D data



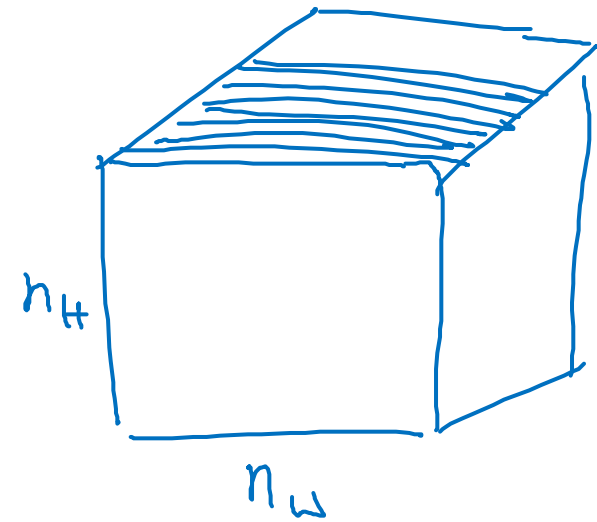
3D data



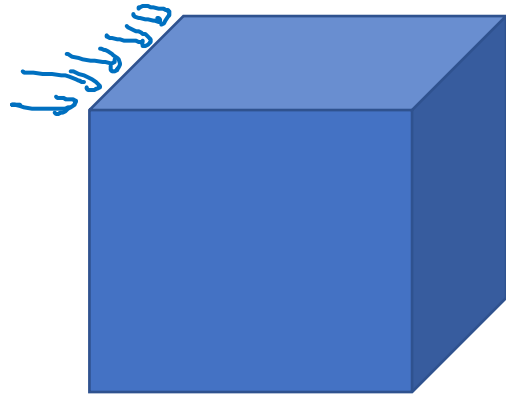
3D data



3D data



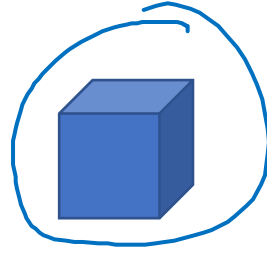
3D convolution



3D volume



*



3D filter

$$\begin{aligned}
 & \begin{array}{cccc} \downarrow & \downarrow & \downarrow & \downarrow^{n_c} \\ \underline{14 \times 14 \times 14} & \times & \underline{1} & \end{array} \\
 & \quad \times \quad \underline{5 \times 5 \times 5 \times 1} \quad 16 \text{ filters} \\
 & \rightarrow 10 \times 10 \times 10 \times \underline{16} \\
 & \quad \times \quad \underline{5 \times 5 \times 5 \times 16} \quad 32 \text{ filters} \\
 & \rightarrow 6 \times 6 \times 6 \times 32
 \end{aligned}$$